The OpenPulseHF Book

An amateur-radio HF software modem:
waveforms, physics, software, and operation

Release 0.16.0 · built 2026-07-18

Written for licensed amateur operators, electronic engineers and software developers. Every technical claim is traceable to a file, symbol, test or measured result in the repository at this release; measured figures carry their conditions, and anything unverified is labelled.

Contents

A complete technical account of an amateur-radio HF software modem — the waveforms, the physics, the software, and how to run it.

Covers v0.15.0. Every technical claim in this book is traceable to a file, symbol, test or measured result in the repository at that release. Where a number is quoted it carries its conditions; where something is unverified or planned it is labelled as such. See Provenance and honest limits at the end for how that was enforced, and for what it does *not* cover.

Abstract

OpenPulseHF is a plugin-based software modem for amateur high-frequency radio, written in Rust. It turns a computer's sound card and a single-sideband transceiver into a data link that adapts to the ionosphere: it carries ten modulation families from a 31-baud non-coherent sub-floor waveform up to multicarrier 64QAM, wraps them in forward error correction, and climbs or retreats through a fourteen-rung speed ladder according to what the channel is actually delivering.

HF is a hostile medium. A signal reflected off the ionosphere arrives as several rays at once, separated by milliseconds and drifting in frequency by a fraction of a hertz, so the received phase rotates unpredictably and the amplitude fades to nothing several times a minute. This book is largely the story of what that does to a modem and what can be done about it. The answers turn out to be specific and often counter-intuitive: an absolutely-phase-encoded waveform cannot hold a carrier reference through a fade at all, so the fading rungs are *differential*; a signal-to-noise estimator built on a residual counts the fade itself as noise and stops tracking the channel; a rate controller that trusts that estimate will sit at the bottom of its ladder while delivering every frame; and an improvement measured in uncoded bit-error rate can be worthless or negative once the error-correcting code is switched on.

The book is written for three readers at once. **Licensed amateur operators** will find what the system does, what it needs from a station, and copy-pasteable configurations for a dozen scenarios — without needing to read Rust. **Electronic engineers** will find the modulation, channel models, acquisition chain, equalisation, soft-decision demodulation and coding treated with their governing mathematics, each tied to the code that implements it and to the measurement that justifies it. **Software developers** will find the architecture: a strictly layered 41-crate workspace, a versioned plugin interface, the engine's pipeline seams, the daemon's

concurrency model, and a testing discipline shaped by specific, documented failures — including the project's recurring discovery that a test can pass while proving nothing.

A word on what this book is not. The adaptive HF-fade behaviour described throughout is validated against the Watterson channel *simulator*. It has not yet been proven on the air. That distinction is maintained everywhere it matters, because the project has already been caught once by a simulator that was itself wrong.

How to read this book

The four chapters are independent enough to enter at any point, but they are ordered so that each answers a question the previous one raises.

If you are	Start at	Then	You can skip
A licensed operator wanting it working	**Chapter 4** (scenarios 1–2)	Chapter 1 for what the modes mean	Chapter 2A's mathematics, Chapter 3 entirely
An operator wanting to understand the ladder	**Chapter 1** §1.4	Chapter 2A §2A.8 (fading physics)	Chapter 3, Chapter 2B
An RF or DSP engineer	**Chapter 2A**	Chapter 1 for the catalogue it implements	Chapter 4 setup detail
A software developer	**Chapter 3**	Chapter 1 for the domain, Chapter 4 to run it	Chapter 2A's derivations
Evaluating it against ARDOP/VARA/JS8Call	**Chapter 1** §1.5 and §1.10	Chapter 2A §2A.8 for why the choices differ	—
Reviewing security	**Chapter 2B**	§2B's "what is not protected" section especially	—

Conventions: file paths are repository-relative; a claim that rests on a test names the test; measured figures carry their channel model, SNR and FEC mode; and anything the project has *not* established is marked, not softened.

Chapter 1 — What OpenPulseHF Is: a deep dive and full showcase

OpenPulseHF is a software modem and data-network stack for amateur HF radio, written in Rust and licensed GPL-3.0-or-later. Plug it between a computer's sound card and an SSB transceiver and it turns the 2.7 kHz audio passband into an adaptive digital data link: it modulates data into audio, keys the rig's PTT, listens for the far station's reply, and steps its speed up and down a ladder of waveforms — from a 16-tone non-coherent mode that decodes below the noise floor, up to 64QAM OFDM at several thousand bits per second — driven by what the channel actually delivers. Everything runs at an 8 kHz audio sample rate around a nominal 1500 Hz carrier, so any radio that passes SSB audio can carry it.

It is more than a modem. Above the waveforms sits a full protocol stack: framing with CRC-16 and a family of forward-error-correction codes; segmentation and reassembly for objects up to 64 005 bytes; an ARQ/HARQ retransmission layer with soft-decision combining; an Ed25519-signed (optionally post-quantum) session handshake with a trust store; multi-hop relay with trust-weighted route scoring; QSY frequency-agility negotiation; JS8-compatible station discovery; a peer-to-peer file transfer protocol; and compatibility front-ends — an ARDOP-style TCP TNC and a KISS/AX.25 TNC — so existing software such as Winlink clients can drive it without knowing it exists. Version 0.15.0 spans 41 workspace crates: 24 core and protocol crates, 10 modulation plugins, 5 applications, plus PKI tooling and a dictionary trainer.

Two things define the project's character. First, every waveform is a plugin behind one trait, so the same engine, FEC, ARQ and session machinery serve BPSK31 and OFDM52-64QAM alike. Second, the engineering method is aggressively empirical: performance claims in this book trace to named tests in the repository, and the project's own documentation records not only what worked but what was built, measured and rejected. One honest caveat governs the whole chapter: the HF-fading behaviour

described here (the v0.13.0–v0.15.0 work) is validated against the Watterson channel *simulator*, not yet on the air. Where a number depends on that, the text says so.

1.1 The signal chain, end to end

The repository's own OSI mapping (`docs/osi-layer-map.md`) is the clearest orientation diagram:

```
      ┌──────────────────────────────────────────────────────────────┐
L7    │ APPLICATION   CLI · TUI · Panel (iced) · Testbench · Daemon  │
      │               ARDOP TNC · KISS TNC · B2F/Winlink · CMS gateway · FreeDV │
      ├──────────────────────────────────────────────────────────────┤
L6    │ PRESENTATION  Compression (LZ4 / gzip) · Signing + manifests  │
      │               PQ crypto (ML-DSA/ML-KEM) · Control-channel encryption  │
      ├──────────────────────────────────────────────────────────────┤
L5    │ SESSION       HPX state machine · Signed + PQ handshake · Trust store  │
      │               Adaptive session profiles · Secure session lifecycle  │
      ├──────────────────────────────────────────────────────────────┤
L4    │ TRANSPORT     ARQ/HARQ retransmission · SAR (segment/reassemble)  │
      │               ACK taxonomy · Rate adaptation (speed-level ladder)  │
      ├──────────────────────────────────────────────────────────────┤
L3    │ NETWORK       Peer cache · Multi-hop relay + route scoring · Query  │
      │               propagation · QSY freq-agility · Mesh · Digipeater · AX.25 │
      ├──────────────────────────────────────────────────────────────┤
L2    │ DATA LINK     Frame + CRC-16 · FEC (RS/Conv/LDPC/Turbo) + interleaver  │
      │               Modem engine · Modulation plugins · CSMA/DCD · AFC/EQ/GPU │
      ├──────────────────────────────────────────────────────────────┤
L1    │ PHYSICAL      Audio backend (CPAL/loopback) · PTT/CAT · SSB transceiver │
      │               + antenna · HF channel  (channel simulator for testing)  │
      └──────────────────────────────────────────────────────────────┘
```

A message travels the stack like this: the operator sends it from the panel (L7); it is optionally compressed and the session authenticated (L6/L5); segmented and queued for reliable, rate-adapted delivery (L4); addressed and, if needed, routed via relays (L3); framed with CRC and FEC and modulated by the selected waveform, after a CSMA clear-channel check (L2); converted to audio and keyed onto the transceiver via PTT (L1). The far station runs the stack in reverse.

Three implementation points matter to anyone touching the chain:

- **The engine is the hub.** `ModemEngine` (`crates/openpulse-modem/src/engine.rs`) owns plugin dispatch, FEC, AFC settle, DCD/CSMA, and the NDJSON event stream. Received audio reaches the demodulators through two entry families — the `receive*` calls and the daemon's streaming `accumulate_capture` path — which funnel through a single shared seam, `route_audio_stage(PipelineStage::InputCapture)`. RX front-end DSP (the receiver notch, AGC) lives at that seam so every path gets it by construction, and

`ModemEngine::notch_blocks_processed()` is a runtime tripwire that stays at zero if an enabled feature never runs on a path.

- **Hardware-free by default.** All tests run with `--no-default-features`, which swaps the CPAL audio backend for `LoopbackBackend`; `openpulse-channel` provides Watterson fading, Gilbert-Elliott burst, QRN/QRM/QSB and chirp impairments so the whole stack — including two-daemon end-to-end runs — is exercised without a radio.
- **The physical layer is the operator's rig.** `openpulse-radio` provides PTT backends (serial RTS/DTR, VOX, rigctld CAT, CM108, GPIO) and CAT control; the transceiver and antenna are external.

1.2 The waveform catalogue

Ten modulation plugin crates live under `plugins/`. Their declared mode strings, taken verbatim from each plugin's `PluginInfo::supported_modes`, total 78 (hand-counted from source). Nine of the ten plugins are registered in the CLI's runtime registry: `openpulse modes` prints 73 modes, because `Js8Plugin` — the `ModulationPlugin` implementation in `plugins/js8` — is registered nowhere. The JS8 waveform is instead driven by direct `js8_plugin::` calls from `openpulse-discovery` and the daemon (§1.6.5), so it is a waveform of the stack but not a selectable modem mode:

Plugin	Modes	Count
bpsk	BPSK31, BPSK63, BPSK100, BPSK250, BPSK250-RRC	5
qpsk	QPSK125, QPSK250, QPSK250-D, QPSK500, QPSK500-D, QPSK1000, QPSK1000-HF, QPSK1000-HF-RRC, QPSK500-RRC, QPSK1000-RRC, QPSK2000, QPSK2000-RRC, QPSK9600, QPSK9600-RRC	14
psk8	8PSK500, 8PSK1000, 8PSK1000-HF, 8PSK1000-HF-RRC, 8PSK500-RRC, 8PSK1000-RRC, 8PSK2000, 8PSK2000-RRC, 8PSK9600, 8PSK9600-RRC	10
64qam	64QAM500, 64QAM1000, 64QAM2000-RRC	3
fsk4	FSK4-ACK (the ACK control channel)	1
mfsk16	MFSK16, MFSK16-ACK	2
js8	JS8-SLOW, JS8-NORMAL, JS8-FAST, JS8-TURBO, JS8-ULTRA (declared; not in the CLI registry)	5
ofdm	OFDM16, OFDM52, OFDM52-8PSK, OFDM52-16QAM, OFDM52-32QAM, OFDM52-64QAM	6
scfdma	SCFDMA16, SCFDMA52, SCFDMA52-P2, SCFDMA52-LP, SCFDMA52-8PSK, SCFDMA52-16QAM, SCFDMA52-32QAM, SCFDMA52-64QAM, SCFDMA52-64QAM-P4, SCFDMA26-8PSK, SCFDMA26-16QAM, SCFDMA26-32QAM	12
pilot	PILOT-{QPSK,8PSK,16QAM,32APSK} × {500, 500-RRC, 1000, 1000-RRC, 2000-RRC}	20

1.2.1 Parameters of the principal modes

The README's per-mode table is the authoritative parameter reference; the excerpt below keeps its values verbatim. The occupied-bandwidth column is the README's practical estimate — note that the `ModulationPlugin::occupied_bandwidth_hz` API returns a

different, deliberately conservative quantity (rectangular main-lobe null-to-null = 2 × baud, used as the receiver-notch protection band), and the two must not be mixed.

Mode	Baud	Bits/ sym	Gross bps	Occ. BW (Hz)	Waveform
BPSK31	31.25	1	31	~50	Single-carrier
BPSK63	62.5	1	63	~70	Single-carrier
BPSK100	100	1	100	~110	Single-carrier
QPSK125	125	2	250	~140	Single-carrier
BPSK250	250	1	250	~275	Single-carrier (+RRC)
QPSK250	250	2	500	~275	Single-carrier
FSK4-ACK	100	2	200	~400	4-FSK, ACK channel only
QPSK500	500	2	1 000	~550	Single-carrier (+RRC)
8PSK500	500	3	1 500	~550	Single-carrier, Gray-coded
64QAM500	500	6	3 000	~550	Single-carrier
PILOT-QPSK500…PILOT-32APSK500	500	2–5	1 000–2 500	~550	Pilot-framed single-carrier
OFDM16	—	2	~889	~625	OFDM, 16 subcarriers, QPSK
SCFDMA16	—	2	~889	~625	SC-FDMA, 16 subcarriers
SCFDMA26-{8PSK,16QAM,32QAM}	—	3–5	~2 167–3 611	~1 000	SC-FDMA, 26 subcarriers
QPSK1000 / 8PSK1000 / 64QAM1000	1 000	2/3/6	2 000/3 000/6 000	~1 100	Single-carrier
OFDM52	—	2	~2 889	~2 031	OFDM, 52 subcarriers, QPSK
OFDM52-{8PSK,16QAM,32QAM,64QAM}	—	3–6	~4 333–8 667	~2 031	OFDM higher-order ladder
SCFDMA52 family	—	2–6	~2 889–8 667	~2 031	SC-FDMA, DFT-CE pilots
QPSK2000-RRC / 8PSK2000-RRC	2 000	2/3	4 000/6 000	~2 700	Single-carrier + RRC
64QAM2000-RRC	2 000	6	12 000	~2 700	Requires SNR ≥ 25 dB
QPSK9600-RRC / 8PSK9600-RRC	9 600	2/3	19 200/28 800	~13 000	**Deferred (post-1.0)**: VHF/UHF, needs ≥ 38.4 kHz Fs

Registered but deliberately outside every profile:

- **`QPSK2000` and `8PSK2000` (plain, non-RRC)** — superseded by their `-RRC` variants. The source comments are blunt: at 4 samples/symbol the plain crossfade pulse leaves 8PSK2000 ISI-limited at ~5–13 % raw BER *with perfect AFC* ("not viable for engine / on-air decode", `plugins/psk8/src/lib.rs`), and QPSK2000 at ~0.4 % residual BER plus an unreliable AFC settle (`plugins/qpsk/src/lib.rs`).
- **`SCFDMA52-P2`** — low-PAPR demonstrator: Zadoff-Chu-phase pilots cut envelope-CCDF PAPR by ~2.15 dB (8.85 → 6.70) at the same geometry and rate.
- **`SCFDMA52-LP`** — low-PAPR demonstrator, ~2 dB lower mean PAPR (11.9 → 9.7), but its single-tap flat-channel estimator makes it AWGN/flat/well-timed only; the README warns it "silently mis-decodes on selectivity/tilt/timing error".
- **Differential 8PSK (`8PSK500-D`)** — built, measured, **rejected**, and not registered (see §1.4.2).

1.2.2 MFSK16 — the sub-floor waveform

`MFSK16` (`plugins/mfsk16/src/lib.rs`) is the answer to "what happens when the fade beats every phase-coherent mode". It is a constant-envelope, non-coherent 16-tone GFSK waveform: 16 tones at 31.25 Hz spacing, 31.25 baud (exactly 256 samples/symbol at 8 kHz), 4 bits/symbol, 500 Hz occupied. A data frame is one fixed RS block — 255 wire bytes as 510 data tones plus three 7-symbol Costas sync blocks, 531 symbols ≈ 17.0 s. `MFSK16-ACK` compresses the return channel to 40 symbols ≈ 1.28 s. The waveform is self-acquiring — Costas correlation searched over timing × frequency, validated through a ±25 Hz tuning offset — so its `estimate_afc_hz` is `None`: it opts out of the engine's coherent AFC chain entirely. Because there is no carrier to track, there is no carrier to lose; measured on Watterson `moderate_f1` (simulator), real-sync 16-GFSK reaches the 0.5 decode crossing ~4.2 dB below coherent BPSK31, and on `poor_f1` it decodes where BPSK31 scores 0.00 at every tested SNR (`docs/dev/research/robust-narrowband-measurement.md`). On AWGN it is ~1 dB *worse* than BPSK31 — it is a fading lever, not a free lunch — and its constant envelope is a PAPR credit of 1.45 dB besides.

1.2.3 JS8 — a native waveform, not a bridge

`plugins/js8` implements the JS8-compatible 8-GFSK weak-signal waveform natively: 79 symbols, Costas 3×7 sync, LDPC(174,87), tables ported from the GPL-3.0 JS8Call sources and validated against the compiled original. Both transmit and receive are native — no JS8Call installation is required. The message layer decodes callsign/grid/compound/directed frames, Huffman varicode, and the full 262k-entry JSC codebook for free text. The weak-signal gate is a regression test: `gate_at_minus_18_db` in `plugins/js8/tests/snr_sweep.rs` requires ≥ 6/8 decodes at −18 dB SNR (2 500 Hz reference bandwidth, deterministic AWGN); the recorded measurement is 11/12 (~92 %) at −18 dB with the decode floor at −21 dB. This waveform is the carrier for the discovery and rendezvous subsystem (§1.6.5).

1.3 Forward error correction

`FecMode` (`crates/openpulse-core/src/fec.rs`) offers ten codecs. Which one runs is set per ladder rung by the session profile (§1.4) or explicitly by the caller via `transmit_with_fec_mode`/

`receive_with_fec_mode`; `Turbo`, `Concatenated`, `RsInterleaved` and `ShortRs` appear in no shipped profile and are caller-selectable options (`ShortRs` is the ACK/control-frame codec).

Mode	Code	Rate	Decisions	Note
`None`	—	1.00	—	raw transmit/receive (the default variant)
`Rs`	RS(255,223), t=16	0.875	hard	the ladder-wide workhorse
`RsInterleaved`	RS + stride interleaver	0.875	hard	burst dispersion — inert for single-block payloads (§1.9)
`Concatenated`	Conv(1/2, K=3) + RS outer	~0.44	hard	DVB-S architecture; ≈ 2.28× overhead
`ShortRs`	short-block RS, 8 ECC bytes, t=4	—	hard	≤ 247 B payload → `len + 8` bytes; a 5-byte FSK4-ACK becomes 13 bytes instead of 255
`RsStrong`	RS(255,191), t=32	0.749	hard	double correction capacity; 25 % vs 14 % ECC overhead
`SoftConcatenated`	soft-Viterbi K=7 + RS(255,223)	~0.44	**soft**	~5 dB over hard `Concatenated`; the OFDM rungs' code
`Ldpc`	rate-1/2 LDPC, k=1024 n=2048, min-sum BP	0.50	**soft**	CPU implementation; GPU path reserved for future work
`LdpcHighRate`	rate ≈ 8/9 LDPC (k=1024, n=1152), PEG	0.89	**soft**	the top-rung throughput code; useless with a hard-decision plugin
`Turbo`	rate-1/3 PCCC, Max-Log-MAP BCJR, 8 iter.	0.33	**soft**	best coding gain for blocks ≤ 256 bits

One negotiation subtlety is deliberate: `FecMode::strength()` ranks `LdpcHighRate` at 1 — *below* `Rs` at 2 — because rate 8/9 carries the least redundancy of any code, so a peer falls back to it only when no more protective mode is mutual.

The single air-interface change in v0.15.0 is **opportunistic free RS strengthening** (`fec.rs::free_rs_strengthening`):

```
pub fn free_rs_strengthening(fec: FecMode, encode_input_len: usize) -> FecMode {
    if fec == FecMode::Rs
        && rs_block_count(encode_input_len, BLOCK_TOTAL - FEC_ECC_LEN_STRONG)
            == rs_block_count(encode_input_len, BLOCK_TOTAL - FEC_ECC_LEN)
    {
        FecMode::RsStrong
    } else {
        fec
    }
}
```

RS blocks are always 255 bytes on the wire, so whenever t=32's extra parity does not spill into an extra block, the stronger code is literally free — and on the weak fading rungs it roughly doubles decode (BPSK31 at 3 dB on `moderate_f1`, simulator: 0.25 → 1.00). The guard exists because the naive version was tried: applied unconditionally, a 200-byte payload framed to one `Rs` block but two `RsStrong` blocks, doubling airtime and dropping `hpx_hf`'s AWGN goodput from 310 to 199 bps — straight through the CI goodput floor. Interop note from the release notes: a v0.15.0 station transmitting small frames on the weak rungs to a pre-v0.15.0

receiver can lose them, because the older receiver does not try the stronger code (the new receiver tries both and lets the CRC decide). Update both ends. Gates: `free_rs_strengthening` (core) and `free_rs_strengthening_ota` (modem).

1.4 The adaptive rate ladders

A `SessionProfile` (`crates/openpulse-core/src/profile.rs`) maps each `SpeedLevel` (SL1–SL20) to a mode string, a per-level FEC, and per-level SNR floor/ceiling. Twelve named profiles ship: `hpx500`, `hpx_modcod`, `hpx_pilot`, `hpx_pilot_rrc`, `hpx_pilot_fast`, `hpx_pilot_fast_rrc`, `hpx_hf`, `hpx_ofdm_hf`, `hpx_wideband`, `hpx_wideband_hd`, `hpx_narrowband`, `hpx_narrowband_hd`. The configured default is `[modem] profile = "hpx_hf"`.

1.4.1 `hpx_hf` — the primary HF ladder (SL1–SL14)

This table is read from `SessionProfile::hpx_hf`. The repository keeps its own copy of it in `docs/mode-fec-ladder.md`, and that copy is *gated*: `ladder_doc_matches_profile` (`crates/openpulse-core/tests/`) parses the doc table and asserts mode/FEC/floor/ceiling equality per level, so doc and code cannot silently drift.

SL	Mode	FEC	~Net bps	Floor (dB)	Ceiling (dB)	Note
1	MFSK16	Rs	~9	—	5	non-coherent sub-floor rung; reached via ChirpFallback
2	BPSK31	Rs	27	3	6	**entry level**
3	BPSK63	Rs	54	4	6.5	
4	BPSK100	Rs	87	4.5	7	breaks the 54 → 219 bps cliff
5	BPSK250	Rs	219	5	9	differentially decoded → fade-robust
6	QPSK250-D	Rs	437	7	11	differential QPSK, the #923 fix
7	OFDM52	SoftConcatenated	1 264	9	12	first multicarrier rung
8	OFDM52-8PSK	SoftConcatenated	1 895	10	14	
9	OFDM52-16QAM	SoftConcatenated	2 527	12	16	
10	OFDM52-32QAM	SoftConcatenated	3 159	14	18	
11	OFDM52-64QAM	SoftConcatenated	3 790	16	20	densest constellation at soft-concat FEC
12	OFDM52-16QAM	LdpcHighRate	5 141	18	21	above SL11, code rate is the only lever
13	OFDM52-32QAM	LdpcHighRate	6 426	19	22	
14	OFDM52-64QAM	LdpcHighRate	7 710	20	—	ladder top; admission gated

Structural rules, all from the profile source: every rung is coded — on a fade there is no such thing as a useful uncoded rung; ceilings are a uniform +2 dB hysteresis over the next rung's floor (`ceiling(L) = floor(L+1) + 2`); `initial_level` is SL2 with `nack_threshold = 3`; and admission to SL14 requires a prior SNR upgrade candidate (`ack_up_requires_snr_candidate_at = Some(SL14)`). Everything above SL6 is OFDM because the coherent single-

carrier modes that used to sit there decode ~0 % on Watterson `moderate_f1` at any SNR up to 40 dB (simulator; §1.9), while OFDM's cyclic prefix rides the delay spread and its per-subcarrier pilots track the fade.

The ladder-wide dead-rung tripwire is `hpx_hf_rungs_survive_fade` (modem tests), three assertions: `every_rung_decodes_on_moderate_f1` requires ≥ 0.25 of 12 frames at floor + 4 dB — excluding SL1 (MFSK16, ~17 s per frame, covered by its own sub-floor gates) and the three `LdpcHighRate` rungs, whose modes are already swept via their `SoftConcatenated` pairs, with the swept count pinned so the exclusions cannot quietly grow; `entry_rung_decodes_on_a_fade` requires ≥ 0.2 for SL2 *at* its own 3 dB floor; and `no_hpx_hf_rung_is_uncoded` asserts no level carries `FecMode::None`. Deliberately a weak bar — "a dead-rung tripwire, not a floor calibration" in its own words.

1.4.2 Why SL6 is differential — and why that stops at QPSK

Issue #923 is the defining measurement of the v0.13.0 release. Coherent QPSK250+Rs decodes **0 % on Watterson `moderate_f1` at every SNR up to 40 dB** (simulator): an absolutely phase-encoded waveform cannot hold a carrier reference through a 1 Hz Doppler fade, and one decision-directed cycle slip at a fade null ruins the frame tail. Two plausible fixes were built and measured to 0.00 — porting 8PSK's two-pass acquire-then-track loop, and routing to the pilot-framed waveform (PILOT-QPSK500+Rs likewise measures 0 % at 40 dB on that channel while decoding 100 % on AWGN down to 10 dB). The fix that works is differential encoding: `QPSK250-D` encodes each dibit as a phase *increment*, so the fade rotation cancels symbol-to-symbol and a slip costs one dibit instead of the tail — recovering the rung to ~0.65 at 20 dB for ~2 dB of AWGN floor. Differential requires FEC (the slipped dibit must be corrected) and has no soft-LLR path.

The obvious extension — differential 8PSK for the old SL9 — was built, measured, and **rejected**: `8PSK500-D` reaches only 0.050 at 20 dB / 0.125 at 40 dB on `moderate_f1`, because differential detection roughly doubles the effective noise and 8PSK's ±22.5° margin cannot absorb it (a ~4–6 dB AWGN penalty versus QPSK's ~2 dB).

The ordering the project distilled from this: robustness tracks phase margin — MFSK16 (non-coherent) > BPSK (±90°) > QPSK (±45°) > 8PSK (±22.5°).

1.4.3 The other ladders, briefly

Profile	Rungs	Character
`hpx_ofdm_hf`	SL5–SL10: `OFDM16` → `OFDM52` → `OFDM52-{8PSK,16QAM,32QAM,64QAM}`, all `SoftConcatenated`	all-multicarrier HF ladder; the uncoded entry rungs failed ~50 % of `moderate_f1` frames until coded
`hpx500`	SL2–SL6: `BPSK31/63/250`, `QPSK250/500`, no FEC	the original 500 Hz ladder; floors are "3 dB headroom above Eb/N0 for 1e-3 BER"
`hpx_modcod`	SL2–SL7: `BPSK250/QPSK250/QPSK500` interleaved with `Ldpc/Rs/None`	DVB-S2-style joint modulation × FEC steps
`hpx_pilot{,_rrc,_fast,_fast_rrc}`	SL2–SL5: `PILOT-{QPSK,8PSK,16QAM,32APSK}` at 500 or 1000 baud	shared per-symbol floors 6/12/17/23 dB (Es/N0-set, baud-independent)
`hpx_wideband`	SL8–SL11: `QPSK500` → `8PSK1000`	SL9+ exceed the 2 700 Hz HF channel width — FM/satellite/VHF-UHF only
`hpx_narrowband`	SL8–SL11 up to `8PSK2000-RRC`	12.5 kHz PMR/LMR at 8 kHz audio
`hpx_narrowband_hd`	SL8–SL9: `QPSK9600-RRC`, `8PSK9600-RRC`	requires a 48 kHz audio path
`hpx_wideband_hd`	SL9–SL15: SC-FDMA 26/52-carrier QAM up to `64QAM2000-RRC`	"Not suitable for HF ionospheric paths (Watterson fading breaks QAM coherence)"; the only profile with `nack_threshold = 2`

Interop safety across versions: `SessionProfile::fingerprint()` hashes only the (level → mode, level → FEC) pairs — deliberately excluding SNR floors and thresholds — so two stations whose ladders differ only in recalibrated floors still interoperate, while a genuine rung divergence is detected before a `recommended_level` can mean different things at the two ends.

1.4.4 How the ladder actually climbs: evidence beats the model

The rate controller's history contains the project's most instructive bug (#934). On a fading channel, any SNR estimator built from the raw decision residual folds the *multiplicative* fade into "noise": BPSK's fallback estimator read a flat ≈ −4 dB from 15 dB to 35 dB of true SNR — a constant across 20 dB of channel — and since `hpx_hf`'s SL2–SL5 are all BPSK, the controller was deciding on a constant. With the only climb path being `snr >= ceiling`, the ladder sat pinned on its entry rung at ~5 bps *while delivering 20 of 20 frames*.

The fix is two rules in `crates/openpulse-core/src/ota_rate.rs`:

1. **Climb on evidence**: `ACK_CLIMB_THRESHOLD = 3` consecutive clean decodes at the current rung promote it, even when the SNR estimate is useless.

2. **Never demote below a level that just decoded**: a decode is proof the rung works; demotion belongs on the failure path.

The maintainer's phrasing: "*A decode is an observation; the SNR is a model; the observation wins.*" The gate goes through the real controller, not the demodulator: `psk_ladder_climbs_off_the_entry_rung_on_a_fade` (linksim) drives `hpx_hf` through the receiver-led rate controller on `moderate_f1` at 20 dB and asserts delivery ratio > 0.9 *and* average level ≥ 3.0 — pre-fix it sat at ~1.5. Every earlier fade gate called the demodulator directly and so proved only that the rungs decode, which they did.

A related boundary is pinned deliberately: the ladder's SNR scales are **per-waveform-family**. Single-carrier PSK (post-#934) reads approximately true channel SNR; the OFDM/SC-FDMA estimators saturate near ~16 dB (ZF noise enhancement on faded subcarriers) and physically cannot report the 20–30 dB the dense rungs run at. So SL2–SL6 floors are true-SNR and SL7–SL14 floors are plugin-domain — two scales, one ladder, made safe by the evidence-based climb. The `snr_scale_boundary` test fails any change that "unifies" the estimators without re-deriving the floors in the same change.

1.5 What makes it different

Comparative claims here are limited to what the repository substantiates.

- **Against ARDOP**: OpenPulseHF is not a fork or reimplementation of ARDOP — it *speaks* ARDOP. The `openpulse-tnc` binary presents an ARDOP-compatible TCP command/data interface (Pat-style command set: `VERSION`, `MYID`, `LISTEN`, `CONNECT`, `GRIDSQUARE`, `ARQBW`, and more), so software written for ARDOP drives the OpenPulseHF engine, waveforms and ladders underneath. ARDOP's design is one of the project's documented research inputs (`docs/dev/research/ardop-research.md`).
- **Against VARA**: VARA is a closed-source product; the repository studies its architecture and ACK taxonomy as research (`docs/dev/research/vara-research.md`) and borrows ideas (the speed-level ladder, the ACK-driven rate machine) into an implementation that is GPL-3.0, inspectable, and gated by

named tests. No performance comparison against VARA exists in the repository, and none is claimed here.

- **Against JS8Call**: OpenPulseHF does not bridge to a running JS8Call — it implements the JS8-compatible waveform natively, TX and RX, validated against the original's tables (§1.2.3), and then uses it as a discovery-and-rendezvous substrate for its own higher-rate ARQ sessions (§1.6.5): stations find each other at JS8's −18 dB sensitivity class, then QSY to a working frequency and hand off to an HPX session.
- **The security layer is unusual for the class**: an Ed25519-signed session handshake with a trust store, an optional in-band post-quantum handshake (ML-DSA-44/ML-KEM-768), authenticated rate ACKs (ECDH-derived keyed MAC; forgery rejected under test), and signed transfer manifests. The project is precise about what this is: an identity *label*, not an access *gate* — see §1.7.
- **Everything is testable without a radio**: the Watterson/Gilbert-Elliott channel simulator, the two-daemon twin harness, and the linksim two-station ARQ simulator make the full stack — controller included — a CI subject.

1.6 The protocol surfaces

1.6.1 ARDOP-compatible TNC (`openpulse-tnc`)

ASCII command port plus a `u16` big-endian length-prefixed binary data port; defaults 127.0.0.1:8515/8516. Command set (from `crates/openpulse-ardop/src/command.rs`): `VERSION`, `MYID`, `LISTEN`, `CONNECT`, `DISCONNECT`, `ABORT`, `STATE`, `BUFFER`, `PTT`, `CLOSE`, `GRIDSQUARE`, `ARQBW`, `ARQTIMEOUT` (30–600 s), `CWID`, `SENDID`, `PING`, `PONG`, `FECSEND`, `FECRCV`, `CONNECT_MESH`, `WAVEFORM`. `VERSION` answers `VERSION 1.0-OpenPulseHF`. Adaptive ARQ over this bridge is opt-in (`[ardop] enable_adaptive_arq = false` by default — fixed-mode operation is the historical behaviour).

1.6.2 KISS/AX.25 TNC (`openpulse-kisstnc`)

Full KISS byte stuffing (FEND/FESC/TFEND/TFESC) and AX.25 UI frames (callsign+SSID addressing, Control 0x03, PID 0xF0) over TCP, default port 8100.

1.6.3 B2F / Winlink

`openpulse-b2f` implements the B2F session state machine (WL2K banner, FC/FS/Ff/Fq proposal frames, RFC-5322-like headers); `openpulse-b2f-driver` drives ISS/IRS sessions over the ARDOP TCP interface; `openpulse-gateway` connects directly to a Winlink CMS over TCP for send and receive.

Type C (LZHUF) compression is removed (PR #948) and must not be described as supported. The source states why (`crates/openpulse-b2f/src/compress.rs`): FBB historically uses the classic Okumura LZHUF, a *different bitstream* from the LHA LH5 implementation that used to live there, and no captured RMS Express/RMS Gateway Type C blob was available to validate against. An inbound Type C proposal is now answered `Reject` — an honest "cannot decode this" instead of a silent corrupt decode (`session.rs`). Type D (gzip) is the supported compression, and its decompressor is bomb-capped.

1.6.4 QSY frequency agility (`openpulse-qsy`)

Ed25519-signed `QSY_REQ`/`QSY_LIST`/`QSY_VOTE`/`QSY_ACK`/`QSY_REJECT` wire frames, a `QsySession` negotiation state machine, a `QsyScanner`, and a `BandplanPolicy` — the daemon validates candidate frequencies against the bandplan, can enforce maximum channel width and segment conventions, and (off by default) can auto-QSY away from persistent interference when the receiver notch reports it.

1.6.5 JS8 discovery and rendezvous (FF-15) — off by default

With `[discovery] enabled = true`, the daemon dwells on the standard JS8 calling frequencies (e.g. 14 078 000 Hz on 20 m), decodes heartbeats, and maintains a station table. The default mode is `"rx_only"`; `"beacon"` adds `@OPULSE`-hinted heartbeats, and `"full"` adds directed queries and the two-message Propose/Accept/Reject rendezvous: peers agree a working channel (carried as an *index* into the band's configured channel list, not a frequency), QSY on a scheduled slot, and hand off to a normal HPX `ConnectPeer` handshake — the post-QSY CONREQ is the authentication. Transmit is gated in `DiscoveryRuntime` on three conditions together: `mode` is `"beacon"` or `"full"`, a non-empty configured callsign, and the clock within `max_clock_skew_ms` of UTC

(default 2 000 ms) — otherwise the runtime degrades to RX-only. Automatic-control responsibility under §97.221 remains the operator's and is documented in `docs/regulatory.md`. Phases A–G are shipped; only Phase H — on-air validation — remains.

1.6.6 File transfer (FF-16) — off by default

`openpulse-filexfer` is a sans-I/O protocol crate (the `openpulse-b2f`/`openpulse-qsy` pattern): `SenderSession`/`ReceiverSession` state machines, offer/accept/reject policy, filename sanitisation, ≤ 48 KiB blocks over SAR with `BlockAck`-bitmap selective retransmit and `.partial` block-level resume; all I/O lives in the daemon glue. Transfers are verified against an inline signed `TransferManifest` (SHA-256, checked against the peer's handshake key); a verification failure appends `.unverified` to the written filename and the panel labels the transfer UNVERIFIED. The default config (`[file_transfer]`) is a conservative safety envelope: `enabled = false` (inbound offers rejected on air with `feature-disabled`), `require_verified_peer = true`, `auto_accept_max_bytes = 0` (always ask the operator), `max_file_bytes = 1 MiB`, and `burst_max_secs = 20.0` so a large transfer never holds PTT past a rig's watchdog. Phases A–E are shipped; on-air Phase F is deferred.

1.6.7 Mesh, repeater, relay

`openpulse-mesh` re-broadcasts beacons with TTL limits and `(session_id, nonce)` duplicate suppression; beacon payloads carry signed peer descriptors in which the peer ID *is* the Ed25519 verifying key. `openpulse-repeater` is a cross-band repeater — two modem engines, two rigs — and carries its own `StationIdTimer`. Relay route selection scores paths by the bottleneck (minimum) hop score = trust_weight × route_quality (Verified = 4, PskVerified = 3, Unknown = 2, Reduced = 1); direct routes are never penalised. The relay config's allow-list doc-comment is candid: the originator ID is not cryptographically authenticated at the relay, so the list is defence-in-depth, not strong authentication.

1.6.8 The daemon and its control protocol

`openpulse-daemon` aggregates modem, PTT, and all protocol services behind an NDJSON-over-TCP control port (default 127.0.0.1:9000) plus a WebSocket port (9001). The `ControlCommand` surface covers modem/rig control (`SetMode`, `SetFreq`, `SetCessb`, `SetNotch`, `SetAgc`,

PTT), QSY accept/reject, repeater enable/disable, spectrum subscription, sessions (`ConnectPeer` / `DisconnectPeer`), messaging, OTA rate control (`StartOtaSession`, `OtaLockLevel`, `OtaSetHysteresis`, …), the logbook, file transfer (`SendFile`, `AcceptFile`, …, `ListFiles`), and discovery (`EnableDiscovery`, `ListStations`, `RendezvousWith`).

1.7 The security surface

The stack secures two distinct links, at different layers (`docs/osi-layer-map.md`):

- **The on-air peer link** (L5–L6): a signed handshake — CONREQ/CONACK frames whose canonical-JSON bodies are Ed25519-signed and evaluated against a trust store — plus an optional in-band post-quantum handshake (`pq_handshake.rs`): ML-DSA-44 signatures (1 312-byte public keys, 2 420-byte signatures) and ML-KEM-768 key encapsulation, transported over SAR because the frames outgrow a single 255-byte payload. `SigningMode::Hybrid` signs with Ed25519 *and* ML-DSA-44 simultaneously. Downstream of the handshake: authenticated rate ACKs (an ECDH-derived keyed MAC; the `authenticated_ack_round_trips_and_forgery_is_rejected` test rejects forged and foreign-key ACKs) and signed transfer manifests.
- **The local control channel** (L6): `openpulse-linksec` wraps the daemon↔client link in a PSK-authenticated Noise channel (`Noise_NNpsk0`, X25519). Authentication is mandatory on any non-loopback bind; `require_auth` extends it to loopback. Secrets live in `openpulse-keystore`'s `FileKeystore`, encrypted at rest (Argon2id KDF → ChaCha20-Poly1305).

The project's own reframe from its 2026-07 handshake audit is worth carrying verbatim: the signed handshake is an identity *label*, not an access *gate*. It tells you verifiably who is talking; it does not, by itself, stop anyone from talking.

1.8 Tooling and observability

Tool	Framework	What it is
`openpulse` (CLI)	clap	Subcommands: `transmit`, `receive`, `devices`, `modes`, `mode-advisor`, `session-metrics`, `audit-bundle`, `identity`, `trust`, `diagnose`, `session`, `benchmark`, `monitor`, `adaptive`, `arq`, `config`, `broadcast`, `beacon`, `qsy`, `calibrate`, `daemon`; global flags for backend, PTT (`none/rts/dtr/vox/rigctld/cm108/gpio`), rig address, and `--max-power`
`openpulse-tui`	ratatui	Three panels: colour-coded HPX state; AFC/rate meters + DCD energy bar; scrollable transitions log
`openpulse-panel`	iced	Operator GUI over the daemon control port: band controls, spectrum/waterfall/ladder, tabs (`Info`, `Stats`, `Files`, `Config`, `Messages`, `Discovery`, `Log`); the earlier egui panel was retired 2026-07
`openpulse-testbench`	egui	Four-column live signal path — TX / noise / mixed / RX — with per-tap FFT spectrum + waterfall, 7 channel models, optional live capture
`openpulse-twinview`	egui	One window, two *real* daemons side by side: the left station's TX level is the A→B rate, the right's is B→A
`openpulse-linksim`	lib + CLI	Two-station bidirectional ARQ simulator measuring effective two-way transfer rate — real FSK4 ACK frames, turnaround, retransmission, and the *real* rate controller ("Rate control is not reimplemented here; the controller is the single source of truth")
`openpulse-testmatrix`	CLI	Automated mode × channel matrix runner; reports to `docs/test-reports/`
`pki-tooling`	axum + Postgres	Key management, trust store, trust-bundle signing web service

Observability is structural, not bolted on: `ModemEngine::subscribe()` yields a broadcast stream of `EngineEvent`s (NDJSON-ready) emitted at transmit, receive, ACK application, HPX transitions and secure-session boundaries; `openpulse monitor` streams them to stdout; every HPX transition is an audit record (`HpxTransition` with timestamp, states, event, and one of ten machine-readable `HpxReasonCode` values — `Success`, `Timeout`, `SignatureFailure`, `QualityDrop`, `RetriesExhausted`, `RecoveryTimeout`, `RelayPolicyFailed`, `RecoveryAttemptsExhausted`, `ManifestVerificationFailed`, `Unclassified`).

GPU acceleration (`openpulse-gpu`, wgpu) provides six WGSL compute kernels — BPSK modulate/demodulate, timing search, soft demodulation, RRC FIR, and a 256-point FFT — consumed by the BPSK, QPSK, 8PSK, 64QAM and SC-FDMA plugins behind a `gpu` feature, with automatic CPU fallback (every GPU call returns `Option<T>` so failure is detectable). Honest caveats: OFDM is *not* GPU-accelerated; the GPU LDPC path is reserved for future work (an `ldpc_bp.rs` module exists but `FecMode::Ldpc` uses the CPU codec); and the `gpu` feature is outside the routine test gates, so it receives less exercise.

1.9 Selected measurements, with their conditions

Every number below is recorded in the repository with its source; all fading figures are Watterson channel **simulator** results (`moderate_f1` = 1 Hz Doppler, 1.0 ms delay spread), not on-air measurements.

Measurement	Result	Conditions / source
Coherent QPSK250+Rs on fade	0 % at every SNR up to 40 dB	`moderate_f1`; `profile.rs`, issue #923
QPSK250-D recovery	0.00 → ~0.65 at 20 dB, for ~2 dB AWGN floor	`moderate_f1`; both variants decode 100 % by 4 dB on AWGN
Uncoded BPSK rungs at their floors	BPSK63 @4 dB 0.000; BPSK250 @5 dB 0.000	`moderate_f1`; the same rungs with `Rs`: BPSK63 @4 dB 0.833, BPSK250 @8 dB 1.00 (`profile.rs`)
Free RsStrong on the entry rung	BPSK31 @3 dB: 0.25 (`Rs`) → 1.00 (`RsStrong`)	`moderate_f1`; `fec.rs`
`RsInterleaved` on single-block payloads	identical to `Rs` (BPSK250 @5/8 dB: 0.17/0.58)	one RS block is position-agnostic; there is nothing to spread
OFDM52 vs 8PSK500 on fade	0.58/0.75/0.83 at 8/12/16 dB vs 0.00 at all three	`moderate_f1`; why SL7+ is OFDM
OFDM vs SC-FDMA, equal gross rate, 16QAM	0.88 vs 0.35 (`moderate_f1` @20 dB); 0.93 vs 0.03 (`moderate_f2`)	`tests/ofdm_scfdma_bakeoff.rs`
MFSK16 vs BPSK31, 0.5-decode crossing	~4.2 dB gain (`moderate_f1`, real sync); unbounded on `poor_f1` (BPSK31 never crosses); ~1 dB *worse* on AWGN	`docs/dev/research/robust-narrowband-measurement.md`
JS8 NORMAL weak-signal floor	11/12 at −18 dB; floor −21 dB	deterministic AWGN, 2 500 Hz ref BW; gate ≥ 6/8 at −18 dB
`LdpcHighRate` vs `SoftConcatenated` floor cost	+4…+8 dB for 2.03× rate, per SC-FDMA mode	AWGN, 62 B payload, 90 % frame-success; why code rate is the *last* lever
CE-SSB average-power gain	+1.6/+2.7/+3.8 dB in channel-sim (OFDM52 at 2.5/2.0/1.5 × rms), zero BER cost; **+1.18 dB confirmed on-air**	FT-991A, 2 m, 20 W via attenuator; software ACPR + SDR spectral-mask check
Goodput floors (regression gates)	`hpx_hf` AWGN 20 dB ≥ 250 bps (baseline ~397); `hpx_ofdm_hf` AWGN ≥ 600 (~919); `hpx_ofdm_hf` `moderate_f1` 25 dB ≥ 280 (~414)	linksim, 200 B frames, 40 frames, seeded
Benchmark harness gate	100 % pass, mean_transitions ≤ 20	`benchmark_integration`; CLI: `benchmark run` + `jq` check

CE-SSB deserves its scope stated exactly, because the project's own `docs/features.md` once got it wrong: `ModemEngine::cessb_benefits` returns `true` for **`OFDM16` and `OFDM52` only** — the QPSK-subcarrier OFDM modes. All SC-FDMA is excluded (it is single-carrier FDM, low-PAPR by construction: CE-SSB recovers only ~1/3 of OFDM's gain while its EVM alone injects ~0.5 % raw BER, collapsing SCFDMA52-{32,64}QAM decode 5/30 vs 30/30 through AWGN 35 dB), and every OFDM constellation at 8PSK or denser is excluded by measured decode collapse. The unifying principle from the source: "CE-SSB trades in-band EVM for average-power gain, and that trade only wins where the

envelope is high-PAPR *and* the decision margins are loose." The config default `cessb_enabled = true` is therefore a no-op outside OFDM16/OFDM52.

1.10 What this does not do yet

An honest inventory, as of v0.15.0.

Not validated on the air. The entire v0.13.0–v0.15.0 HF-fade arc — the `hpx_hf` ladder's differential SL6, the OFDM upper rungs, the evidence-based climb, MFSK16's sub-floor gains — is validated against the Watterson simulator only. The draft 1.0 criteria (`docs/dev/project/release-1.0-criteria.md`, itself marked "Draft for review — nothing here is agreed yet") make the point sharply: v0.14.1 already caught the simulator misleading the project once, and "A 1.0 that ships fade behaviour no one has heard on a radio is making a claim it has not earned." The 1.0 gate demands a real two-station HF QSO on `hpx_hf`, the ladder observed climbing and demoting on a real fading channel, and an end-to-end Winlink message over RF. Requirement coverage stands at 118 of 141 covered, 16 gaps, 7 planned for 1.x.

Off by default (opt-in), from `crates/openpulse-config/src/lib.rs`:

Feature	Key	Default
JS8 discovery	`[discovery] enabled`	`false` (and `mode = "rx_only"` — no TX)
File transfer	`[file_transfer] enabled`	`false`
Receiver-led OTA rate stepping	`[modem] ota_enabled`	`false`
Multi-mode receive monitor	`[monitor] enabled`	`false`
Receiver notch / AGC	`[modem] notch_enabled` / `agc_enabled`	`false` / `false`
ADIF logbook	`[logbook] enabled`	`false`
Session compression	`[compression] enabled`	`false`
Cross-band repeater	`[repeater] enabled`	`false`
ARDOP adaptive ARQ	`[ardop] enable_adaptive_arq`	`false`
Auto-QSY on interference	`[qsy] auto_qsy_on_interference`	`false`
Control-channel auth on loopback	`[control_security] require_auth`	`false` (mandatory on non-loopback regardless)

Removed, deferred, or reserved:

- **Winlink Type C (LZHUF) is removed** (PR #948); inbound Type C proposals are answered `Reject`. Restoring it requires a captured genuine RMS Type C blob to validate against.

- **JS8 discovery Phase H** (on-air) and **file transfer Phase F** (on-air) — the only remaining phases of FF-15/FF-16, both hardware/on-air-gated.
- **On-air regulatory validation** (Phase 5.5-reg) — deferred, no target date.
- **`QPSK9600-RRC` / `8PSK9600-RRC`** — deferred post-1.0 (VHF/UHF; need ≥ 38.4 kHz sample rate).
- **Wide-channel VHF/UHF (REQ-BW-01..07)** — an explicit 1.x non-goal.
- **Relay envelope authentication at intermediate hops** — blocked on a key-distribution design decision; 1.0 requires it be documented, not solved. The relay allow-list is defence-in-depth only.
- **GPU LDPC** — reserved for future work; `[discovery] group`, `[radio.rig_a]`, multi-rig `RigConfig` fields, and `[mesh] relay_policy` are accepted-but-unwired config, each flagged as such in the config source.
- **Proprietary-protocol compatibility (REQ-PERF-05/06)** — out of scope pending legal review.

Process status. The maintainer-supplied figure for the suite is 2 146 passing tests under `cargo test --workspace --no-default-features` (not independently re-run for this chapter). The quality gates — clippy with warnings-as-errors, the benchmark harness, the goodput floors, the doc-ladder equality test — exist and are run locally; the GitHub Actions workflow automation is disabled by the maintainer's deliberate choice, so passing gates are a local discipline rather than a badge.

1.11 A note on method

The repository's distinctive asset is not any single waveform but its recorded discipline, and it is why this book can cite so precisely. Docs tables are gated against code (`ladder_doc_matches_profile`); dead rungs trip a test (`hpx_hf_rungs_survive_fade`); fade fixes are counterweighted by clean-channel goodput floors; and the project's hard-won rules are written down next to the regressions that taught them: *delete the mechanism — if the number doesn't move, it was never the mechanism* (three accepted explanations falsified in a row by ablation); *a modem that fails at every SNR has a bug, not a limitation* (a flat 2–7 % decode rate from 8 to 32 dB sat

recorded as "by design" for two releases until a noiseless two-ray test exposed a channel-estimator bug); *an uncoded-BER win is not a win*; *test what an LLR means, not just its sign* (bits promising `|L| ≈ 12` confidence were wrong 71× more often than promised, invisible to every frame-success metric). Where the following chapters state a behaviour, the expectation this chapter sets is that a named test enforces it — and where none does, the text will say so.

Chapter 2, Part A — The physical layer: mathematics, physics, electronics, DSP and audio

This chapter explains what OpenPulseHF actually does to a stream of audio samples — the mathematics, the physics that forced each choice, and the file that implements it. It is written for three readers at once. If you are an operator, each section opens with what the mechanism means on the air. If you are an engineer, the governing equations and the measured numbers follow, each traceable to a source file or a named test. If you are a developer, every symbol named here exists in the tree and can be grepped.

One caveat governs everything that follows and is repeated where it matters: **all fading-performance figures in this chapter were measured against the Watterson channel simulator** (`crates/openpulse-channel`), not on air. The v0.13.0–v0.15.0 HF-fade work is simulation-validated; on-air validation is a separately tracked, deferred phase.

2A.1 The audio path — sample rate, bandwidth, analytic signals

2A.1.1 Why 8 kHz

Everything in the modem runs at a sample rate of 8000 Hz, mono. That is the default in `crates/openpulse-core/src/audio.rs`:

```
impl Default for AudioConfig {
    fn default() -> Self {
        Self { sample_rate: 8000, channels: 1, buffer_size: None }
    }
}
```

For an operator the important distinction is that **sample rate is not channel bandwidth**. An 8 kHz sample rate gives a usable passband up to the Nyquist limit of 4 kHz; an HF SSB channel is

roughly 300–2700 Hz, so 8 kHz covers it with margin. Two independent constraints decide whether a mode can run at this rate (`docs/mode-fec-ladder.md` §8):

1. **Occupied bandwidth below the 4 kHz Nyquist limit.** Every HF mode in the ladder occupies ≤ 2700 Hz; the widest multicarrier mode, SCFDMA52, tops out near 2.5 kHz.
2. **Enough samples per symbol.** A single-carrier demodulator needs roughly four samples per symbol for clean timing recovery, i.e. `Fs ≥ ~4 × baud`. At 8 kHz that caps the symbol rate near 2000 baud — which is exactly why `QPSK2000-RRC` is the fastest single-carrier HF mode.

The 9600-baud wide modes show both constraints biting at once: 9600 baud × (1 + 0.35 RRC rolloff) ≈ 13 kHz occupied — over three times Nyquist — and at 8 kHz you get 0.83 samples per symbol, which cannot be demodulated at all. Those modes stay registered (for a future higher-rate transport) and the loopback and test-matrix runners skip them with a stated reason rather than dropping them silently (`docs/mode-fec-ladder.md` §8).

Consumer soundcards clock 48 kHz (or 44.1 kHz), not 8 kHz; ALSA's `plug` layer resamples 8 ↔ 48 kHz and cpal opens the device at 8 kHz. A chirp probe confirmed the resampler is flat well past 3 kHz (`docs/dev/virtual-loopback.md`).

2A.1.2 The analytic signal — three implementations, three purposes

Several parts of the system need the complex (analytic) form of a real signal, `a(t) = s(t) + j·ŝ(t)` where `ŝ` is the Hilbert transform. Three implementations exist because the accuracy/latency trade differs per use:

- **FIR Hilbert (production I/Q fallback)** — `hilbert_iq(real, fc, fs)` in `crates/openpulse-core/src/iq.rs`: a 63-tap Hann-windowed FIR with a group delay of 31 samples. Its doc comment warns that only the middle of the output is free of edge artefacts; the first and last ~31 samples carry window roll-on/off errors. BPSK and QPSK override `ModulationPlugin::modulate_iq` and bypass it entirely.
- **FFT Hilbert (channel simulator)** — `analytic_signal(planner, x)` in `crates/openpulse-channel/src/fading.rs`. The textbook

frequency-domain method: forward FFT, double the positive-frequency bins, zero the negative half, inverse FFT, scale by `1/n`. The test `analytic_signal_of_a_cosine_has_constant_envelope_and_recovers_the_real_part` asserts, over interior samples, `|Re{analytic} − input| < 1e-2` and `||analytic| − 1| < 5e-2`.

- **Quadrature companion (DSP crate)** — `openpulse_dsp::acquisition::quadrature(x)`: the imaginary part alone, used to build carrier-phase-insensitive matched filters (§2A.5.2).

2A.1.3 Spectrum and waterfall taps

`crates/openpulse-channel/src/dsp.rs` fixes `FFT_SIZE = 1024`, `FREQ_BINS = 512`, `WATERFALL_ROWS = 200`, a Hann window power-normalised by `sqrt(2/N)` (since `Σw² = N/2` for Hann), and returns single-sided dBFS in roughly [−120, 0].
`PowerSpectrum::compute_welch` averages up to `max_segments` half-overlapped Hann segments spread across the burst — deliberately *bounded* averaging so a finite-sample variance remains and the trace varies naturally from frame to frame. The engine-side tap is `ModemEngine::last_audio()`, bounded to `SPECTRUM_TAP_MAX = 16384` samples (`crates/openpulse-modem/src/engine.rs`).

2A.2 HF propagation and the channel models

An HF skywave path is not a wire with hiss on it. The ionosphere delivers the signal over two (or more) refracted rays with different delays, and each ray's amplitude and phase wander as the reflecting layer moves — Doppler spread. The result is *multiplicative* fading: the channel multiplies the signal by a time-varying complex gain `h(t)`, on top of which additive noise and interference arrive. `crates/openpulse-channel` implements ten model configurations behind one trait:

```rust
pub trait ChannelModel: Send {
    fn apply(&mut self, input: &[f32]) -> Vec<f32>;
    fn generate_noise(&mut self, length: usize) -> Vec<f32>;
}
```

`generate_noise` exists for the testbench's standalone noise tap. Only AWGN and QRN return an actual noise process from it; the multiplicative models (`Watterson`, `FlatFading`, `Qsb`) return silence because fading is not independent additive noise, and `Qrm`, `Chirp`

and `Sro` return silence too — their impairment is a tone, a sweep or a resample, none of which is a noise tap (`crates/openpulse-channel/src/lib.rs`, `watterson.rs`). The `ChannelModelConfig` variants are `Awgn`, `GilbertElliott`, `Watterson`, `Qrn`, `Qrm`, `Qsb`, `Chirp`, `Composite`, `Sro`, and `FlatFading`, built by `build_channel(config, seed)`.

2A.2.1 AWGN

`awgn.rs` keys the noise standard deviation to the RMS of each input block:

```
fn noise_sigma(&self, signal_rms: f32) -> f32 {
    signal_rms / 10f32.powf(self.config.snr_db / 20.0)
}
```

so the labelled SNR is the per-block signal-to-noise ratio regardless of drive level.

2A.2.2 Watterson — the two-ray ionospheric model

`watterson.rs` is the workhorse behind every fading claim in this book: two delayed Rayleigh-faded rays, each with an independent complex Gaussian fading envelope shaped by a Gaussian Doppler-spread filter — the standard ITU-R style HF channel structure. The standard profiles (`crates/openpulse-channel/src/lib.rs`):

Constructor	Doppler spread	Delay spread	Default SNR
`good_f1`	0.1 Hz	0.5 ms	20 dB
`good_f2`	0.5 Hz	1.0 ms	15 dB
`moderate_f1`	1.0 Hz	1.0 ms	10 dB
`moderate_f2`	1.0 Hz	2.0 ms	10 dB
`poor_f1`	2.0 Hz	2.0 ms	5 dB
`poor_f2`	2.0 Hz	5.0 ms	3 dB
`extreme`	10.0 Hz	10.0 ms	0 dB

`moderate_f1` — 1 Hz Doppler, 1 ms delay — is described in the repo as a routine moderate HF channel and is the reference channel for the fade-aware ladder (`crates/openpulse-core/src/profile.rs`).

The application loop (`watterson.rs`):

```
let ray0 = analytic[i] * env0[i] * ray_scale;
let ray1 = if i >= delay_samples { analytic[i - delay_samples] * env1[i] * ray_scale }
           else { Complex32::new(0.0, 0.0) };
out[i] = (ray0 + ray1).re + noise_sigma * self.rng.sample::<f32, _>(StandardNormal);
```

Two normalisations here are load-bearing, and each one fixed a real bug:

> **Boxed insight — a channel model can have physics bugs, and they masquerade as modem bugs.**
>
> 1. **Apply the fade to the analytic signal, not the real signal.** The correct operation is `out = Re{ analytic(s) · h }`: this scales by the Rayleigh magnitude `|h|` and turns `arg(h)` into a harmless carrier-phase rotation. The previous implementation multiplied the real passband signal by `Re{h}` directly, which drops the quadrature term — so the signal was annihilated whenever `arg(h) ≈ ±90°`, a deep fade *independent of* `|h|` *or SNR*, with spurious sign inversions. The regression gate `flat_fading_does_not_phase_annihilate` measured the buggy path deep-fading ~16 % of realisations below 0.2× amplitude at 60 dB SNR with zero delay, against the Rayleigh-theoretical ~4 % (`watterson.rs`).
> 2. **Normalise total path power to one.** Two independent equal-power rays each with `E[|h|²] = 1` sum to power 2, delivering the signal +3 dB hot relative to the input-keyed noise — so every labelled Watterson SNR read ~3 dB optimistic. Fixed by scaling each ray by `ray_scale = 1/√2`; gated by `total_path_power_normalized_to_unity`.
>
> The symptom of a model bug is a result that physics forbids — "multipath *improves* decode" or "the fade depth doesn't depend on SNR". Suspect the model before the modem.

The Doppler envelope (`fading.rs`, `doppler_envelope`) is synthesised in the frequency domain with a rate trick: generate the fading process at a low internal rate `fs_env = clamp(doppler × 8, 50 Hz, Fs)` and linearly interpolate up to the signal rate. The envelope is hugely oversampled relative to its bandwidth at 8 kHz, so the interpolation is essentially exact, while direct generation at 8 kHz would demand a shaping FFT of `2·Fs/doppler = 160 000` points — 2^18 after rounding up — for the 0.1 Hz good-F1 profile. The rate trick keeps that inside the

module's `MAX_FFT = 1 << 16` cap. An i.i.d. complex Gaussian spectrum is shaped by a Gaussian Doppler filter `exp(−½(f/σ_bins)²)` and inverse-FFT'd; the power normalisation `scale = 1/√(2·filter_energy)` cancels rustfft's unnormalised IFFT so `E[|h|²]` is independent of FFT size. A historical bug: good F1 at 0.1 Hz used to collapse to a near-constant envelope because the shaping `σ_bins` fell below its floor; the gate `f1_envelope_has_non_trivial_variation` requires a coefficient of variation of windowed RMS > 0.10 over 10 s.

Continuous mode. The default path draws an independent fade realisation per `apply()` call, so a streaming caller feeding frame-by-frame sees an uncorrelated fade each frame. Setting `WattersonConfig::continuous` swaps in a phase-persistent sum-of-sinusoids fader (`SosFader`, `fading.rs`): `h[k] = (1/√M) Σ_m exp(j(2π f_m k/fs + φ_m))` with 48 oscillators, `f_m ~ N(0, σ_d)` (a Gaussian spread of Doppler shifts gives a Gaussian PSD) and uniform random phases drawn once at construction, so `E[|h|²] = 1` and phase (held in `f64`) does not drift over long runs. Gate: `continuous_fade_correlates_across_calls` requires lag-1 autocorrelation of frame RMS > 0.5 in continuous mode.

2A.2.3 Gilbert-Elliott — the burst channel

Static crashes and burst interference are modelled by a two-state Markov chain (good/bad SNR states). The critical implementation detail in `gilbert_elliott.rs` is that the chain steps **once per symbol, not once per sample**:

```
fn step_state(&mut self, i: usize) {
    if !i.is_multiple_of(self.config.symbol_samples.max(1)) { return; }
    let u: f32 = self.rng.gen();
    if self.in_bad { if u < self.config.p_bg { self.in_bad = false; } }
    else if u < self.config.p_gb { self.in_bad = true; }
}
```

Per-sample stepping made the "bursts" statistically indistinguishable from elevated-variance AWGN, so any interleaver or burst-FEC conclusion drawn from the model was vacuous (config doc, `lib.rs`). Mean burst length is `1/p_bg` symbols; the acceptance gate is `bursts_span_whole_symbols_with_mean_one_over_pbg`. Presets `light`/

`moderate` / `heavy` / `severe` give mean bursts of 10/20/50/100 symbols at bad-state SNRs of 3/0/−3/−6 dB (all with `p_gb = 0.02`, good-state 20 dB, `symbol_samples = 8`).

2A.2.4 The remaining models

- **QRN** (`qrn.rs`) — Middleton-class atmospheric noise: background Gaussian at a configured SNR plus Poisson-arrival impulse spikes. Expected spikes per block = `impulse_rate_hz × n / sample_rate`, actual count drawn from a Poisson distribution; each spike's amplitude is `~N(0, (rms × impulse_amplitude_ratio)²)`, held for up to `max_spike_duration_samples` (a `u8`, so ≤ 255 samples). There are no named presets in the crate; the testbench supplies values.
- **QRM** (`qrm.rs`) — man-made interference: a list of phase-coherent discrete tones (`ToneConfig { frequency_hz, amplitude }`, amplitude relative to signal peak) plus an optional noise floor.
- **QSB** (`qsb.rs`) — slow sinusoidal amplitude fading: `env = (1 + depth)/2 + (1 − depth)/2 · sin(phase)`, phase continuous across calls. This is amplitude-only — a deliberately simplified fade. The *realistic* flat fade is `FlatFadingChannel` (`flat_fading.rs`): a single Doppler-shaped complex Rayleigh ray applied to the analytic signal, so the gain also rotates the carrier phase by `arg(h)` — "it stresses carrier recovery the way real fading does" (file header). Presets: `slow` (0.2 Hz), `moderate` (1.0 Hz), `fast` (5 Hz).
- **Chirp** (`chirp.rs`) — an additive linear sweep `f_start → f_end` over a period, at `amplitude × block_rms`. The sweep position is taken from an explicit sample counter rather than the wrapped phase accumulator (which would give a near-zero position), and phase is wrapped at `1000π` against float precision loss.
- **SRO** (`sro.rs`) — sample-rate offset: two stations' soundcards never share a clock. Each block is resampled by `ratio = 1 + ppm·1e-6` with 4-point Catmull-Rom cubic interpolation, "clean enough at HF audio frequencies that any resulting decode failure is attributable to the timing slip, not interpolation artifacts" (file header). Typical USB soundcard crystals are ±20–100 ppm, so the relative offset between two cards can reach a few hundred ppm.

- **Composite** (`lib.rs`) — a series chain of any of the above: stage N's output feeds stage N+1.

2A.2.5 The fading-aware raw-audio SNR estimator

`openpulse_channel::estimate_additive_snr_db(reference, received)` measures *additive* SNR with the multiplicative channel removed first: form both analytic signals, then per 256-sample window solve the least-squares complex gain

```
g = ⟨a_rcv, a_ref*⟩ / ⟨a_ref, a_ref*⟩
```

and measure the residual `a_rcv − g·a_ref`. The doc comment states the reason: a naive `|ref|² / |ref − received|²` counts the multiplicative fading as noise, so on any fading channel it collapses to ∼−3 dB regardless of the actual SNR. Gate: `additive_snr_matches_awgn_and_ignores_fading` — within 4 dB of a configured 15 dB AWGN, and above 15 dB on Watterson `good_f1` at 40 dB. This is the raw-audio twin of the symbol-domain estimator in §2A.7.4, and the same bug class recurs there.

2A.3 Modulation as implemented

Every waveform is a `ModulationPlugin` (`crates/openpulse-core/src/plugin.rs`); the mode string (`"BPSK250"`, `"OFDM52-16QAM"`, …) is the wire-level identity. Plugins optionally override `demodulate_soft`, `estimate_afc_hz`, `estimate_snr_db`, `occupied_bandwidth_hz`, `modulate_iq`, `supports_soft_demod`, and `frame_geometry` — each override matters somewhere in this chapter.

2A.3.1 BPSK — differential by construction

Modes: `BPSK31`, `BPSK63`, `BPSK100`, `BPSK250`, `BPSK250-RRC` (`plugins/bpsk/src/lib.rs`). A note for operators: the baud parser maps `"31" → 31.25` and `"63" → 62.5`, so BPSK31 is really 31.25 baud and BPSK63 is 62.5 baud — the names are the conventional PSK31-family rounding, not the actual rates.

Frame layout:

```
┌──────────────┬──────────────────┬──────────┐
│  preamble    │  data symbols    │  tail    │
│  32 symbols  │  8 × N symbols   │  8 syms  │
└──────────────┴──────────────────┴──────────┘
```

Each bit is **NRZI-encoded** — "1" flips the phase, "0" keeps it — and pulse-shaped with a 50 % overlapping half-Hann crossfade to minimise occupied bandwidth (`lib.rs` header). NRZI makes BPSK **differential by construction**, and that single fact is why BPSK survives HF fading where coherent QPSK dies (§2A.8.2). `occupied_bandwidth_hz` returns `2 × baud` — the rectangular main-lobe null-to-null width, a documented safe over-estimate.

2A.3.2 QPSK, and the differential `-D` modes

Modes run from `QPSK125` through `QPSK9600-RRC`, including two differential modes, `QPSK250-D` and `QPSK500-D` (`plugins/qpsk/src/lib.rs`; `is_differential(mode) = mode.ends_with("-D")`). The coherent constellation is Gray-mapped:

```
match (b0, b1) {
    (false, false) => ( 1/√2,  1/√2),   // 45°
    (false, true ) => (-1/√2,  1/√2),   // 135°
    (true,  true ) => (-1/√2, -1/√2),   // 225°
    (true,  false) => ( 1/√2, -1/√2),   // 315°
}
```

In the `-D` modes the dibit selects a **phase increment**; the phase reference is the last preamble symbol, derived from `preamble_symbols()` rather than hardcoded — "a preamble change must move both ends together or every `-D` frame silently decodes to noise" (`modulate.rs`). The decoder computes the conjugate product of adjacent symbols and quantises the phase step to the nearest π/2:

```
let re = i1 * i0 + q1 * q0;
let im = q1 * i0 - i1 * q0;
let dphi = im.atan2(re);
let r = (dphi / FRAC_PI_2).round().rem_euclid(4.0) as u8;
```

The differential path runs **no carrier PLL and no LMS equaliser** — the differential detection is inherently phase-drift-invariant — and skips straight from `demodulate_symbols` through `cancel_crossfade_isi` (§2A.4.2) to `differential_decode_to_bytes` (`plugins/qpsk/src/demodulate.rs:168-175`). One deliberate constraint: **there is no soft path**. `qpsk_demodulate_soft` returns an error on `-D` modes rather than emit miscalibrated coherent LLRs. The mechanism test to know:

`differential_qpsk_confines_a_cycle_slip_to_two_dibits` — a slip that would rotate every subsequent symbol in a coherent mode costs exactly two dibits here.

2A.3.3 8PSK, and the shared constellation library

`plugins/psk8` provides Gray-coded 8-phase modes (`8PSK500` through `8PSK9600-RRC`), 3 bits/symbol at a ±22.5° decision margin — the tightest phase margin in the single-carrier family, which is why 8PSK is both the throughput step above QPSK and the regression canary for every acquisition change (§2A.5.7).

The dense constellations live in one shared library, `openpulse_dsp::constellation` (`crates/openpulse-dsp/src/constellation.rs`), used by OFDM and SC-FDMA. All constellations are Gray-coded and normalised to unit average power; the scale constants are `1/√2` (QPSK), `1/√10` (16QAM), `1/√20` (cross-32QAM), `1/√42` (64QAM). The PAM axes are Gray-coded (`00→−3, 01→−1, 11→+1, 10→+3` for PAM-4; the analogous 8-level table for PAM-8), with slicer thresholds at even multiples of the scale.

Cross-32QAM deserves a note for the engineering reader: Gray coding in 2D is **not a closed-form construction** for a cross-shaped constellation (36-point grid minus 4 corners). The label table `QAM32_BY_LABEL` was optimised numerically — simulated annealing in `crates/openpulse-dsp/tests/qam32_gray_optimizer.rs` — to minimise the total Hamming distance between Euclidean-adjacent points: an average of 1.36 bits per nearest-neighbour transition versus 2.04 for the old 1D-Gray-over-raster mapping. That number is what the soft demodulator's LLRs and the bit error rate actually depend on, so the optimised table was frozen into the source.

2A.3.4 MFSK16 — the non-coherent sub-floor rung

When the fade is fast enough, no phase-tracking waveform survives, so the ladder's bottom rung abandons carrier phase entirely. `plugins/mfsk16` is a constant-envelope, non-coherent 16-tone GFSK waveform: 16 tones at 31.25 Hz spacing, 31.25 baud (an exact 256 samples/symbol at 8 kHz), 4 bits/symbol, 500 Hz occupied, tone synthesis reused from the JS8 GFSK primitives. The data frame is one 255-byte RS block: 510 data tones plus three 7-

symbol Costas sync blocks = 531 symbols, about 17.0 s on the air. A short variant, `MFSK16-ACK`, carries a 13-byte ShortFec-encoded ACK in 40 symbols (≈ 1.28 s).

The Costas sync array is `COSTAS16 = [8, 4, 10, 12, 2, 6, 0]` — the FT8 legacy Costas array doubled, preserving the distinct-difference property. Acquisition is a normalised per-symbol tone-fraction correlation searched jointly over timing and frequency, so the plugin self-acquires (a ±25 Hz tuning offset was validated); `estimate_afc_hz` returns `None` — the non-coherent plugin opts out of the engine's coherent AFC chain entirely (`lib.rs` header).

Measured performance (`docs/dev/research/robust-narrowband-measurement.md`, real-sync sweep, 0.5-decode-crossing SNR on Watterson channels):

Channel	BPSK31 crossing	16-GFSK real-sync crossing	Net gain
`good_f1`	~−4 dB	~−4 dB	~0 (no regression)
`moderate_f1`	~+3.7 dB	~−0.5 dB	**~4.2 dB**
`poor_f1`	never (0.00 through +6 dB)	~−0.3 dB	unbounded

On `poor_f1` (2 Hz Doppler), real-sync 16-GFSK decodes 0.00/0.67/1.00/1.00 at −3/0/+3/+6 dB while BPSK31 decodes 0.00 at every point — the mechanism in the clear: 2 Hz Doppler breaks coherent carrier tracking, but a non-coherent Goertzel energy detector is immune, and the 500 Hz span gives per-symbol frequency diversity the FEC harvests. Two further facts from the same document: ΔPAPR = −1.45 dB is a *credit* (16-GFSK is 0.00 dB constant-envelope versus BPSK's 1.46 dB), and the AWGN sanity check holds — 16-GFSK crosses ~1 dB *worse* than BPSK31 on AWGN, as a fading-only lever must. Acquisition erosion from genie-sync to real-sync measured ~1 dB on `moderate_f1` and ~0.3 dB on `poor_f1`.

2A.3.5 OFDM

Above the phase-margin wall (§2A.8.4) the ladder switches family to OFDM. Shared geometry (`plugins/ofdm/src/params.rs`): FFT size 256, cyclic prefix 32 samples, symbol length 288 samples, subcarrier spacing 31.25 Hz = 8000/256, pilots every 5th occupied subcarrier at amplitude 1.0. The 32-sample CP is **4.0 ms at 8 kHz** — long enough to ride the delay spread of every standard profile up to `poor_f2` (whose 5.0 ms exceeds it).

Mode	Occupied SCs	Data/pilot SCs	Bits/SC	Gross bps (doc)	Occupied BW
OFDM16	20 (SC 38–57)	16 / 4	2	~889	~625 Hz
OFDM52	65 (SC 16–80)	52 / 13	2	~2889	~2031 Hz
OFDM52-8PSK	65	52 / 13	3	~4333	~2031 Hz
OFDM52-16QAM	65	52 / 13	4	~5778	~2031 Hz
OFDM52-32QAM	65	52 / 13	5	~7222	~2031 Hz
OFDM52-64QAM	65	52 / 13	6	~8667	~2031 Hz

Both mode families centre on subcarrier 48 = 1500 Hz (asserted by the `ofdm16_geometry` / `ofdm52_geometry` tests). The preamble loads only even subcarriers at amplitude √2 (half the band loaded, so √2 keeps total preamble power comparable to a data symbol), with a deterministic whitening hash choosing each subcarrier's sign — a pseudo-random pattern keeps preamble PAPR moderate and the autocorrelation peak sharp, where an all-`+1` comb would be highly peaked (`params.rs`).

Acquisition is Schmidl-Cox: stage 1 is coarse, CFO-robust presence detection via the classic half-symbol autocorrelation metric P^2/R_2^2; stage 2 selects the **leading** multipath tap rather than the argmax (§2A.5.4), with a threshold "low enough to catch a weak leading path, high enough to reject pre-peak noise" (`plugins/ofdm/src/demodulate.rs`).

Channel estimation and equalisation (`plugins/ofdm/src/channel.rs`): least-squares estimates at the pilot subcarriers, linearly interpolated across the data subcarriers, then zero-forcing equalisation (divide each data bin by its channel estimate). Before interpolating, `deramp_timing` removes the dominant linear phase ramp across subcarriers: a residual symbol-timing offset of δ samples imprints a slope of `−2πδ/N` radians per subcarrier on the FFT output, and linear interpolation between sparse pilots across such a ramp is lossy — so the slope is estimated from adjacent pilot pairs and de-rotated first. CFO is estimated from inter-symbol pilot phase drift across up to 8 data symbols; since the rotation per symbol is `2π·CFO·SYM_LEN/Fs`, the unambiguous range is `±Fs/(2·SYM_LEN) ≈ ±13.9 Hz`.

Soft demodulation scales the per-subcarrier effective noise by $\text{mean}|H|^2 / |H_{sc}|^2$, so faded subcarriers yield lower-confidence LLRs (`demodulate.rs`). This per-subcarrier weighting is where OFDM's robustness to frequency-selective fading is actually realised — a faded bin is not wrong, it is *known to be unreliable*, and the soft FEC spends its redundancy there.

PAPR handling in the modulator follows three rules (`modulate.rs`): only QPSK-subcarrier frames are clipped (`clip_iterative` to a 12 dB target; clipping injects broadband distortion the dense constellations cannot absorb); data and preamble are clipped separately so the preamble's high comb-PAPR cannot raise the data clip threshold; and higher-order frames are instead peak-normalised to 0.9 so a real DAC never hard-clips them — the inherent PAPR backoff, taken deliberately rather than accidentally.

2A.3.6 SC-FDMA

`plugins/scfdma` shares OFDM's geometry exactly (FFT 256, CP 32, 31.25 Hz spacing) so each SC-FDMA mode occupies the same bandwidth as the corresponding OFDM mode; the difference is a DFT precoding of the data (single-carrier FDM). Its `deramp_timing` doc states the physics difference: on the direct per-subcarrier OFDM path a phase ramp is benign, but SC-FDMA's DFT de-spread coherently combines all subcarriers, so an uncorrected ramp smears across every recovered data symbol (`plugins/scfdma/src/channel.rs`).

Its channel estimator, `DelayCe`, is a physical-delay-basis estimator (taps at 0.21 ms spacing, reach ±10 samples ≈ two-sided ~2.5 ms after `deramp_timing` re-centres, inside the CP) with a Wiener ridge weighted by an exponential power-delay prior of RMS ≈ 0.19 ms. Three pieces of measured reasoning from its comments are worth repeating because each pins a swept trade-off:

- The prior is a regulariser, not a claim about the channel: "a flat prior ridges every tap equally, so widening the reach simply gives pilot noise more places to hide (measured: ~6 dB of AWGN frame-success lost going from ±4 to ±10 samples)… the value was swept, not derived."
- The noise window carries an explicit margin before a comb tap is treated as noise: without it, a channel tap between two comb taps Dirichlet-leaks into the "noise" set and σ^2 saturates at an apparent ~8 dB SNR on any selective channel, over-ridging the estimator and de-rating every LLR.
- A mode with fewer pilots uses fewer taps at the same spacing — it loses *reach*, not resolution; spreading a small tap set across the full reach instead costs the 6-pilot SCFDMA26

modes ~2 dB near their floor (measured 0.17 → 0.62 frame success at 4 dB on SCFDMA26-32QAM).

`DelayCe` replaced a broken estimator whose story is told in §2A.8.5. Two demonstrator modes exist, each selected by a `ScFdmaParams` flag: `SCFDMA52-LP` sets `localized` (contiguous data block, single-tap flat estimation, the low-PAPR layout) and `SCFDMA52-P2` sets `pn_pilots` (Zadoff–Chu quadratic pilot phases, `π·k·(k+1)/13` — ideal autocorrelation spreads the pilot comb's energy uniformly in time instead of letting 13 equal-phase cosines peak together, while constant modulus keeps the channel-estimate division well-conditioned; wire-incompatible with the equal-phase modes, so only versioned modes set it).

2A.3.7 Pilot-framed modes

`plugins/pilot` registers 20 modes (`PILOT-{QPSK,8PSK,16QAM,32APSK}{500,1000}` in rectangular and `-RRC` form, plus 2000-baud `-RRC` variants). Known pilot symbols at a fixed cadence drive `openpulse_dsp::pilot_tracker::PilotTracker` — a type-2 PLL corrected at every pilot by a *known* symbol, so the loop is immune to the decision errors and ±90°/±45° cycle slips that defeat a decision-directed Costas loop on dense constellations at low SNR (`crates/openpulse-dsp/src/pilot_tracker.rs`).

The essential caveat, measured and documented (`docs/mode-fec-ladder.md` §1): **cycle-slip immunity is not fade robustness.** `PILOT-QPSK500+Rs` decodes 0 % on Watterson `moderate_f1` at 40 dB — worse than `QPSK250-D`'s 65 % on the same channel — while being 100 % on AWGN down to 10 dB. The ablation shows both impairments bite independently (delay-only 0.33, Doppler-only 0.21): at 500 baud a 1.0 ms delay spread is half the 2 ms symbol, and this family has no equaliser for it. The pilot family's immunity is specifically to carrier-offset and sample-rate-offset slips.

2A.3.8 FSK4-ACK and JS8

`plugins/fsk4` is the narrow ACK channel: 4 tones at `fc ± 50` and `fc ± 150` Hz (default fc = 1050 Hz), 100 baud, 2 bits/symbol, Hann-windowed, Goertzel-demodulated; a 5-byte ACK frame is 20 symbols = 200 ms. It is deliberately hard-decision only

(`supports_soft_demod()` returns `false`): the ACK channel always uses the hard-decision `ShortFecCodec`, and FSK4-ACK never carries LDPC or turbo payloads (`lib.rs`). Gate: `fsk4_integration`.

`plugins/js8` implements the JS8-compatible 8-GFSK weak-signal waveform for discovery. JS8Call runs at 12 kHz; OpenPulse runs at 8 kHz — tone spacing and baud are sample-rate-independent protocol facts, and every submode's samples-per-symbol is an exact integer at 8 kHz, so no resampling is needed (`plugins/js8/src/submode.rs`). 79 symbols per transmission (3×7 Costas sync + 58 data symbols of 3 bits = a 174-bit codeword), FEC is LDPC(174,87) — 75 message bits + 12 CRC — with a sum-product belief-propagation decoder whose tables are ported from JS8Call (GPL-3.0). The Normal submode (6.25 baud, 15 s slots, quoted −24 dB sensitivity) is the interop baseline; the acceptance gate is `snr_sweep gate_at_minus_18_db` for the native decoder.

2A.4 Pulse shaping

2A.4.1 Root-raised-cosine

The `-RRC` mode suffix selects square-root raised-cosine pulse shaping with **α = 0.35**, the stated HF default (`crates/openpulse-dsp/src/rrc.rs`; the suffix check lives in each plugin, e.g. `mode.ends_with("-RRC")` in `plugins/qpsk/src/demodulate.rs`). The impulse response `srrc_at(t, α)` handles the two standard removable singularities explicitly — `t = 0` gives `1 + α(4/π − 1)`, and `|4αt| = 1` gives `(α/√2)·[(1 + 2/π)sin(π/4α) + (1 − 2/π)cos(π/4α)]` — with the general form

```
h(t) = [sin(πt(1−α)) + 4αt·cos(πt(1+α))] / [πt(1 − (4αt)²)]
```

Coefficients are normalised to unit energy so the TX/RX matched-filter pair satisfies the Nyquist criterion; the gate `matched_filter_pair_satisfies_nyquist` convolves the filter with itself and asserts `|RC[centre ± k·sps]| < 0.05` for k = 1, 2, 3. Cost is negligible at 8 kHz: the file's own benchmark (2026-05-07, deliberately pessimistic naive-FIR state) puts a 512-tap filter at ~3.1 ms per second of audio, with the production `VecDeque`-based `FirFilter` 2–5× cheaper — well under 1 % real-time even on a Raspberry Pi estimate.

2A.4.2 The rectangular "crossfade" pulse and its invisible ISI

This is the best worked example in the codebase of a DSP defect that no bit-error test can see.

The "plain" (non-RRC, non-`-HF`) QPSK and 8PSK modulator does not actually emit a rectangular pulse. It emits a raised-cosine **crossfade between adjacent symbols**: sample `i` of slot `k` is `sym_k·w_tail(i) + sym_{k+1}·w_head(i)` with `w_tail = ½(1 + cos(πi/n))`, `w_head = 1 − w_tail`. A matched integrate-and-dump over one slot therefore recovers not `sym_k` but

```
p_k = sym_k + β·sym_{k+1},    β = Σ w_head·w_tail / Σ w_tail² = 1/3
```

and the value is independent of `n` — both sums scale with it (`CROSSFADE_ISI_BETA = 1/3`, `plugins/qpsk/src/demodulate.rs`). 8PSK is worse and different: its matched demod integrates against the *squared* window, so `β = Σ w_head·w_tail² / Σ w_tail³` becomes n-dependent — 0.182 at 16 samples/symbol (8PSK500), 0.167 at 8 (8PSK1000) — and is computed from the actual window rather than a constant (`crossfade_isi_beta(n)`, `plugins/psk8/src/demodulate.rs`).

Since `p_k = sym_k + β·sym_{k+1}` is bidiagonal, backward substitution recovers the symbols exactly:

```rust
fn cancel_crossfade_isi(symbols: &mut [(f32, f32)]) {
    let beta = CROSSFADE_ISI_BETA;
    for k in (0..symbols.len().saturating_sub(1)).rev() {
        symbols[k].0 -= beta * symbols[k + 1].0;
        symbols[k].1 -= beta * symbols[k + 1].1;
    }
}
```

The recursion is stable — each step scales the running error by β = 1/3, so it decays backward — the terminal is exact (the modulator sets the last data symbol's successor to zero), and noise is amplified by only `1/(1 − β²) = 1.125`, i.e. +0.5 dB (function doc, `demodulate.rs`).

> **Boxed insight — a −9.5 dB ISI floor is invisible to BER and fatal to soft FEC.**

> A deterministic leak of β = 1/3 of the next symbol puts an error floor at `β² = −9.5 dB` on recovered-symbol EVM — nowhere near QPSK's 45° decision margin, so **no BER test could see it**. What it did do: it stalled `mean(|LLR|)` above ~12 dB SNR and floored EVM at −9.7 dB regardless of SNR, capping every soft consumer. The QPSK500 soft-FEC floor was stuck at 0.00 and decodes after the fix (PR #695); 8PSK500 EVM cleared −13.7 → −20.0 dB at 40 dB SNR.
>
> The ISI is **anti-causal** — it is the *next* symbol — so the decision-feedback equaliser, which feeds back *past* decisions, cannot reach it. And the cancellation carries its own trap: the `cosine_overlap` / `-HF` pulse is a per-symbol `sin²` bump with **no** inter-symbol overlap, so cancelling there *injects* a third of the next symbol as error. The shipped QPSK fix originally ran unconditionally — a latent soft-path corruption on `QPSK1000-HF` — and both plugins now guard it: `if !cosine_overlap { cancel_crossfade_isi(&mut raw); }`.

2A.5 Acquisition and carrier recovery

Blind acquisition — recovering timing, frequency and phase simultaneously from a 16-symbol preamble — is the single most-churned and most-misdiagnosed area of the modem (60+ AFC/carrier commits; the project's DSP playbook in `CLAUDE.md` exists because of it). The chain, all in `crates/openpulse-modem/src/engine.rs`, is:

```
audio ──► energy gate ──► refine_onset ──► afc_mini_settle ──► decode ──► carrier tracker
          (presence)      (where exactly)  (coarse CFO)                    (fine CFO + phase)
```

2A.5.1 The engine chain

Energy gate (`EnergyGate`, `engine.rs`). Not a fixed threshold: it keeps a 128-window history and uses the 25th percentile as a robust noise-floor estimate (tolerant of up to 75 % signal-bearing windows), gating at 3× that floor, clamped to `[1e-4, 3.2e-3]` mean-square. The rationale in the source: a fixed 1e-4 gate passes *every*

position when the band noise floor is elevated (on-air QRM ≈ 1.5e-3 was measured), firing the expensive AFC settle at each scan step.

`refine_onset`. The gate's wide window (~32 symbols) trips up to a full window *before* the true onset, because its tail catches the first signal samples — far beyond the demodulator's one-symbol timing search. The fix scans symbol-length sub-windows across the gate span and returns the first whose energy reaches a quarter of the span's peak, so the preamble lands within one symbol period.

`afc_mini_settle`. One wide-scan anchor pass, then five fine-tracking passes. Critically, it runs on the *refined-onset* window: settling on the coarse gate window (which may be mostly silence) produced confident-but-bogus estimates — QPSK500 once acquired a spurious ~257 Hz correction that way.

`AFC_SETTLE_DEADBAND_HZ = 2.0`. A settled correction below 2 Hz is snapped to zero: applying a spurious sub-Hz correction over-corrects a zero-offset frame and breaks modes that re-fit carrier phase from the preamble (8PSK's `carrier_phase_correct` enters a fragile drift-fit branch at ≥ 0.5 Hz). Real HF offsets are tens to hundreds of hertz — the measured inter-rig offset on the project's hardware is ~400 Hz — so the deadband never suppresses a real one (`engine.rs`).

2A.5.2 Carrier-phase-insensitive matched filtering

A bare real cross-correlation `Σ a·b` against a known passband template is carrier-phase sensitive: an arbitrary carrier phase rotates the received waveform, and at ~90° the real correlation collapses to near zero, so the search locks to a wrong offset. This failure was independently found and fixed in the QPSK, OFDM (#385) and SC-FDMA (#386) plugins before being centralised: `openpulse_dsp::acquisition::IqMatchedFilter` correlates against both the template and its Hilbert quadrature companion and takes the I/Q magnitude (`crates/openpulse-dsp/src/acquisition.rs`). The same module provides `preamble_corr_sq` (the symbol-domain twin, `|Σ r·conj(e)|²`) and `estimate_cfo_mth_power`: raising each symbol to the M-th power strips M-ary PSK modulation, leaving a phasor rotating at `M·2π·Δf/baud` per symbol; the mean phase of consecutive products gives Δf, with unambiguous range `±baud/(2M)`

and validity restricted to near-constant-modulus constellations (QAM data adds heavy self-noise; use a data-aided preamble estimator there).

2A.5.3 Acquire on ρ, not the raw score

> **Boxed insight — when the preamble itself fades, energy-weighted acquisition is exactly backwards.**
>
> `IqMatchedFilter::search` uses the unnormalised argmax, which favours high-correlation *and* high-energy alignment — on the reasoning that a deep-fade low-energy window cannot win. When the **preamble** is the faded part, that reasoning inverts. Measured on SC-FDMA under a flat Watterson fade: at the true offset the normalised correlation was ρ = 0.994 with window energy 19.4, while a data-region window 4896 samples later scored higher on energy alone with ρ = 0.657 — and the demodulator locked onto it. ρ is amplitude-invariant, so it is unmoved by the fade. `search_normalized(samples, bound, min_energy_frac)` is the fix; its energy floor is what keeps ρ meaningful, because on a near-silent window both numerator and denominator vanish and ρ is numerical noise (doc comment, `acquisition.rs`).

2A.5.4 Lock ahead of the correlation peak, never on it

A matched filter's argmax sits on whichever multipath ray is instantaneously strongest — the *delayed* one about half the time. A late FFT-window start pulls the next symbol into the window; the cyclic prefix only protects an **early** start, where the window begins inside the symbol's own prefix — a circular shift that `deramp_timing` removes. SC-FDMA locked on the argmax and lost half of all Watterson frames for it; the fix (PR #688) moved the `good_f1` decode sum from 9.19 to 29.57 of 42 with AWGN bit-for-bit unchanged. OFDM already scanned back for the leading tap. Note the asymmetry that hid the bug: with the direct ray stronger the argmax is already correct, so a *symmetric* static two-ray test passes either way — reproducing it requires `a_delayed > a_direct`.

2A.5.5 The dedicated frequency-acquisition stage

`crates/openpulse-dsp/src/freq_acquire.rs` implements a qdetector-style two-pass joint estimator (design doc: `docs/dev/design/freq-acquisition-design.md`):

1. **Coarse (joint timing + CFO).** For each candidate timing τ, de-rotate the received window by the known preamble (`rx[τ+n]·conj(p[n])`) and take an L-point FFT. De-rotation strips the preamble modulation, leaving `γ·exp(j(2πfn + φ))`, whose FFT is a single peak at the CFO bin — so the maximum bin magnitude over all `(τ, k)` is a carrier-phase- and CFO-insensitive timing metric, and its bin is the coarse CFO.
2. **Fine (CFO + phase + gain).** At the winning τ, zero-pad and FFT again for finer bins, then quadratically interpolate the magnitude peak for sub-bin CFO; the peak's complex value gives phase and gain.

CFO is reported in cycles per sample in `[−0.5, 0.5)`. The function is pure over complex baseband — no engine state. Gate: `crates/openpulse-modem/tests/freq_acquire_accuracy.rs`. This stage exists because the external references the project mined (gnuradio's FLL band-edge, liquid-dsp's framesync, the qo100-modem) all pair RRC shaping with a *dedicated* frequency-acquisition stage; a single decision-directed loop doing everything was the outlier design.

2A.5.6 The Costas/PLL core

`crates/openpulse-dsp/src/pll.rs`, `CarrierPll`: a second-order loop with gains from the standard Mengali & D'Andrea formulation — damping `1/√2`, `α = 2·damp·Bn`, `β = Bn²`, where `loop_bw` is the normalised bandwidth `Bn·Ts` (0.01–0.05 typical). Discriminants per PSK order:

psk_order	Modulation	Discriminant
1	BPSK	`e = Q · sign(I)`
2	QPSK	`e = Q·sign(I) − I·sign(Q)` (standard Costas form)
3	8PSK	decision-directed: `wrap(angle − nearest_8PSK_phase)`

QAM is explicitly out of scope for this primitive ("QAM needs a decision-directed loop, not a PSK discriminant"), and the update order matters: the current phase estimate is applied *before* computing the error, so the discriminant sees phase-corrected I/Q.

2A.5.7 Acquire-then-track, and the playbook rules

Carrier recovery is **two loops, not one**. A gentle (low-bandwidth) loop holds lock but cannot acquire even a ~1 Hz residual over a short 60–200-symbol frame; a single high-bandwidth loop acquires but regresses the clean dense modes (8PSK9600). So 8PSK (`dd_track_seeded`) and 64QAM (`dd_carrier_track_2pass`) both run pass 1 wide to acquire the frequency, then pass 2 narrow, *seeded* with it, to track cleanly.

Three further hard-won rules from the playbook deserve statement here because they are physics, not project trivia:

- **Do not extract sub-Hz CFO from a 16-symbol preamble by a magnitude-peak frequency search.** Its frequency resolution is only ~baud/16 (31–62 Hz for these modes) and the magnitude metric is sidelobe-ridden; coarse scans locked to spurious peaks at −100 to −256 Hz. Use the scan only for coarse acquisition; the data-aided mean-phase-increment estimator is the precise stage (ISI-biased by ~0.9 Hz, which the tracker absorbs).
- **Dense constellations are the regression canaries.** 8PSK (±22.5°) and 64QAM surface every timing/phase/AFC weakness that BPSK and QPSK hide. Validate acquisition changes against them.
- **Diagnose an "AFC" failure with the swept-applied-correction experiment first.** Modulate at `fc + Δ`, then demodulate with a manually swept `afc_correction_hz`. If decoding fails even at the exactly correct Δ, the estimator is innocent — the bug is in timing, onset, or the carrier tracker. This one experiment relocated the 8PSK carrier-offset gap (PR #417) from "AFC precision" — where earlier work had spent days on FLL ports and preamble redesigns — to a broken drift-fit branch in `carrier_phase_correct`.

For channels whose Doppler is itself changing, `crates/openpulse-dsp/src/doppler_tracker.rs` provides `DopplerTracker` (phase-slope estimation across symbol windows, returning a rate in Hz/s with a confidence) and `AdaptiveAfcLoopBandwidth`, which adjusts the PLL loop bandwidth from SNR and Doppler rate.

2A.6 Timing recovery and equalisation

2A.6.1 Gardner, and why it is not enough

`crates/openpulse-dsp/src/timing.rs` implements the classic non-data-aided Gardner detector:

```
e[n] = s_mid × (s_next − s_prev)
```

For a Nyquist-filtered signal with perfect timing, the midpoint sits at the zero-crossing of the ISI-free eye, making the mean error exactly zero. The implementation strobes at `round(sps + mu)` samples and updates `mu += gain × error`, with `mu` clamped strictly inside ±0.5 (±0.49): allowing `|mu| ≥ 0.5` would make the strobe round to `sps ± 1`, skipping or doubling a symbol and corrupting every subsequent byte. `pre_arm()` sets the phase so the next sample strobes immediately, for use when a brute-force preamble search has already found the ISI-free position.

But that clamp means `GardnerDetector` **cannot actually adjust the sampling instant** — the strobe interval always equals `sps`, a fixed stride from the initial preamble lock. With two free-running soundcard clocks, a 50 ppm sample-rate offset drifts the ISI-free sampling point by one full sample every ~2.5 s at 8 kHz; long frames slip into heavy ISI with no mechanism to recover (`crates/openpulse-dsp/src/farrow.rs` header). `FarrowTimingLoop` is the answer: a proportional-plus-integral loop that tracks both a fractional phase (P term) and the actual samples-per-symbol period (I term), interpolating sample values at arbitrary fractional positions with a cubic (Farrow) interpolator. Its Gardner error is computed on the **complex baseband** — `e = Re{z_mid · conj(z_prev − z_cur)}` — which is invariant to a common carrier-phase rotation, unlike the I-channel-only error previously fed to `GardnerDetector`, which is data-dependent for quadrature constellations.

2A.6.2 The LMS/DFE equaliser

`crates/openpulse-dsp/src/equalizer.rs`, `LmsEqualizer`: symbol-rate, complex, with a supervised training phase on known preamble symbols followed by decision-directed adaptation; `dfe_len = 0` gives a pure forward filter, non-zero enables a feedback section that convolves past hard decisions with the DFE taps and subtracts

them. A tap-energy guard (`MAX_TAP_ENERGY = 16.0`) bounds divergence. QPSK's per-mode profiles (`plugins/qpsk/src/demodulate.rs`) all set `dfe_len = 0` — the shipped QPSK path is forward-only LMS, with (fwd, μ) = (11, 0.010) for `-HF-RRC` 1000-baud, (15, 0.015) for `-HF` 1000-baud, (7, 0.02) otherwise, trained over the preamble. Gate: `qpsk_hf_rrc_forward_only`. The sharpest single line on the equaliser's limits was already stated in §2A.4.2: the crossfade ISI is anti-causal, and a DFE feeds back past decisions — it cannot reach the next symbol.

2A.6.3 The notch bank

`crates/openpulse-dsp/src/notch.rs`, `NotchBank`, removes narrowband CW interferers with second-order IIR notch biquads (RBJ cookbook, unity passband gain). Detection keys on *local spectral prominence*: a CW tone concentrates its energy in a handful of FFT bins and stands far above its immediate neighbours, while a modem signal spreads across many bins so no single bin is prominent. The documented remaining failure mode — a tone landing inside a narrowband signal's own main lobe — is physical, not a detector bug: the notch then removes signal too. A silence-persistence tracker separates external interferers from the modem's own spectral lines: a CW interferer is present even when our signal is not, so a tone confirmed during silence is genuinely external and can be notched (out of band) or flagged for QSY (in band) without per-block false-positive risk.

2A.7 SNR estimation, soft demodulation and LLR calibration

2A.7.1 The LLR contract

The codebase-wide convention, declared in `constellation.rs`, `turbo.rs`, and each soft demodulator: **positive LLR = bit more likely 0**, enforced by the gate `llr_convention_conformance.rs`. The core primitive is max-log-MAP over the constellation:

```
pub fn symbol_llrs(symbol, bits_per_sc, noise_var, points) -> Vec<f32> {
    let inv_noise = 1.0 / noise_var.max(1e-6);
    // per bit: (min over points with bit=1 of |symbol − pt|²·inv_noise)
    //        − (min over points with bit=0 of ..)
}
```

The load-bearing consequence: `symbol_llrs` **already divides every distance by** `noise_var`, so a calibrated plugin emits true LLRs whose magnitude is ∝ 1/σ². For repeated observations of the same bits, true LLRs simply **add** — `combine_llrs_map` is the MAP combine and *is* inverse-noise weighting. The engine used to re-weight that sum by a `1/mean(|LLR|)` proxy — a second 1/σ² — costing 0.75 dB on graded HARQ attempt sets (fixed in PR #686). `combine_llrs_weighted` exists only for LLRs with a noise-blind scale (the ±1.0 trait default). Gate: `llr_calibration.rs` fails any plugin whose `mean(|LLR|)` stops growing with SNR.

2A.7.2 Testing what an LLR means

A true LLR L predicts `P(bit wrong) = 1/(1 + e^{|L|})`. The reliability tests bin emitted LLRs by `|L|`, count actual errors, and compare, with a uniform bar of worst-bin error ≤ 4× the promised rate (`llr_reliability` tests in the `qam64-plugin`, `mfsk16-plugin`, `ofdm-plugin`, `pilot-plugin` and `scfdma-plugin` crates). The case study: SC-FDMA's `mmse_llr_noise_var` modelled only the additive noise, omitting channel-estimate error and a residual-ISI variance term; per the project record (CLAUDE.md; fix in PR #690), bits with `|L| ≈ 12` were wrong **71× more often than promised, on a flat channel** — and no frame-success metric could see it, because soft Viterbi, min-sum LDPC and max-log turbo are all scale-invariant and the missing terms were nearly a per-frame constant. The fix produced *no measured decode gain*; it matters for HARQ combining and any iterative structure that derives feedback reliability from LLR posteriors. That is the point of a calibration test: it checks a promise the decode-rate tests cannot.

2A.7.3 Choosing the noise estimator

A demodulator's residual is not all thermal noise: pulse-shaping ISI and equaliser misadjustment vary the symbol *amplitude* with no SNR dependence, so moment estimators (M2/M4) and naive distance-to-nearest-point estimators stop tracking SNR. `constellation.rs` provides three tools, each matched to a situation:

- **Constant-modulus PSK — use the orthogonal component.** With `e = z·conj(ŝ)` and `|ŝ| = 1`, `Re(e)` carries the amplitude and `Im(e)` carries only noise; `psk_symbol_noise_var` returns `(amplitude, noise_var_per_dim)` and `snr_db_from_amp_noise`

converts to `10·log10(A²/2σ²)`. Decision-directed, so it saturates once symbol errors are common — the safe direction.

- **Differential detection — use the quadrature companion.** With `dot_k = Re(z_k·conj(z_{k−1}))` and `cross_k = Im(z_k·conj(z_{k−1}))`: `dot` is antipodal with mean ±A², `cross` has mean 0 and variance ≈ 2A²σ², so `2·mean|dot| / var(cross) → 1/σ²` **and the amplitude cancels exactly** (`differential_llr_scale`) — immune to the amplitude variation that defeats a variance-of-`|dot|` estimate.
- **Non-constant-modulus QAM — decision-directed distance.** `estimate_decision_noise_var` (mean squared distance to the nearest point) and `qam_symbol_snr_db`. Documented biases: invariant to a uniform gain (e.g. a ZF scale), and on a selective channel ZF noise-enhancement on faded subcarriers inflates σ², so it *under*-reads SNR — conservative, hence safe, and the direct cause of the multicarrier saturation in §2A.7.5.

The fallback is the classical M2M4 estimator (`crates/openpulse-core/src/snr_estimate.rs`): for M-PSK in complex Gaussian noise, with `M2 = E[|r|²]`, `M4 = E[|r|⁴]`: `S = √(2M2² − M4)`, `N = M2 − S`. The base function `m2m4_snr_db` deliberately does *not* gate out silence — the full envelope distribution is needed for the moments to be correct, and leading/trailing silence biases the estimate low, which is the conservative direction; it expects the caller to hand it the active signal region. The engine's own fallback path is the gated real-input wrapper `m2m4_snr_db_gated_from_real` (§2A.7.5).

2A.7.4 Issue #934 — the estimator that counted the fade as noise, three times

> **Boxed insight — an SNR estimator built on a residual counts the fade as noise.**
>
> On a fading channel `z ≈ h·s + n`. Any estimate built from the raw residual `z − ŝ` — or its orthogonal component, or M2M4's moments — folds the *multiplicative* `h` into "noise" and stops tracking SNR entirely. Measured on `moderate_f1`: BPSK had no `estimate_snr_db` at all, so the engine's M2M4 fallback read a **flat ≈ −4 dB from 15 dB of true SNR to 35 dB** — the same number across 20 dB of channel (project record; the symbol-domain doc in `constellation.rs` records

> the companion figure, a flat ≈ −6.6 dB from 15 dB upward). `hpx_hf`'s SL2–SL5 are all BPSK, so the rate controller was deciding on a constant.
>
> This was the **third occurrence of the identical bug class**: PR #484 fixed it in the linksim's tx-vs-rx estimator; the linksim was then migrated onto the plugin estimators, which had it too; and BPSK never had an estimator at all. The fix is always the same shape — **remove a per-window least-squares complex gain first**, then measure the residual.

`additive_snr_db_windowed(rx, decisions, window)` (`constellation.rs`) implements the fix in the symbol domain: per window, `g = ⟨rx, ŝ*⟩/⟨ŝ, ŝ*⟩`, then accumulate `|g·ŝ|²` as signal and `|rx − g·ŝ|²` as noise. The window sizing is a genuine two-sided constraint: small enough that `h` is approximately constant across it (a 1 Hz fade has a coherence time of hundreds of symbols at 250 baud), large enough that the LS gain does not absorb the noise it is meant to measure (the gain soaks up ≈ 1/window of it, so ≥ 8 is enforced). Because callers supply their own decisions, it works for any constellation — including BPSK, which `map_symbol` does not model, and differentially-encoded streams, whose arbitrary global sign the per-window gain absorbs.

BPSK's use of it (`plugins/bpsk/src/demodulate.rs`) exposes the **second trap — scale conversion**. The estimate is taken after the matched filter, so it is symbol-domain Es/N0 carrying the mode's processing gain: a 31-baud rung reads ~17 dB above the channel SNR, a 250-baud rung ~8 dB. Left unconverted, the receiver over-recommends badly (a 2 dB AWGN channel drove the ladder to SL5). The conversion is `snr = es_n0 − 10·log10(fs/baud) + MATCHED_FILTER_LOSS_DB` with the loss constant 7.1 dB — measured to hold within ~0.3 dB across BPSK31 and BPSK250, an order of magnitude apart in baud, so it is the pulse's noise-bandwidth loss and not a per-mode fudge (it *is* pulse-specific: fitted to this plugin's Hann/crossfade matched filter). Gates: `bpsk_snr_awgn_scale_matches_channel_snr`, `bpsk_snr_tracks_a_fade`, and `additive_snr` in `openpulse-dsp`.

Two honest limits, both worth internalising. First, at low baud the fix cannot work even in principle: at 31 baud a 1 Hz fade decorrelates in ~6 symbols, so no window is simultaneously short enough to track `h` and long enough to average `n` — BPSK31's estimate stays flat. Second, **a good estimator was not enough**. The rate controller's only climb path was `snr >= ceiling`, so a flat estimate pinned `hpx_hf` on its entry rung at ~5 bps *while delivering 20/20 frames* on a fade. Two rules were added: the ladder climbs on consecutive clean decodes even with a useless SNR reading, and it never demotes below a level that just decoded. The framing the project uses: *a decode is an observation; the SNR is a model; the observation wins.* Gate: `psk_ladder_climbs_off_the_entry_rung_on_a_fade` — driven through the controller, which every prior demod-only fade gate was blind to; plus `success_based_climb`.

2A.7.5 The per-family SNR scale boundary

`ModemEngine::rx_snr_db(mode, samples)` dispatches to the plugin's `estimate_snr_db`, falling back to silence-gated M2M4. The plugins report **different quantities, and cannot be unified**: single-carrier PSK (post-#934) reports approximately true additive channel SNR; the multicarrier plugins report a saturation-bounded plugin-domain SNR, because zero-forcing equalisation enhances noise on faded subcarriers — the estimate flattens near ~16 dB and physically cannot report the 20–30 dB the dense rungs run at (a true 20 dB link reads ~14.4; `docs/mode-fec-ladder.md` §4).

So `hpx_hf`'s SL2–SL6 floors are on the true-SNR scale and SL7–SL14 floors are plugin-domain — two scales, one ladder. This looks like a bug; it is a physical boundary, and it is **pinned by a test** so it cannot be "cleaned up" into a regression: `crates/openpulse-modem/tests/snr_scale_boundary.rs` asserts (through `rx_snr_db`, AWGN, 64-byte payloads) that BPSK250 reads within ±4 dB of true at 5/15/25 dB; that OFDM52's estimate is monotone at the low end but reads ≤ 22.0 at a true 30 dB; and that at true 30 dB the two families differ by ≥ 6 dB. The second test's failure message states the policy: if OFDM ever starts tracking true SNR, the OFDM ladder floors must be re-derived to the true-SNR scale *in the same change* — the two scales are one decision. Forcing unification was scoped

and declined: it would put the top rungs' floors above anything the estimate can read, recreating the v0.14.0 "floors never clear" stall, and the evidence-based climb already bridges the mismatch.

2A.8 Fading physics: what actually survives an HF path

(Everything in this section is measured against the Watterson simulator, principally `moderate_f1` — 1 Hz Doppler, 1 ms delay spread — not on air.)

2A.8.1 The ordering: robustness tracks phase margin

The single organising principle of the mode ladder (`docs/mode-fec-ladder.md` §1):

```
MFSK16 (non-coherent — no carrier phase at all)
   >  BPSK  (±90° decision margin, differential via NRZI)
   >  QPSK  (±45°)
   >  8PSK  (±22.5°)
```

Not baud rate. BPSK250's *longer* frame still beats QPSK250 by 3× on `moderate_f1`. What kills a mode on a fade is how much phase error it can absorb, and how much of the frame one tracking slip destroys.

2A.8.2 Coherent versus differential — the #923 result

In a coherent, absolutely-encoded mode, each symbol's meaning is its absolute phase, so the receiver must hold a carrier reference for the whole frame. On a fading path it cannot: at a fade null the decision-directed loop slips, and because the encoding is absolute, **every symbol after the slip is rotated** — the frame tail is lost. In a differential mode each symbol is a phase *increment* from its predecessor; the fade rotation is common to both symbols of each pair and cancels, so a slip costs one symbol instead of the tail. The price: ~2 dB of AWGN floor at QPSK (differential detection roughly doubles the effective noise), and it needs FEC to repair the symbol each slip still costs.

The #923 measurement set, all on Watterson `moderate_f1`:

Configuration	Decode rate	Condition
Coherent QPSK250 + Rs	**0.00**	at every SNR up to 40 dB
QPSK250, no FEC	0.00	
2-pass `dd_track_seeded` ported to QPSK	**0.00**	measured; dead end
`PILOT-QPSK500` + Rs	**0.00**	at 40 dB; 1.00 on AWGN by 10 dB
`QPSK250-D` + Rs	**~0.65**	at 20 dB
Differential QPSK, no FEC	0.00	
Both QPSK250 and QPSK250-D on AWGN	1.00	by 4 dB

> **Boxed insight — the ablation is the decisive step.** Removing the Doppler rescues coherent QPSK250 (0.82); removing the delay spread does not (0.00). The failure is carrier tracking — not ISI, not noise. Two entirely plausible fixes were built and measured to exactly 0.00 before the differential answer: porting 8PSK's two-pass acquire-then-track loop, and routing to the pilot waveform. The fix was never "a better tracker"; an absolutely phase-encoded waveform is the wrong tool for a channel that rotates phase through nulls. (Sources: `crates/openpulse-core/src/profile.rs` SL6 comment; `docs/mode-fec-ladder.md` §1. Gates: `qpsk_differential_fading`, `cargo test -p qpsk-plugin differential`.)

And differential does not scale to 8PSK — built, measured, rejected. 8PSK500-D reached only 0.050 at 20 dB / 0.125 at 40 dB on `moderate_f1` (versus QPSK250-D's 0.675). The implementation was correct, not broken: its AWGN control decodes 1.000 by 16 dB. But that same control exposes the cost — coherent 8PSK500 is at 0.975 by 8 dB, so the differential penalty at 8PSK is ~4–6 dB versus QPSK's ~2 dB. Differential detection roughly doubles the effective noise, and at a ±22.5° margin that eats more than the fade immunity returns: strictly worse on AWGN *and* still useless on fading. The project's standing note: do not re-attempt it without a different mechanism — pilots dense enough to track the fade, or a non-coherent waveform.

2A.8.3 An HF ladder calibrated on AWGN is not an HF ladder

The `hpx_hf` floors originally came from AWGN sweeps, and on a routine `moderate_f1` fade most of the ladder did not work at the floors it advertised. Effective throughput (decode × net bps) at 20 dB across the old ladder read 346 → **0** → 125 → **0** → 395 → 1816 —

a four-rung dead zone the rung-by-rung adapter had to cross to reach the rungs that worked (`profile.rs` comments). Three rules fell out, all measured:

Rule 1 — there is no useful uncoded rung on a fade. BPSK is differential too (NRZI), so #923's "differential needs FEC" is the whole ladder's law. At their own floors on `moderate_f1`:

Rung	Uncoded	Coded
BPSK31 @3 dB	**0.00** (also 0.00 at 6 and 9 dB)	0.25 with `Rs`; **1.00** with `RsStrong`
BPSK63 @4 dB	0.000	0.833
BPSK100 @4.5 dB	0.04	~1.00
BPSK250 @5 dB	0.000	0.58

Rule 2 — `RsInterleaved` is inert at these frame sizes; code strength is the lever. BPSK250 on `moderate_f1` at 5/8 dB: 0.17/0.58 with `RsInterleaved` — *identical* to plain `Rs`. A ≤ 223-byte payload is one RS block, and a single block is position-agnostic, so there is nothing for the interleaver to spread. The doc table in `docs/mode-fec-ladder.md` §2 used to bill `RsInterleaved` as "best for HF burst/fading"; the measurement forced it to be rewritten, and it now reads "multi-block payloads (> 223 B) on bursty channels; **inert at or below one block**". The lesson is the one that survived: measure the rung, do not trust the table. Interleaving earns its place only across multiple blocks (§2A.10.3).

Rule 3 — "`RsStrong` is free" is true only up to 191 bytes. RS(255,223) and RS(255,191) both emit one 255-byte block, so below 191 B the doubled correction capacity (t = 32) costs nothing and roughly doubles the fading decode. At 192–223 B it needs a second block and **doubles the airtime** — ordinary traffic. The worked numbers: a 200-byte payload frames to 210 B → RS input 214 B → one `Rs` block but two `RsStrong` blocks; BPSK31 airtime goes 66 s → 132 s; `hpx_hf`'s AWGN goodput fell 310 → 199 bps in the link simulator, through the `goodput_gate` floor (a locally-run gate — this project's CI workflows are deliberately disabled). At 64 B, BPSK250+Rs and BPSK250+RsStrong have identical 8.32 s airtime. The project record is candid: "I measured 'free' at 64 B and generalised straight past the boundary that made it true; the linksim goodput gate caught it." The rule is now code:

```
pub fn free_rs_strengthening(fec: FecMode, encode_input_len: usize) -> FecMode {
    if fec == FecMode::Rs
        && rs_block_count(encode_input_len, BLOCK_TOTAL - FEC_ECC_LEN_STRONG)
            == rs_block_count(encode_input_len, BLOCK_TOTAL - FEC_ECC_LEN)
    { FecMode::RsStrong } else { fec }
}
```

(`crates/openpulse-core/src/fec.rs`) — `Rs` is upgraded to `RsStrong` only when the stronger code needs no additional 255-byte blocks; anything but `Rs` passes through unchanged. Gates: `free_rs_strengthening` (core) and `free_rs_strengthening_ota` (modem).

Two counterweight gates hold the whole trade in tension: `hpx_hf_rungs_survive_fade` (every rung must decode ≥ 0.25 at floor + 4 dB on `moderate_f1` — deliberately a weak "dead-rung tripwire, not a floor calibration", because deep-fade outage makes 1.0 unreachable at any SNR) and the linksim `goodput_gate` (clean-channel effective bps ≥ 65 % of baseline — what stops a fade fix from quietly costing 1.5× the AWGN throughput).

2A.8.4 Why OFDM above SL6

Above SL6 the phase margin runs out, so the ladder changes family rather than modulation order. The mechanism that survives instead: OFDM's 4 ms cyclic prefix rides the delay spread, its per-subcarrier pilots track the fade, and its $|H|^2$-weighted soft LLRs turn faded subcarriers into known-unreliable bits the FEC can absorb. Measured on `moderate_f1` (`profile.rs`, `docs/mode-fec-ladder.md` §4): OFDM52 decodes 0.58/0.75/0.83 at 8/12/16 dB where 8PSK500 decodes 0.00 at all three. At equal gross rate OFDM also beats SC-FDMA on selective fade (`tests/ofdm_scfdma_bakeoff.rs`: `moderate_f1` at 20 dB with 16QAM, OFDM 0.88 vs SC-FDMA 0.35; `moderate_f2`, 0.93 vs 0.03) — which is why the dense rungs are OFDM too. One curiosity: `OFDM16` is the most fade-robust OFDM mode and the narrowest at 625 Hz, but its ~401 net bps sits below SL6's 437, so it has no monotonic slot in `hpx_hf`.

2A.8.5 Method lessons

These are the repo's methodological findings, and for the engineering reader they are as valuable as any waveform:

Boxed insight — a modem that fails at *every* SNR has a bug, not a limitation. SC-FDMA's old `dft_ce_estimate` mis-reconstructed every frequency-selective channel (a coarse 3.94-sample delay grid, and negative taps read as large positive ones). Its signature was a flat 2–7 % Watterson decode rate from 8 to 32 dB — recorded as "correct and by design" for two releases. It was found by **taking the noise away**: a static two-ray FIR inside the cyclic prefix, no Doppler, 90 dB SNR — a receiver that cannot decode that has nowhere to hide. Uncoded BER, flat-channel CE MSE and all 58 unit tests were green throughout. The replacement is `DelayCe` (§2A.3.6). Write the noiseless test first.

Boxed insight — delete the mechanism; if the number doesn't move, it was never the mechanism. Three accepted explanations were falsified in a row by removing the impairment each depends on: "dense QAM can't hold coherence on HF" died against a noiseless in-CP two-ray channel (#685); "notch smearing" died at 60 dB SNR — a noise-enhancement mechanism must shrink with SNR, and the selective-vs-flat gap was 0.50 at 32 dB and 0.51 at 60 dB (#688); "the channel estimate lags a moving channel" died when `smooth_ce` was temporarily ablated and the flat-fade numbers came back bit-identical (#689) — the smoother stays in the shipped demod; it just was not the mechanism. The sync was. Run the ablation *before* building the fix the explanation implies.

Boxed insight — an uncoded-BER win is not a win. SC-FDMA's iterative block DFE halved uncoded BER on a static notch and moved coded frame success by **zero**: iterative feedback trades average residual for *confidently-wrong bits*, which is exactly what destroys soft FEC. Its own model noise variance was 90× optimistic — the feedback error correlates with the noise it is subtracted

from. Built, measured, reverted (`docs/dev/research/scfdma-improvements.md`, *Rejected — P7*). Always take the coded number.

Boxed insight — soft combining does not dominate plain retry; take the union. Summing HARQ attempts wins when every attempt is partially ruined and they carry complementary information; it *loses* when one attempt is simply clean and the sum dilutes it. Measured on `moderate_f1`, SCFDMA52 at 20 dB: plain retry 0.97, combining alone 0.95. `receive_with_llr_combining` therefore decodes each attempt standalone before falling back to the MAP sum — one extra RS decode over LLRs already in memory, and success becomes a strict superset of both (SCFDMA52-16QAM at 28 dB: 0.43 / 0.48 → **0.67**, PR #694). Deep-fade *outage* is what limits SC-FDMA on HF, and diversity is the only lever that touches outage.

Boxed insight — code rate is the last lever, not the first. Higher-rate FEC buys throughput by *spending* SNR. Measured on SC-FDMA (AWGN, 62-byte payloads, 90 % frame-success floors): `LdpcHighRate` (r ≈ 8/9) costs +4…+8 dB of floor over `SoftConcatenated` (r ≈ 0.44) for 2.03× the rate — e.g. SCFDMA52-16QAM 7 → 14 dB, SCFDMA52-64QAM 13 → 21 dB (`profile.rs` comment table). ~6 dB for 2× the rate is a worse trade than climbing one modulation order (8PSK → 16QAM buys 1.33× for ~2 dB), so a rate swap on a rung that still has a denser constellation above it *loses* throughput. LDPC earns rungs only at the ladder's top, where 64QAM is already the densest constellation available.

2A.9 The transmit envelope: AGC, CE-SSB, PAPR and the limiter

2A.9.1 Receiver AGC

`crates/openpulse-dsp/src/agc.rs`: an exponential-envelope loop modelled on liquid-dsp's `agc_crcf` — a smoothed output-power estimate drives a log-domain multiplicative gain update. Engine defaults are `Agc::new(0.3, 0.02, 40.0)` — target RMS 0.3 (headroom below ±1.0), bandwidth 0.02, ±40 dB symmetric gain clamp — overridable at runtime via `ModemEngine::configure_agc`. Two design points matter: `lock()`/`unlock()` freeze the gain during burst processing so a mid-frame gain change cannot corrupt soft-decision scaling; and placement is *inside the plugin demodulation chain* (AGC → symbol timing → carrier loop), the placement fielded HF modems use — not on the raw capture buffer, whose long leading silence would ramp the gain to its clamp before the burst arrives. Squelch/channel-busy detection stays in `DcdState`; the AGC only normalises level. Gates: `agc_amplitude_sweep` (decode is level-invariant with AGC on or off, and the AGC tracks level), plus the `agc_blocks_processed` tripwire counter (§2A.12.1).

2A.9.2 CE-SSB — controlled-envelope conditioning, and where it is allowed

CE-SSB (after David L. Hershberger W9GR, "Controlled Envelope Single Sideband", QEX Nov/Dec 2014) limits the complex envelope so average power can be raised at fixed peak (PEP), using a look-ahead "peak stretcher" so the limiting itself does not overshoot: `peak_stretch_gain(env, level, lookahead)` divides by the windowed-max envelope over ±lookahead samples, so the gain ramps down *before* a peak (`crates/openpulse-dsp/src/cessb.rs`). The engine operates at `level = 2.0 × rms_env` with a 16-sample lookahead, and rescales the conditioned frame back to the original peak so the freed headroom becomes average power at the same PEP (`cessb_condition_tx`, `engine.rs`). The headline metric is `papr_db = 20·log10(peak/rms)`: the average-power gain available at fixed peak equals the reduction in this value.

The gate is `ModemEngine::cessb_benefits(mode)`, and it returns `true` **only for `OFDM16` and `OFDM52`** — the QPSK-subcarrier OFDM modes. Every exclusion was decided by end-to-end decode through the real engine-plus-channel path, not synthetic raw BER (`engine.rs` doc block):

1. **All SC-FDMA is excluded.** Despite the subcarrier structure it is a single-carrier FDM waveform, low-PAPR by construction, so CE-SSB recovers only ~⅓ of OFDM's average-power gain (2.6 vs 8.5 dB at the operating point on the 64QAM rung) while its EVM alone injects ~0.5 % raw BER — collapsing SCFDMA52-{32,64}QAM decode from 30/30 to 5/30 through AWGN at 35 dB.
2. **OFDM at 16QAM and above is excluded.** OFDM52-32QAM 0/20 and -64QAM 3/20 with CE-SSB on, versus 20/20 off (soft FEC, AWGN). 16QAM is the instructive marginal case: it survives easy AWGN but breaks on the fading path — OFDM52-16QAM soft-FEC on Watterson good-F1: 0/16 on, 16/16 off.
3. **OFDM52-8PSK is excluded**: a marginal-SNR sweep goes 12/12 → 0/12 with CE-SSB on; peak-fair measurement shows its BER going 0.0000 → 0.0026.

The principle, quoted from the source: "CE-SSB trades in-band EVM for average-power gain, and that trade only wins where the envelope is high-PAPR *and* the decision margins are loose. QPSK-subcarrier OFDM sums ~52 carriers into a near-Gaussian envelope that rarely nulls hard, so envelope limiting costs almost no EVM; higher-order (8PSK/QAM/APSK) subcarriers and single-carrier QAM transit near the constellation origin, where the envelope passes through zero and the instantaneous phase goes discontinuous." The method note attached to it repeats the playbook: an earlier claim that CE-SSB helped the higher-order OFDM modes measured raw BER at a fixed operating point and missed the acquisition/decode failure on the real path — validate FEC-protected modes with their FEC, against the fading channel. Measurements live in `openpulse-linksim/tests/cessb_ab.rs` and `tests/cessb_power_evm.rs`.

2A.9.3 PAPR across the mode families

Representative measurements from `docs/mode-fec-ladder.md` §7 (simulator/rig measurements, not a specification; the occupied-bandwidth column is that document's measured quantity, **not** the plugins' `occupied_bandwidth_hz()`, which deliberately over-estimates as `2 × baud` for single-carrier modes):

Mode	Gross bps	Occ. BW (Hz)	~SNR floor (dB)	PAPR (dB)
BPSK250	250	275	5	4.2
QPSK500	1 000	550	11	4.2
8PSK500	1 500	550	14	4.2
64QAM500	3 000	550	26	6.3
QPSK2000-RRC	4 000	2 700	11	6.6
64QAM2000-RRC	12 000	2 700	~30	7.4
OFDM16	~889	625	8	11.7
OFDM52	~2 889	2 031	11	12.0
SCFDMA52	~2 889	2 031	11	12.1
SCFDMA52-16QAM	~5 778	2 031	16	12.7
SCFDMA52-64QAM	~8 667	2 031	28	12.2

The finding to absorb: SC-FDMA's textbook PAPR advantage is **not realised here** — its measured ~11–12 dB equals OFDM's. The root cause was itself corrected by a later ablation (use this corrected version, not the original hypothesis): about three quarters of the recoverable PAPR is simply **pilot count** — 13 equal-phase pilot cosines peak together — and only ~0.5 dB is the localized contiguous mapping; contiguous data *with* 13 pilots recovers ~0 dB. The residual ~10 dB (against a true single carrier's ~6–7 dB) is the real-valued-passband + rectangular-LFDMA ceiling: this SC-FDMA is a real passband signal with Hermitian symmetry about a 1500 Hz centre, and the ~3 dB real-bandpass penalty floors it above textbook complex-baseband SC-FDMA. The clean realisation of the pilot insight is `SCFDMA52-P2` — Zadoff–Chu pilot phases decorrelate the comb without dropping any pilot: envelope-CCDF@1e-3 measured 8.85 dB → 6.70 dB at identical geometry and rate, retaining full frequency-selective channel estimation. The decision recorded in `docs/mode-fec-ladder.md` §7: the SC-FDMA PAPR redesign was dropped; OFDM higher-order is the HF high-throughput path, and OFDM's ~12 dB PAPR is managed by drive backoff (leveling), QPSK-only clipping, and the 0.9 peak-normalisation of dense frames (§2A.3.5).

2A.9.4 The TX soft limiter

`openpulse_audio::tanh_limit(samples, threshold)` is a memoryless soft limiter:

```rust
pub fn tanh_limit(samples: &mut [f32], threshold: f32) {
    if threshold <= 0.0 { return; }
    let inv = 1.0 / threshold;
    for s in samples.iter_mut() { *s = threshold * (*s * inv).tanh(); }
}
```

Threshold ≤ 0 is a no-op, and the engine default is 0.0 — disabled. It is applied immediately before the audio write. Tests pin the three properties that matter: the peak is bounded, zero threshold is a no-op, and small signals pass approximately linearly (`tanh(s/t)·t ≈ s` for `|s| ≪ t`). A documented gap: there is no IQ-domain equivalent yet, so an IQ transmit path relies on a hardware/PA limiter (`engine.rs`).

2A.10 FEC mathematics

2A.10.1 The mode set

`crates/openpulse-core/src/fec.rs` defines ten `FecMode` variants; negotiation picks the strongest mode present in both peers' lists, by `strength()`:

Mode	Code	Rate	Input	Strength
`None`	—	1.00	—	0
`LdpcHighRate`	rate ≈ 8/9 LDPC (PEG, min-sum)	~0.89	soft	1
`Rs`	RS(255,223), t=16	0.875	hard	2
`RsInterleaved`	RS + stride interleaver	0.875	hard	3
`Concatenated`	Conv(1/2, K=3) inner + RS outer	~0.44	hard	4
`ShortRs`	short-block RS, no padding/prefix	—	hard	5
`RsStrong`	RS(255,191), t=32	0.749	hard	6
`SoftConcatenated`	soft-Viterbi (K=7) inner + RS(255,223)	~0.44	soft	7
`Ldpc`	rate-1/2 LDPC (min-sum BP)	0.50	soft	8
`Turbo`	rate-1/3 PCCC	0.33	soft	9

`LdpcHighRate` is deliberately the weakest non-`None` option for negotiation — it carries the least redundancy of any mode, so a peer falls back to it only when no more-protective mode is mutual (enum comment). The enum doc credits `SoftConcatenated` with ~5 dB over hard `Concatenated`; `docs/mode-fec-ladder.md` §2 gives the general soft-versus-hard rule of thumb as ~3–4 dB.

2A.10.2 Reed-Solomon

GF(2^8), block size 255 always. The standard codec appends 32 ECC bytes per block (t = 16, corrects 16 byte errors ≈ 6.3 % of a block); `FecCodec::strong()` appends 64 (t = 32, ≈ 12.5 %). Wire layout: a 4-byte big-endian original-length prefix, data padded and split into `data_per_block` chunks, each encoded to 255 bytes. `rs_block_count` mirrors the blocking exactly and is what `free_rs_strengthening` (§2A.8.3) uses to decide whether t = 32 is free.

`ShortFecCodec` is the answer to the short-payload waste: no padding, no prefix, output exactly `payload + ecc_len` bytes. At the default `SHORT_FEC_ECC_LEN = 8` (t = 4 per the doc comment), a 5-byte FSK4-ACK frame becomes 13 bytes instead of a 255-byte block. The engine's `FecMode::ShortRs` data path uses `with_ecc_len(32)`, so a small data frame travels as `Frame(payload) + 32` bytes (≈ payload + 42) instead of 255. Limitation, stated in the sharp-edges record: only plugins whose demodulator emits the exact transmitted byte count are supported — loopback and well-framed half-duplex paths, not the padded OFDM/SC-FDMA modes. Gates: `short_fec_data_frame_engine_loopback`, `short_fec_data_frame_rejects_oversized_payload`.

2A.10.3 Interleaving — the theory and its operating point

The stride interleaver's depth is designed against the Gilbert-Elliott moderate profile: `DEFAULT_INTERLEAVER_DEPTH = 5 × 20-symbol mean burst = 100` (`fec.rs`). The design arithmetic: with 10 RS blocks (2550 encoded bytes), a burst of 100 distributes at most `⌈100 × 255 / 2550⌉ ≈ 10` errors per block — inside the 16-byte correction capacity; and when paired with a convolutional code of constraint length k, depth must be ≥ 2(k−1) so burst fragments span distinct constraint windows. The theory is correct — **and the `hpx_hf` operating point makes it inapplicable**, because at those frame sizes there is exactly one RS block and `RsInterleaved` measures identically to `Rs` (§2A.8.3, Rule 2). Both facts belong side by side: a mechanism can be sound and still inert at the operating point in force. Note that the multi-block case is sound in principle but, as `docs/mode-fec-ladder.md` §2 states plainly, **has not been measured in this repo** — the interleaver's benefit above 223 bytes is theory, not a result.

2A.10.4 Convolutional + Viterbi

`conv.rs`: rate-1/2, K = 3 (4 states), generators G = {7, 5} octal, 2 tail bits, hard-decision Viterbi, stateless, with the same 4-byte length-prefix wire convention as `FecCodec`. The Phase-3.2 evaluation (`tests/fec_comparison.rs`) is a textbook illustration of error-model dependency: at a channel BER of 1 % (random errors, AWGN regime), RS post-decode BER was **0.497** — the random errors exceed the 16-byte-per-block capacity and RS collapses — versus ConvCodec's **0.0004**, at 3.8× the CPU. The decision: ConvCodec is an optional alternative for AWGN-dominant paths; RS plus interleaving remains the default for HF burst-error profiles. The same code is a thousand times better or useless depending on whether the errors arrive scattered or in bursts. (Note: `SoftConcatenated`'s inner code is a K = 7 *soft* Viterbi per the `FecMode` doc — a different decoder from this K = 3 hard one.)

2A.10.5 LDPC

`ldpc.rs`: two presets, both systematic `H = [H_s | I_m]` so encoding is a single XOR pass; decoding is min-sum belief propagation for up to 50 iterations. `LdpcCodec::new()` is k = 1024, n = 2048, rate 1/2, PEG-constructed with regular variable degree 3; `high_rate()` is k = 1024, n = 1152, rate ≈ 8/9, with a documented waterfall ≈ 4 dB Es/N0 (a random parity structure at this rate is useless — PEG is what makes it work). PEG (Progressive Edge-Growth) places each information variable's edges one at a time onto the check that is farthest away in the current graph (BFS), breaking ties toward the lowest-degree check — maximising local girth, which is what belief propagation's independence assumption feeds on. The identity block earns its place twice: it makes encoding one XOR pass, and it gives each parity bit a degree-1 check connection that anchors BP convergence (module comments). Block limits for the rate-1/2 codec are named constants: `LDPC_MAX_INFO_BYTES = 128` (k = 1024 bits) in, `LDPC_CODEWORD_BYTES = 256` (n = 2048 bits) out; the high-rate codec takes the same 128 information bytes and emits 1152 bits.

2A.10.6 Turbo

`turbo.rs`: rate-1/3 parallel-concatenated convolutional code, 3GPP-style, with the QPP (quadratic permutation polynomial) interleaver table `(K, f1, f2)` and Max-Log-MAP BCJR decoding (8 iterations

per the `FecMode::Turbo` doc comment). The wire layout packs `sys[K] ‖ par1[K] ‖ par2[K]`, where the K-bit block carries a u16-LE length, the data, and a CRC-16, zero-padded to K bits. Its positioning in the enum doc: higher coding gain than LDPC for short blocks (≤ 256 bits).

2A.10.7 Pairing rules

From `docs/mode-fec-ladder.md` §5, hardware-validated combinations that do not make sense: any dense mode (16QAM and up) with `None` or bare `Rs` (RS's 6.3 % capacity is below what those modes leave on a realistic channel — soft FEC closes 8PSK, and narrowing plus soft FEC closes 16QAM/32QAM); single-carrier 64QAM on a marginal link (needs ~25–26 dB and a tight clock, and below that no FEC rescues it economically); and `Turbo`/`Ldpc` on a clean high-SNR link (a 0.33/0.5 code rate throws away throughput you did not need to spend). And the standing playbook rule: **test FEC-protected modes with their FEC** — dense modes only ever run protected, so a no-FEC loopback is an unrealistic bar, and soft FEC (~+6 dB) was the bigger lever the loopback had never exercised.

2A.11 The `hpx_hf` ladder as shipped

The full adaptive-rate machinery is a protocol topic, but the ladder is where every measurement in this chapter lands, so here it is as the code builds it. Source of truth: `SessionProfile::hpx_hf()` (`crates/openpulse-core/src/profile.rs`); `initial_level = SL2`, `nack_threshold = 3`, admission to SL14 gated behind a prior SNR upgrade candidate (`ack_up_requires_snr_candidate_at = Some(SL14)`). Ceilings follow one rule — a uniform +2 dB hysteresis over the next rung's floor, `ceiling(L) = floor(L+1) + 2` — so every rung dwells the same margin before climbing; SL14 has none.

SL	Mode	FEC	~Net bps	Floor (dB)	Ceiling (dB)	Floor scale
SL1	MFSK16	Rs	~9	—	5	—
SL2	BPSK31	Rs	27	3	6	true SNR
SL3	BPSK63	Rs	54	4	6.5	true
SL4	BPSK100	Rs	87	4.5	7	true
SL5	BPSK250	Rs	219	5	9	true
SL6	QPSK250-D	Rs	437	7	11	true
SL7	OFDM52	SoftConcatenated	1264	9	12	plugin
SL8	OFDM52-8PSK	SoftConcatenated	1895	10	14	plugin
SL9	OFDM52-16QAM	SoftConcatenated	2527	12	16	plugin
SL10	OFDM52-32QAM	SoftConcatenated	3159	14	18	plugin
SL11	OFDM52-64QAM	SoftConcatenated	3790	16	20	plugin
SL12	OFDM52-16QAM	LdpcHighRate	5141	18	21	plugin
SL13	OFDM52-32QAM	LdpcHighRate	6426	19	22	plugin
SL14	OFDM52-64QAM	LdpcHighRate	7710	20	—	plugin

Every structural feature of this table was derived in this chapter: SL1 is the non-coherent sub-floor rung reached via the deep-fade fallback path (§2A.3.4); SL2–SL5 are differential-by-NRZI BPSK, every rung coded (§2A.8.3, Rule 1), with `free_rs_strengthening` upgrading `Rs` to `RsStrong` on the wire whenever it is block-count-free; SL6 is differential QPSK because coherent QPSK is dead on a fade (§2A.8.2); SL7–SL11 change family to OFDM because phase margin runs out (§2A.8.4); and SL12–SL14 re-use SL9–SL11's modulations at r ≈ 8/9 LDPC — MODCOD pairs, because above 64QAM code rate is the only lever left (§2A.8.5, last box). The floors live on two scales by physical necessity (§2A.7.5). The "SNR floor" column's units per family are pinned by `snr_scale_boundary`; the whole mode/FEC/floor/ceiling table is pinned against `docs/mode-fec-ladder.md` by `ladder_doc_matches_profile`, and every rung's fade viability by `hpx_hf_rungs_survive_fade`.

(One documentation footnote, in the spirit of this book's ground rules: `hpx_hf`'s ASCII comment table in `profile.rs` does match the code rung for rung, but the *prose* around it has drifted. Its doc-comment header still advertises "SL2–SL19" and says high-rate LDPC "appears as SL15–SL17"; a later comment says "SL2–SL5 carry `RsStrong` (t=32)" where the code assigns `Rs`, with `RsStrong` applied opportunistically per frame by `free_rs_strengthening`; and the comment above `snr_floors` says "SL7 (OFDM52) = 10" where the code assigns 9.0. `ladder_doc_matches_profile` has two tests — `mode_fec_ladder_doc_table_matches_hpx_hf_profile` pins the code against `docs/mode-fec-ladder.md`, and `profile_comment_table_matches_the_executable_floors` pins the ASCII

comment table — so the *tables* cannot drift. The surrounding prose is checked by nothing, which is exactly why it did. The table above reproduces the code.)

For contrast, the original `hpx500` profile (BPSK31 → QPSK500, floors 3/4/5/9/11, no per-level FEC) states its derivation as "3 dB headroom above the Eb/N₀ required for 10^{-3} BER" — a code comment with no test or sweep behind the specific figures that we could verify, and in any case an AWGN-derived calibration, which §2A.8.3 showed is exactly the class of ladder that does not survive contact with a fade.

2A.12 Cross-cutting engineering rules

2A.12.1 The seam rule

Captured audio reaches the demodulator by two distinct routes: the `receive*` family (`stage_capture_input` → `receive_from_samples`) and the daemon's streaming path (`accumulate_capture` → `accumulate_routed` — the one the running daemon's `rx_ticker` actually uses). They funnel through exactly one shared seam, `route_audio_stage(PipelineStage::InputCapture)` (~19 call sites in `engine.rs`). Any front-end transform that must run on *every* receive — the notch bank, the AGC — belongs at that seam, never in one caller. The original notch bug put the transform in `stage_capture_input` only: every `receive`-family test passed, and the daemon never ran it. Two tripwire counters — `notch_blocks_processed()` and `agc_blocks_processed` — stay at zero if an enabled feature never runs on a path, and the gate `ota_production_capture_path` exercises the production entry. The checklist that came out of it: trace top-down from the binary; place the transform at the single seam; never claim "covers all paths" from a callers-grep — prove it with a test that fails without the wiring; add a runtime tripwire; and test through the production entry, not only the convenience harness.

2A.12.2 Rebuild both ends

For any loopback experiment, rebuild both ends. The preamble sequence and frame geometry are shared protocol, and a one-sided rebuild fails silently with "invalid magic" — a lesson the differential

QPSK implementation encodes structurally by deriving its phase reference from `preamble_symbols()` rather than a constant, so a preamble change moves both ends together.

2A.12.3 Reproducing this chapter's numbers

Every gate named in this chapter runs without audio hardware:

```
cargo test --workspace --no-default-features
cargo test -p openpulse-channel --lib bursts_span_whole_symbols_with_mean_one_over_pbg
cargo test -p openpulse-channel --lib continuous_fade_correlates_across_calls
cargo test -p openpulse-dsp additive_snr
cargo test -p openpulse-modem --test bpsk_snr_tracks_a_fade
cargo test -p openpulse-modem --test snr_scale_boundary
cargo test -p openpulse-modem --test hpx_hf_rungs_survive_fade
cargo test -p openpulse-modem --test qpsk_differential_fading
cargo test -p openpulse-modem --test agc_amplitude_sweep
cargo test -p openpulse-modem --test llr_calibration
cargo test -p openpulse-core --test ladder_doc_matches_profile
cargo test -p openpulse-core --test success_based_climb
cargo test -p openpulse-core free_rs_strengthening
cargo test -p mfsk16-plugin
cargo test -p fsk4-plugin --test fsk4_integration
cargo test -p js8-plugin --test snr_sweep gate_at_minus_18_db
cargo test -p openpulse-linksim goodput_gate
```

The multi-hour research sweeps behind the measured decode-rate tables are not CI gates — they live in the research documents cited throughout (`docs/dev/research/robust-narrowband-measurement.md`, `docs/dev/research/scfdma-improvements.md`, the comment blocks in `profile.rs`) with their channel, SNR, FEC and payload conditions attached. Where this chapter quotes a number, that is where it came from; and, one final time: every fading figure is a Watterson-simulator result, awaiting on-air validation.

Chapter 2, Part B — Cryptography and trust

OpenPulseHF carries more cryptography than most amateur digital modes: Ed25519 signatures on the handshake, an X25519 key agreement, an HMAC-authenticated rate-control ACK, an ML-DSA/ML-KEM post-quantum layer, a Noise-encrypted control socket and an Argon2-protected keystore. It is easy to misread that list as "an encrypted mode". It is not, and cannot be. One rule organises everything else — **on the air, cryptography authenticates; it never conceals** — and this part works from that rule through the primitives, the handshake, the trust model, the post-quantum design with its real on-air cost, key management, and an explicit list of what is *not* protected.

2B.1 The regulatory frame: sign, don't cipher

For the operator, the rule of thumb is simple: everything OpenPulseHF transmits over RF is readable by anyone with the (public) protocol specification. Cryptography adds a *tag* that proves who sent a frame and that nobody altered it — the same way a signed letter is still a readable letter.

The legal basis, quoted from `docs/regulatory.md`:

> **§97.309(a)(4):** Unspecified digital codes are permitted provided the control operator makes the necessary technical information available to the FCC upon request, and provided the emission is not used to obscure the meaning of the message.

and the project's own statement of policy, from the same document:

> **Authentication is not encryption.** The protocol uses cryptography only to *authenticate* — to prove who sent a frame and that it was not altered — never to hide message

> content. Ed25519 signatures on the handshake, the peer descriptors, and relay envelopes, and the keyed MAC on the OTA rate-control ACK (E7), all leave the underlying message fully readable; they add a verifiable tag, not a cipher. The X25519 key agreement in the handshake derives a key used **only** for that ACK MAC, not to encrypt any content. Because nothing obscures the meaning of the message, these mechanisms are consistent with §97.309(a)(4) and §97.113(a)(4) (no messages encoded to obscure meaning). Operators in jurisdictions that restrict even authentication should verify local rules.

The code restates the same contract where the temptation to drift would be greatest — the session-key module that performs an actual Diffie–Hellman exchange (`crates/openpulse-core/src/session_key.rs`):

```
//! This is **authentication, not encryption**: the shared secret keys a MAC over ACK content that
//! stays in the clear, so it is compatible with amateur-radio rules that forbid obscuring meaning.
```

The clean dividing line, verified across the whole codebase: **every cryptographic operation that crosses the antenna produces a detached authenticator over cleartext.** Real ciphers appear in exactly two places, and both stay off the air:

Surface	Cipher	Why it is permitted
Daemon ↔ client control socket	ChaCha20-Poly1305 via Noise (`openpulse-linksec`)	A wired/local link between the operator's own machines — not an amateur transmission
On-disk keystore (`openpulse-keystore`)	ChaCha20-Poly1305, key from a master password	The operator's own disk

The control channel is the interesting asymmetry: it carries commands that key the transmitter (`PttAssert`, `SendMessage`, …), so an *unauthenticated* control port would let anyone who reaches the socket transmit under your callsign. Off-air, encryption is legal and correct, so the control link gets full mutual authentication and encryption (§2B.8) while the RF payloads it triggers stay readable. Compliance also cuts the other way: `docs/regulatory.md` requires the callsign in handshake frames and in periodic in-band ID (§97.119(a), ≤ 10 minutes), noting that signed identity records

satisfy identification only "if the callsign is included in plaintext in the signed payload" — which the CONREQ's signed `station_id` field is.

2B.2 Primitive inventory

All verified in `Cargo.toml` manifests (workspace root and `crates/openpulse-core/Cargo.toml`):

Primitive	Crate (version)	Used for
Ed25519	`ed25519-dalek` 2.0	Handshake, transfer manifests, peer descriptors, relay-envelope origin signatures, QSY frames, file-transfer offers, PKI trust bundles
X25519 ECDH	`x25519-dalek` 2.0	Ephemeral key agreement for the OTA-ACK MAC key
HKDF-SHA256	`hkdf` 0.12	Key derivation from the ECDH secret
HMAC-SHA256	`hmac` 0.12	24-bit truncated tag on the authenticated ACK
SHA-256	`sha2` 0.10	Payload hashes everywhere
ML-DSA-44 (FIPS 204)	`ml-dsa =0.1.0-rc.11`	Post-quantum signatures
ML-KEM-768 (FIPS 203)	`ml-kem =0.3.0`	Post-quantum key encapsulation
Noise (`NNpsk0`, X25519 + ChaCha20-Poly1305 + BLAKE2s)	`snow` 0.9	Control-channel security
Argon2 (crate default, documented as Argon2id)	`argon2` 0.5	Keystore master-password KDF
ChaCha20-Poly1305	`chacha20poly1305` 0.10	Keystore AEAD
CRC-16/CCITT (0x1021, init 0xFFFF)	`openpulse-core::frame`	Data-frame integrity (not security)
CRC-8/SMBUS (0x07)	`openpulse-core::ack`	Legacy ACK integrity; replaced by the MAC in authenticated mode

One engineering note: the two post-quantum crates are the only exact-pinned (`=`) dependencies in the list. Both are pre-1.0 (`ml-dsa` is a release candidate); the pin is deliberate, because their APIs are not yet stable.

2B.3 The signed handshake: CONREQ and CONACK

2B.3.1 Wire framing

Both handshake frames share one outer envelope (`crates/openpulse-core/src/handshake.rs`):

`decode` rejects a buffer under 9 bytes, a wrong magic, a version other than 1, and any length field that does not *exactly* equal the remaining byte count — trailing garbage is a decode error, not silently ignored. A minimal CONREQ (callsign `W1AW`, one signing mode, empty session id, no compression or FEC lists, no optional fields) encodes to **591 bytes**; adding one compression algorithm, one FEC mode and a six-character session id takes it to **602**. Both measured in-process against `ConReq::create(...).encode()`. Expect a few bytes of variation run to run: `serde_json` renders `pubkey` and `signature` as arrays of decimal numbers, so the encoded length depends on the actual byte values of the key and signature. The wire spec's "~530 B" at `docs/dev/design/protocol-wire-spec.md:104` is low. That exceeds the 255-byte `Frame` payload limit, which is why the daemon transports handshakes via SAR fragmentation (§2B.6.4).

The signed CONREQ body carries: `station_id`, the 32-byte Ed25519 `pubkey`, offered `signing_modes`, the cleartext `session_id`, `supported_compression`, `supported_fec_modes`, `station_grid`, the OTA-ladder `profile_name`/`profile_fingerprint`, a replay-freshness `timestamp_ms`, and an ephemeral X25519 `kex_pubkey` (§2B.5). The CONACK mirrors it with singular `selected_*` fields plus the echoed `session_id`. Every field except the trailing 64-byte `signature` is inside the signed region.

Two details are load-bearing:

- **The "canonical JSON" is deterministic, not key-sorted.** The wire spec and `docs/features.md` say the signature covers JSON "with keys sorted recursively". The code does something simpler: `ConReq::canonical_bytes` rebuilds a private body struct and calls `serde_json::to_vec`, which emits fields in *declaration order*. Signer and verifier build the same struct, so the bytes are identical — determinism is the property the security argument needs, and it holds. Recursive key sorting exists only in the PKI service (§2B.8.4).
- **Optional fields are omitted, not zero-filled.** `#[serde(skip_serializing_if = ...)]` on the grid, timestamp, fingerprint and kex-key fields means a frame carrying none of them is byte-identical to the pre-feature wire format, so old

signatures still verify. Test: `empty_grid_conreq_is_byte_identical_to_legacy` (`handshake.rs`).

2B.3.2 Verification order, and the lesson inside it

`verify_conreq` runs, in order:

1. **Signature** — rebuild the canonical bytes, verify Ed25519 against the frame's **own** embedded `pubkey`. Failure: `InvalidSignature`.
2. **Freshness** — check the signed `timestamp_ms` (deliberately after step 1: the timestamp is inside the signed body, so it cannot be altered without breaking the signature).
3. **Key binding** (`bind_frame_key`) — if the trust store already holds a key for this `station_id`, the frame's key must equal it. Failure: `PublicKeyMismatch`.
4. **Trust lookup** — `trust_store.trust_level(station_id)`, then map to a certificate source (`Full` ⇒ `OutOfBand`, everything else ⇒ `OverAir`).
5. **Policy evaluation** — `evaluate_handshake(...)` (§2B.4).

Step 3 is the fix for the audit's one CRITICAL finding (F1, `docs/dev/reviews/2026-07-15-handshake-trust-audit.md`). Before it, the classical path verified the signature against the frame's own key and then looked up trust *by callsign* — so any station could self-sign a CONREQ claiming a trusted callsign under its own fresh key and inherit that callsign's trust level. The general lesson: **a valid signature proves possession of a key, never an identity, until the key is bound to the name.** The asymmetry in `bind_frame_key` is deliberate: an *unknown* station has no stored key, passes the bind, and proceeds at `Unknown` trust — over-air trust-on-first-use. The bind only rejects a frame claiming a callsign the operator has already bound to a *different* key.

`verify_conack` additionally checks the session-id echo and that the responder's `selected_compression` and `selected_fec_mode` were actually offered in the CONREQ.

2B.3.3 Replay freshness

`Freshness { now_ms, max_skew_ms }` rejects a zero timestamp (`MissingTimestamp` — a legacy timestampless frame is refused when freshness is required) and any skew beyond the window

(`StaleTimestamp`), computed with `abs_diff` so a *future-dated* frame is rejected symmetrically. The daemon applies `HANDSHAKE_MAX_SKEW_MS = 120_000` (±120 s) to both inbound CONREQ and CONACK (`crates/openpulse-daemon/src/lib.rs`). Six inline tests cover fresh/stale/future/missing/skip/ stale-CONACK (`handshake.rs`, `fresh_conreq_within_window_is_accepted` through `stale_conack_is_rejected`). A historical note: the 2026-07-15 audit lists replay freshness as *deferred* ("a captured valid handshake replays"); the code has since implemented it, and the audit document was never updated — anyone describing this system from the audit alone describes July 2026, not v0.15.0.

2B.3.4 What the daemon actually does with a handshake

Inbound RF handshakes are evaluated under `PolicyProfile::Permissive` with a local minimum of `SigningMode::Normal` (`crates/openpulse-daemon/src/lib.rs`). The design intent, stated at the call site: the signature proves key possession, the trust classification is *recorded*, and an unknown first-seen peer is still allowed to connect. On this inbound path the handshake is an identity **label**, not an access **gate** — `verify_conreq` applies no minimum-trust floor (that floor is `begin_secure_session`'s, on the *outbound* session path, §2B.4.3), and the only production data path that consults the verified-peer record is file transfer, itself off by default with `require_verified_peer = true`.

One genuine gate does sit here: if the daemon has no valid local callsign (`runtime_state.local_callsign_valid()` false), it logs a warning and **refuses to transmit a CONACK at all** — replying would key the transmitter, and automatic identification is impossible without a callsign (§97.119; audit finding E6).

The station's long-term identity is a 32-byte Ed25519 seed at `[station] identity_key_path` (default `~/.config/openpulse/identity.key`). New keys come from `OsRng`, written with `create_new(true).mode(0o600)` — atomically, so the file never exists under a looser umask, with an `AlreadyExists` branch that re-reads a concurrent winner's file. Existing files are permission-validated (owner-only, REQ-SEC-CTL-05) and must be exactly 32 bytes. One posture wart: if the key *fails to load*, the daemon warns and falls

back to a **random ephemeral seed** (`server.rs`) — failing open into an unrecognisable identity rather than refusing to start, the opposite of the trust store's fail-closed load (§2B.4.4).

2B.4 The trust model

2B.4.1 Two enums, not one

Repo documentation frequently conflates two distinct types in `crates/openpulse-core/src/trust.rs`; the code does not:

- `PublicKeyTrustLevel` — what the *operator* asserts about a key: `Full`, `Marginal`, `Unknown`, `Untrusted`, `Revoked`.
- `ConnectionTrustLevel` — what a *connection* earns, ordered ascending: `Rejected < Low < Unverified < Reduced < PskVerified < Verified`.

`classify_connection_trust(key_trust, certificate_source, psk_validated)` maps between them. The full table as implemented:

Key trust	Cert source	psk_validated	Connection trust	Reason code
Untrusted / Revoked	any	any	Rejected	policy_rejected
Full	OutOfBand	any	Verified	verified_out_of_band
Full	OverAir	true	PskVerified	psk_validated_over_air_certificate
Full	OverAir	false	Reduced	over_air_certificate_without_psk
Marginal	OverAir	true	PskVerified	psk_validated_over_air_certificate
Marginal	other	any	Reduced	over_air_certificate_without_psk
Unknown	OutOfBand	any	Unverified	key_trust_unknown
Unknown	OverAir	true	PskVerified	psk_validated_over_air_certificate
Unknown	OverAir	false	Low	unknown_key_over_air_certificate_without_psk

In practice the handshake reaches only four outcomes. `cert_source_for_trust` maps a `Full` key to `OutOfBand` and everything else to `OverAir`, and all four `evaluate_handshake` call sites in `openpulse-core` (`verify_conreq`, `verify_conack`, `verify_pq_conreq`, `verify_pq_conack`) hardcode `psk_validated = false` — so **`PskVerified` is unreachable from the CONREQ/CONACK path**. (The fifth call site, `ModemEngine::begin_secure_session`, takes the flag from its `SecureSessionParams`; every production caller — the daemon's `ConnectPeer` and the ARDOP TNC's `CONNECT` — passes `false` there too.) A connection ends up `Verified` (operator marked the key `Full`), `Reduced` (`Marginal`), `Low` (unknown key, the TOFU case), or errors out (`Untrusted`/`Revoked`).

2B.4.2 Signing modes and negotiation

`SigningMode` with its strength ordering (`mode_strength`, `trust.rs`):

Mode	Strength	Meaning
`Relaxed`	1	weakest label
`Psk`	2	declared; no PSK challenge is implemented anywhere in the tree (unverified beyond the enum)
`Normal`	2	Ed25519
`Paranoid`	3	declared; no distinct behaviour implemented (same caveat)
`Pq`	4	ML-DSA-44 only
`Hybrid`	5	Ed25519 + ML-DSA-44

`select_signing_mode` intersects the local profile's allow-list with the peer's offered modes (`NoMutualSigningMode` if empty), sorts by descending strength, takes the strongest, and rejects it (`WeakSigningModeRejected`) if it falls below the local minimum. Strongest-wins, with a local floor.

2B.4.3 What policy profiles actually gate

`allowed_signing_modes`:

Profile	Allowed modes
`Strict`	`Normal, Paranoid, Pq, Hybrid`
`Balanced`	`Normal, Psk, Relaxed, Pq, Hybrid`
`Permissive`	`Normal, Psk, Relaxed, Pq, Hybrid`

Note where the trust gate is *not*: `evaluate_handshake` in `trust.rs` uses the profile **only** to pick a signing mode, and its sole trust-based rejection — `Untrusted`/`Revoked` keys classifying as `Rejected` — is profile-independent. So `verify_conreq`/`verify_conack` apply no per-profile trust floor at all.

The floor that `docs/features.md` documents (Strict = Verified, Balanced = PskVerified, Permissive = Reduced) is real, but it lives one layer up, in the modem crate:

```rust
// crates/openpulse-modem/src/engine.rs
fn minimum_trust_for_profile(profile: PolicyProfile) -> ConnectionTrustLevel {
    match profile {
        PolicyProfile::Strict => ConnectionTrustLevel::Verified,
        PolicyProfile::Balanced => ConnectionTrustLevel::PskVerified,
        PolicyProfile::Permissive => ConnectionTrustLevel::Reduced,
    }
}
```

`ModemEngine::begin_secure_session` calls `evaluate_handshake` and *then* rejects the session if `handshake.trust.decision < minimum_trust_for_profile(...)`. That path is live — the daemon's `ConnectPeer`, the ARDOP TNC's `CONNECT`, and the CLI `session`

command all go through it. The engine's default profile is `Balanced`. Read the two layers together: verifying a CONREQ off the air applies no trust floor; *starting a session* applies the documented one.

One genuine correction to `docs/features.md`: `Balanced` and `Permissive` allow **identical** signing-mode sets; the only implemented distinction in `allowed_signing_modes` is that `Strict` excludes `Psk` and `Relaxed` (test: `strict_profile_rejects_psk_only_peer`). They differ only in the minimum-trust floor above.

2B.4.4 The trust store and its operator surface

The `TrustStore` trait is two methods — `pubkey_for(station_id)` and `trust_level(station_id)` (defaulting to `Unknown` for absent entries). The on-disk format (`crates/openpulse-core/src/trust_store_file.rs`):

```
{ "schema_version": "1.0.0",
  "records": [ { "station_id": "W1AW", "key_id": "<64 hex chars>", "trust": "full" } ] }
```

`key_id` is the hex-encoded Ed25519 verifying key; `trust` values are kebab-case `PublicKeyTrustLevel` names. Malformed records are skipped with a warning count; a missing file yields an empty store; loading validates file permissions first. But — audit finding T4 — a *configured* store that **fails to load** refuses daemon startup outright (`crates/openpulse-daemon/src/server.rs`: "refusing to start with an empty store that would drop revocations"). A store silently loaded as empty would forget every revocation it carried.

The operator surface is `openpulse trust show | explain | import | list | revoke | policy show|set` (`crates/openpulse-cli/src/cli.rs`), plus `openpulse diagnose handshake|manifest|session`. The QSY frequency-agility protocol consumes trust directly: `[qsy] allow_trustlevels` defaults to `["verified", "psk_verified"]` (`crates/openpulse-config/src/lib.rs`).

2B.5 Session keys and the authenticated rate-control ACK

2B.5.1 The attack, in the repo's own words

The rate-control ACK is a 5-byte frame on its own low-SNR waveform. Before hardening (audit E7), its only session binding was a 16-bit FNV-1a hash of the `session_id` — which rides *in the clear* in the CONREQ, so any listener could recompute the hash and forge ACKs "and drive a link's rate ladder" (`session_key.rs` module docs). The audit rated this LOW: rate-ladder manipulation, not an access bypass. The fix is a textbook lightweight design.

2B.5.2 Key agreement inside the signed handshake

`generate_kex_ephemeral()` produces an X25519 keypair; the public half rides in the Ed25519-**signed** handshake body (`kex_pubkey`), so a man-in-the-middle cannot substitute it without breaking the identity signature. Both sides compute `derive_ack_key(my_secret, peer_public)`: X25519 ECDH → HKDF-SHA256 with info string `"openpulse-ota-ack-key-v1"` → a 32-byte MAC key. Tests: `both_sides_derive_the_same_key`, `distinct_pairs_derive_distinct_keys` (which also checks a third party derives a different key). A peer advertising no `kex_pubkey` arms no MAC key — the link degrades gracefully to the legacy ACK.

2B.5.3 One frame, two layouts — still exactly 5 bytes

```
Legacy (CRC + public hash):              Authenticated (E7):
byte 0: type|has_rev|has_rec bits        byte 0: type|has_rev|has_rec bits   (content)
byte 1: session_hash hi  (FNV-1a)        byte 1: MAC[0]  ┐
byte 2: session_hash lo                  byte 2: MAC[1]  │ 24-bit truncated
byte 3: recommended_level|reverse_ack    byte 3: recommended_level|reverse_ack (content)
byte 4: CRC-8/SMBUS over bytes 0–3       byte 4: MAC[2]  ┘ HMAC-SHA256(key, b0‖b3)
```

The authenticated form (`AckFrame::encode_authenticated`, `ack.rs`) replaces the hash and CRC bytes with the first three bytes of an HMAC-SHA256 over the two content bytes. The consequences, each stated in the doc comment: the tag **subsumes the CRC** (a MAC detects corruption), it **replaces the anti-collision filter** (a co-channel session has a different key, so its ACKs fail the MAC), and `session_hash` is simply not carried. The engine acts on the last point: with a MAC key set, `expected_session_hash` filtering is bypassed entirely (`crates/openpulse-modem/src/engine.rs`).

Be honest about 24 bits: a blind forgery succeeds with probability $2^{-24} \approx 1$ in 16.7 million per attempt — a deliberate trade for keeping the frame at 5 bytes where airtime is the scarce resource. It stops a casual forger, not a determined attacker with unlimited on-air attempts. The repo claims no specific security level for it, and neither does this book.

The diversity-ACK path is authenticated too: `decode_ack_from_llr_copies_maybe_auth` applies the union rule (decode each of K soft copies standalone, MAP-sum only as fallback) with the keyed decode as the validity check. The measurement for the underlying K=3 scheme, at 400 trials (`docs/dev/research/robust-narrowband-measurement.md`, Finding 2):

Channel, SNR	single ACK (1× airtime)	K=3 union (3.8×)
`moderate_f1` −3 dB	0.66	**0.99**
`moderate_f1` 0 dB	0.90	**1.00**
`poor_f1` −3 dB	0.56	**1.00**
`poor_f1` 0 dB	0.92	**1.00**

So the union holds 0.99–1.00 at 3 dB *below* the floor, where a single 1.28 s ACK falls to 0.56–0.66. Note that the same document retracts an earlier, widely-quoted "the single ACK decodes only ~0.6 at the floor" — that was a 40-trial sampling artifact; at 400 trials the single ACK is 0.90/0.92 at 0 dB. The module docstring in `plugins/mfsk16/src/robust_ack.rs` still carries the old figure. These are **Watterson channel-simulator numbers, not on-air measurements** — true of all v0.13.0–v0.15.0 fade work.

Evidence chain: `tampering_an_authenticated_ack_fails_verification` (unit, `ack.rs`); `authenticated_ack_round_trips_and_forgery_is_rejected` (`crates/openpulse-modem/tests/ack_exchange_integration.rs`); and, composed with the sub-floor return channel, `authenticated_k3_subfloor_ack_round_trips_and_forgery_is_rejected` (`tests/mfsk16_arq_subfloor.rs`).

One adjacent fact to prevent a misreading: `trust.rs` also contains `derive_session_keys`, a fuller transcript-bound X25519+HKDF derivation. It has **no consumer** outside its own determinism test; the ACK MAC key from `session_key.rs` is the derivation that is actually wired and used.

2B.6 Signatures beyond the handshake

2B.6.1 Transfer manifests

`TransferManifest` (`crates/openpulse-core/src/manifest.rs`) has exactly four fields: `payload_hash` (SHA-256), `payload_size`, `sender_id`, `signature` (Ed25519). (No timestamp — `docs/features.md` says otherwise and is wrong.) The API's shape is the teachable part: `verify_manifest(manifest, pubkey)` checks the signature only, its doc comment making the caller responsible for the payload hash; `verify_manifest_with_payload(manifest, pubkey, payload)` checks the signature **then** `sha256(payload) == payload_hash` (`PayloadHashMismatch` otherwise), and is documented as the preferred, fail-closed entry point — a valid signature over a tampered payload cannot return `Ok`. An easy-to-half-use API was hardened by adding the composed call, not by documentation alone. Tests: `manifest_rejects_tampered_hash`, `verify_with_payload_rejects_tampered_payload`, among six in `manifest.rs`.

The file-transfer `FileOffer` (`crates/openpulse-filexfer/src/offer.rs`) deliberately does *not* reuse the manifest signature: audit finding F-2 showed that signing only the content hash lets an attacker replay a valid hash under a spoofed filename or geometry. The offer's Ed25519 signature covers the whole body, and `signing_bytes`/`encode_body` share one prefix helper "so the signed and wire forms can't drift".

2B.6.2 Peer descriptors: the key is the name

`PeerDescriptor` (`crates/openpulse-core/src/peer_descriptor.rs`) makes `peer_id` *be* the 32-byte Ed25519 verifying key. `verify_peer_descriptor` therefore needs no key store: it reconstructs the key from `peer_id` and verifies the signature over `peer_id`, `callsign`, `capability_mask`, `timestamp_ms`. This proves the descriptor was issued by the holder of the key that names it — nothing more. Whether that key belongs to the claimed callsign is, as always, the trust store's job.

2B.6.3 The relay envelope: wire v2 and the mutable-field trick

The mesh/relay `OPHF` envelope is where the 2026-07-15 audit is most misleading if read today: it (E3) reported a 16-byte `auth_tag` that was never verified and a relay that forwarded unauthenticated frames. In wire **v2** (`crates/openpulse-core/src/wire_query.rs`), the `auth_tag` no longer exists:

```
/// Wire schema version. v2 replaced the unauthenticated fixed 16-byte `auth_tag` with an *optional*
/// 64-byte Ed25519 origin `signature` verifiable against `src_peer_id` (E3): signed frames append 64
/// bytes, unsigned frames append none.
const VERSION: u8 = 2;
```

`src_peer_id` is the originator's verifying key (the `PeerDescriptor` pattern again), so `verify_origin()` needs no key distribution. Signed-ness is inferred structurally: the envelope is length-delimited by its carrying `Frame`, so a 64-byte trailer after the payload is a signature, zero bytes means unsigned, and any other trailer length is a corrupt frame. Three lines solve the classic mutable-field problem (compare the IP TTL and IPsec AH): `hop_index` must be incremented by every relay, so it is zeroed inside the signed region —

```
fn signing_bytes(&self) -> Result<Vec<u8>, WireQueryError> {
    let mut buf = self.header_and_payload()?;
    buf[HOP_INDEX_OFFSET] = 0;
    Ok(buf)
}
```

— and a relay's increment leaves the originator's end-to-end signature valid.

`RelayForwarder::forward()` (`crates/openpulse-core/src/relay.rs`) enforces, in order: hop-limit; **origin authentication** (`require_authentication` is enabled by default, and the check runs before any dedup-table capacity is spent on a forged frame); a capacity guard (4096 seen entries); `(session_id, nonce)` duplicate suppression; and the trust policy on the originator id. End-to-end tests: `authenticated_relay_forwarding` and `impersonated_origin_rejected_at_relay` (`crates/openpulse-mesh/tests/mesh_loopback.rs`) — both in the project's acceptance-criteria table.

One structural wrinkle: `RelayTrustPolicy.min_trust_filter` is never read inside `forward()` — its step 5 checks only the deny/allow lists (`RelayTrustPolicy::allows`). The trust-level filter is applied one

layer up, by the mesh node: `MeshNode::step` calls `trust_filter_allows(&peer_cache, &envelope.src_peer_id, self.relay_trust_filter)` before handing a `RelayDataChunk`/`RelayHopAck` to `forward()` (`crates/openpulse-mesh/src/lib.rs`). Say it precisely: the shipped mesh daemon enforces both origin authentication and the trust filter, but a caller that drives `RelayForwarder` directly gets only origin authentication. The 2026-07-15 audit's E3 wording ("`min_trust_filter` is unread in `forward()`") describes the forwarder, not the mesh node built on it.

2B.6.4 SAR reassembly under adversarial fragments

Handshakes travel as SAR fragments (251 data bytes per 255-byte frame payload, `crates/openpulse-core/src/sar.rs`). The reassembler keys on `(session_id, segment_id)`, and the daemon feeds every inbound handshake fragment the *same* constant session string — `const HANDSHAKE_SAR_SESSION: &str = "handshake"` — with no per-peer separation, so one crafted stray fragment could poison an in-flight reassembly. The reassembler's defence is to keep up to `MAX_CANDIDATES_PER_KEY = 8` *consistent candidates* per key: a conflicting fragment starts a new candidate instead of corrupting the existing one, and when the per-key cap is hit the **oldest candidate is evicted** so a flood cannot lock out a legitimate stream. The separate global cap `MAX_PENDING_SLOTS = 4096` does not evict — it rejects the new candidate with `SarError::TooManyPendingSegments`. `ingest` returns every candidate a fragment completed, and the daemon dispatches all of them, letting signature verification drop the bogus ones. Tests: `sar::tests::poison_fragment_does_not_block_legit_reassembly`; daemon-level `poison_fragment_does_not_block_conreq_verification`.

2B.7 The post-quantum handshake

2B.7.1 Sizes — constants, not prose

`crates/openpulse-core/src/pq_handshake.rs` implements a hybrid classical/post-quantum handshake over ML-DSA-44 (FIPS 204 signatures) and ML-KEM-768 (FIPS 203 key encapsulation). The sizes are compile-time constants, verified by `ml_dsa_44_keypair_sizes_are_correct` and `ml_kem_768_keypair_sizes_are_correct` (`tests/pq_handshake_integration.rs`):

Constant	Bytes	Meaning
`ML_DSA_44_PUBKEY_SIZE`	1312	ML-DSA-44 verifying key
`ML_DSA_44_SIG_SIZE`	2420	ML-DSA-44 signature
`ML_KEM_768_EK_SIZE`	1184	ML-KEM-768 encapsulation key
`ML_KEM_768_DK_SIZE`	64	decapsulation key in `d‖z` **seed** form — local only, never on the wire
`ML_KEM_768_CT_SIZE`	1088	ML-KEM-768 ciphertext
`ML_KEM_768_SS_SIZE`	32	shared secret

Both generators return *seeds* (32 B for ML-DSA-44, 64 B for ML-KEM-768), reconstructing the expanded keys at each use — which is why the stored decapsulation key is 64 bytes rather than the ~2.4 kB of an expanded FIPS-203 `dk`.

2B.7.2 Hybrid vs Pq-only, and the downgrade guard

`PqConReq` carries both identity keys (32-byte Ed25519 + 1312-byte ML-DSA-44), the 1184-byte KEM encapsulation key, and up to two signatures over the same canonical bytes. A Pq-only frame (`is_pq_only`: offers `Pq` and not `Hybrid`) leaves `classical_signature` empty. Verification always requires the ML-DSA-44 signature, requires the Ed25519 signature unless Pq-only, binds the classical key against the trust store (the F1 lesson, present here from the start), and *syntactically* validates the KEM key — it must parse as a well-formed `EncapsulationKey<MlKem768>`, a structural check, not proof of possession. `verify_pq_conack` adds a check the classical path lacks: the responder's `selected_mode` must be one of the modes the initiator offered, else `UnauthorizedMode` — a signing-mode downgrade guard.

The KEM flow: the responder encapsulates against the initiator's KEM key and returns the 1088-byte ciphertext in the CONACK; the initiator recovers the 32-byte shared secret with `kem_decapsulate`. **Nothing in the shipped code consumes that secret** — it is returned to the caller, and the integration test asserts both sides agree; that is all. More broadly, a grep across `crates/`, `apps/` and `plugins/` finds **no production caller of `pq_handshake` outside `openpulse-core`**: an implemented, tested protocol layer not yet wired into the daemon or any front-end. Where `docs/features.md` marks it "Implemented", read "the crate exists and passes its tests", not "in use on the air".

2B.7.3 What post-quantum costs on an HF channel

The PQ frames travel as JSON via `encode_pq_conreq`/`encode_pq_conack`, and `serde_json` serialises `Vec<u8>` as an array of decimal numbers — roughly 3–4 wire bytes per key byte. Measured in-process (these numbers are this book's measurements; the repo does not record them):

Frame	Raw crypto material	JSON bytes	251-byte SAR fragments
`PqConReq` (Hybrid, `W1AW`)	5012 B (Ed25519 key 32 + ML-DSA key 1312 + KEM ek 1184 + Ed25519 sig 64 + ML-DSA sig 2420)	≈ 18 000	72
`PqConAck` (Hybrid)	4916 B (32 + 1312 + KEM ct 1088 + 64 + 2420)	≈ 17 700	71

The JSON figures are approximate on purpose: a single measured run gave 18 044 and 17 683 bytes, but ML-DSA signing is randomised and every key byte is rendered as a one-, two- or three-digit decimal, so the encoded length moves by a few hundred bytes between runs. The encoding costs roughly 3.6× over the raw material, which is itself ~8.5× a classical CONREQ. At the working rates of the `hpx_hf` ladder that is a long transmission — the book does not quote a duration, because the repo records no measured PQ-handshake airtime. The gate test `pq_conreq_serialized_size_fits_in_sar_capacity` asserts only that the frame fits the 64 005-byte SAR ceiling (255 fragments × 251 bytes) — transportability, not efficiency. The honest status: post-quantum authentication over HF is *possible* here and *expensive*, and the encoding is the first place a future change would claw bytes back.

2B.8 Key management

2B.8.1 The control channel: Noise, and a gate that fails closed

`openpulse-linksec` secures the daemon↔client control link with `Noise_NNpsk0_25519_ChaChaPoly_BLAKE2s` via `snow`: no static keys, a 32-byte PSK mixed at position 0, so both endpoints prove knowledge of the PSK during the two-message handshake (mutual authentication), then exchange ChaCha20-Poly1305 AEAD frames with forward secrecy from the ephemeral DH. The module docs record why not TLS: rustls has no external/raw TLS-PSK support, and OpenSSL would add a C dependency to a pure-Rust workspace.

The entire bind policy is one line (`lib.rs`):

```rust
pub fn auth_required(bind_addr: &str, configured_require_auth: bool) -> bool {
    configured_require_auth || !is_loopback_bind(bind_addr)
}
```

Authentication is mandatory on any non-loopback bind and can be forced on loopback. The daemon enforces it fail-closed at startup (`server.rs`): auth required + no PSK ⇒ refuse to start, with an error naming the fix (`OPENPULSE_CONTROL_PSK`, 64 hex chars). Note the deliberate inverse: on a loopback bind without `require_auth`, a PSK that *is* set is discarded — the channel is plaintext by policy, never by accident. Per-client, a failed PSK handshake drops the connection before any command is processed. The WebSocket control port carries the same command protocol but has no authentication path, so whenever auth is required for either bind it is **disabled, not weakened** (`ws_disabled_for_auth`, §2B.9). The PSK source is currently the `OPENPULSE_CONTROL_PSK` environment variable parsed by `load_control_psk()`; the config key `[control_security] psk_key_id` exists but has no consumer — keystore-backed PSK loading is a stated follow-up, not shipped.

2B.8.2 The keystore

`FileKeystore` (`crates/openpulse-keystore`) stores named secrets in one encrypted file:

```
┌────────┬──────┬──────────┬──────────┬──────────────────────┐
│ "OPKS" │ 0x01 │ salt 16 B │ nonce 12 B │ ChaCha20-Poly1305 CT+tag  │
└────────┴──────┴──────────┴──────────┴──────────────────────┘
```

The plaintext is a `BTreeMap<String, Vec<u8>>` as JSON (deterministic key order). The encryption key is derived from the operator's master password with `Argon2::default()` — the `argon2` crate's default, documented by that crate as Argon2id; the repo pins no explicit memory/time/parallelism parameters, so this book quotes none. `save()` draws a fresh random salt and nonce from `OsRng` on every write and writes owner-only (`0o600`); `open()` validates owner-only permissions *before* touching the content (REQ-SEC-CTL-05), and a wrong password or any tampering surfaces identically as an AEAD `Decrypt` failure.

Above it, the `SecretStore` trait has two backends: `FileStore` (wraps `FileKeystore`) and `KeychainStore` (the OS secret service via `keyring`, feature `keychain`, default-on but dropped by CI's `--no-default-features`). `KeychainStore::available()` probes an entry and counts `NoStorageAccess`/`PlatformFailure` as unavailable while a merely-missing entry counts as reachable — exactly the distinction a headless host needs to fall back to the file store.

2B.8.3 The station identity key

Covered in §2B.3.4: a 32-byte Ed25519 seed, generated atomically at `0o600`, permission-validated on load — but failing *open* to a random ephemeral identity on a load error.

2B.8.4 PKI trust-bundle signing

The PKI web service (`pki-tooling`) signs trust bundles server-side: `PKI_SIGNING_KEY` (base64 32-byte seed) is **required by default**, with an ephemeral key only under an explicit `PKI_ALLOW_EPHEMERAL_KEY=true` plus a logged warning (tests: `signing_key_is_required_by_default`, `invalid_signing_key_length_is_rejected`). `bundle_canonical_body` is the one place in the codebase that recursively sorts JSON keys — its doc comment says why: signatures must survive a PostgreSQL JSONB round-trip, which does not preserve key order — and it excludes `is_current` and `bundle_signature` from the signed body as operational state rather than content. Tests: `bundle_canonical_body_is_deterministic`, `bundle_canonical_body_survives_key_reorder`.

2B.9 What is NOT protected

An honest security chapter ends with the gaps. Every item below is verified against v0.15.0 source.

1. **On-air payloads are cleartext — by design and by law.** There is no confidentiality on RF under §97.113(a)(4)/§97.309(a)(4). Anything you send, the world can read.
2. **`RelayForwarder::forward()` alone enforces *who*, not *how trusted*.** Origin authentication is on by default, but `min_trust_filter` is never read inside `forward()`; the trust-level check sits in `MeshNode::step` (`trust_filter_allows`). A component

reusing `RelayForwarder` outside the mesh node inherits no trust filtering.

3. **The WebSocket control port is disabled rather than authenticated** whenever control auth is required (a second unauthenticated door would defeat the first door's lock); Noise-over-WS is the stated follow-up. The panel's WS transport is unavailable against an authenticated daemon (whether the panel falls back to TCP was not traced for this book).
4. **The control PSK lives in an environment variable**, not the keystore; `psk_key_id` has no consumer.
5. **The PQ handshake has no production caller**, its ML-KEM shared secret is consumed by nothing, and `derive_session_keys` in `trust.rs` is likewise implemented and unused.
6. **`PskVerified` is unreachable from the handshake** (`psk_validated` is hardcoded `false` at all four `openpulse-core` verification call sites, and every production caller of `begin_secure_session` passes `false`). That makes the `Balanced` profile's `PskVerified` minimum-trust floor reachable only via `Verified` — i.e. an operator-marked `Full` key. `Balanced` and `Permissive` allow identical signing-mode sets; `SigningMode::Psk` and `::Paranoid` are negotiation labels with no distinct implemented behaviour found.
7. **The inbound handshake is an identity label, not an access gate** (audit E1, architectural, open). Inbound RF runs `PolicyProfile::Permissive` with no minimum-trust floor in `verify_conreq`, and admits unknown first-seen peers; only file transfer (off by default) consults the verified-peer record, and the QSY gate reads `rf_peer_trust()` — the trust of whichever peer was verified this session — rather than binding to the specific station whose QSY frame arrived.
8. **The station identity key fails open**: a load error yields a random ephemeral identity, not a startup refusal — the opposite posture to the trust store's fail-closed load.
9. **The ACK authenticator is 24 bits** — a considered trade for a 5-byte frame, adequate against casual forgery, not a 128-bit MAC.
10. **The keystore master password is held in memory as a `String`**; no zeroisation was found in the tree (an open question from grep absence, not a confirmed defect).

11. **The 2026-07-15 audit document is stale**: its deferred list (relay `auth_tag`, unsigned ACKs, replay freshness, SAR poisoning) has since been implemented, but the document was not revised — documentation describing a system as it was, where a reader expects the present tense.
12. **All fade-performance figures in this part are Watterson channel-simulator results.** None of the v0.13.0–v0.15.0 HF-fade work, and no cryptographic exchange described here, has an on-air measurement.

2B.10 Test evidence

All runs executed against v0.15.0 (`main` @ `21baaec`) with `--no-default-features`. The CI, Docs Checks and doc-stamping workflows are `disabled_manually` by the maintainer's deliberate choice (confirmed via the GitHub Actions API); these gates run locally. `cargo test -p openpulse-core --lib`: 316 passed. Integration suites: `handshake_integration` 22, `pq_handshake_integration` 16, `manifest_integration` 6, `peer_descriptor_integration` 9. Crate suites: `openpulse-linksec` 8, `openpulse-keystore` 5 (in ~2.2 s — Argon2 is deliberately slow).

Acceptance-criteria rows from `CLAUDE.md` covered by this part, tests confirmed present in the tree: relay origin authentication (`authenticated_relay_forwarding`, `impersonated_origin_rejected_at_relay` — `crates/openpulse-mesh/tests/mesh_loopback.rs`); handshake replay freshness (`handshake::tests`, six cases); SAR poison resistance (`sar::tests::poison_fragment_does_not_block_legit_reassembly`); the authenticated OTA ACK (`ack_exchange_integration.rs`) and its composition with the K=3 sub-floor return channel (`mfsk16_arq_subfloor.rs`).

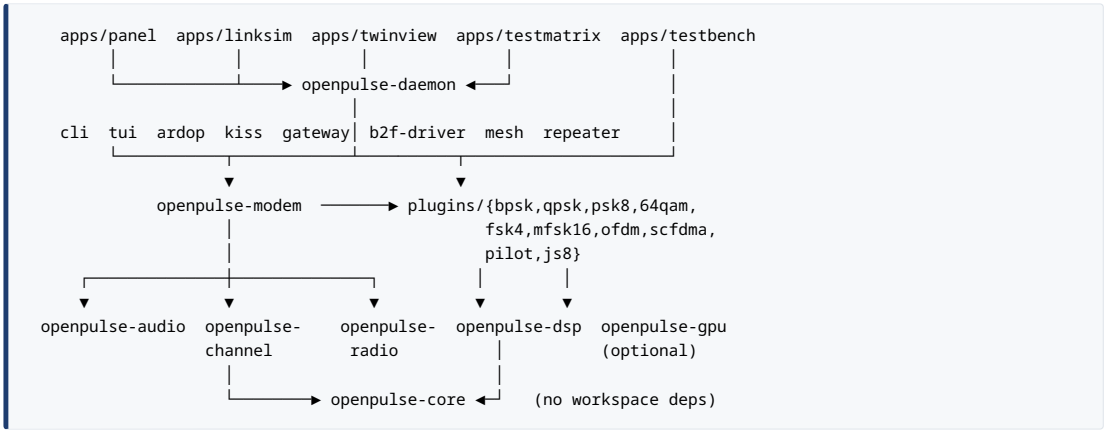
Chapter 3 — Computer science: architecture and implementation

For the operator, this chapter explains why OpenPulseHF behaves the way it does as a program: why one binary (`openpulse-server`) does almost everything, why a new waveform arrives as a "plugin", and why the project can claim its HF-fade behaviour is tested without ever keying a transmitter. For the developer, it is a navigation chart: where the code lives, how the layers depend on each other, how to add a mode, and — most importantly — the testing discipline the project has learned the hard way. Everything in this chapter cites a file, symbol, or test in the tree at v0.15.0; measured numbers carry their conditions, and the fade results are all from the Watterson channel *simulator*, not on-air operation.

3.1 The workspace: 41 crates, one strict layering

OpenPulseHF is a single Cargo workspace of 41 crates (`Cargo.toml`, `resolver = "2"`), versioned as a unit — every crate is `0.15.0`, licence GPL-3.0-or-later (`[workspace.package]`). Total source across `crates/*/src`, `apps/*/src` and `plugins/*/src` is about 108 000 lines.

The dependency graph is a strict layering, verified from the individual `Cargo.toml` files:



At the bottom, `openpulse-core` has **no workspace dependencies** by design. Its own header states the rule (`crates/openpulse-core/src/lib.rs:1-5`): "*Every other crate in the workspace depends on*

this crate. It intentionally has no heavy dependencies so it can be embedded in plugins without pulling in audio or DSP libraries." It holds the things everything must agree on: the frame format and CRC, the FEC codecs, the HPX session state machine, SAR, the ACK/rate-adaptation types, trust and handshake, compression, and — crucially — the plugin trait and registry.

The payoff of that rule is visible in any plugin manifest. `plugins/bpsk/Cargo.toml` is representative:

```
[dependencies]
openpulse-core = { workspace = true }
openpulse-dsp  = { workspace = true }
openpulse-gpu  = { workspace = true, optional = true }
num-complex    = { workspace = true }
```

A plugin author needs exactly two workspace crates: `openpulse-core` for the trait and `openpulse-dsp` for the shared DSP primitives (RRC filters, PLL, timing recovery, constellation helpers, equalizers).

One deliberate wrinkle: `openpulse-daemon` is declared in `[workspace.dependencies]` with `default-features = false`, so consumers that only need its *protocol types* do not pull in the GPU or native server code — the panel depends on it unconditionally, `openpulse-linksim` only behind its `serve` feature (`dep:openpulse-daemon`, "for the wire types"). The daemon crate splits its dependency table by target — the server runtime lives under `[target.'cfg(not(target_arch = "wasm32"))'.dependencies]`, and `#[cfg(not(target_arch = "wasm32"))]` is applied per-item through `daemon/src/lib.rs`, so the control-protocol types compile for WASM.

3.1.1 Where the code mass is — and where the conventions strain

The project's stated convention is "one concept per file; prefer small focused modules over large files" (CLAUDE.md). The size distribution (`wc -l` over all `src/*.rs`) shows where that convention has held and where it has lost:

Lines	File
5713	crates/openpulse-modem/src/engine.rs
5245	crates/openpulse-daemon/src/lib.rs
2712	crates/openpulse-daemon/src/server.rs
2082	plugins/qpsk/src/demodulate.rs
1974	apps/openpulse-panel/src/ui.rs
1784	apps/openpulse-testmatrix/src/bench.rs
1637	apps/openpulse-testbench/src/signal_path.rs
1614	apps/openpulse-linksim/src/gui.rs
1601	crates/openpulse-config/src/lib.rs
1480	apps/openpulse-linksim/src/lib.rs

`engine.rs` and `daemon/lib.rs` are the two accretion points. The `ModemEngine` struct (`engine.rs:271-401`) carries 47 fields, and the file declares 155 `pub fn` items on the engine's impls (counts taken by `grep` over the struct body and the file; a few of those methods sit in test-support impls). This is the measurable cost of a design in which one object owns the audio backend, the plugin registry, the pipeline scheduler, AFC state, the rate/ARQ machinery, DCD/CSMA, TX conditioning, the regulatory transmit log, and the RX front end. The chapter returns to how the project keeps this tractable (the seam rule, §3.3.3); it does not pretend the strain is not there.

3.2 The plugin system

To an operator, a "mode" like `BPSK250` or `OFDM52-16QAM` is a menu entry. Internally, every mode string is owned by exactly one *modulation plugin*: a compiled-in Rust object implementing one trait, registered at startup, and looked up by mode name for every transmit and receive. There is **no dynamic loading** — no `.so` files, no ABI boundary. Plugins are statically linked and registered by explicit `register_plugin(Box::new(...))` calls (the daemon does this in `server.rs:108-127`). The versioning machinery below is forward-looking for out-of-tree plugins; in-tree it has never had to reject anything, since all ten shipped plugins declare `"1.0"`.

3.2.1 The trait

Everything lives in one file, `crates/openpulse-core/src/plugin.rs`. Abbreviated but verbatim:

```rust
/// Current plugin trait version. Format: `<major>.<minor>.<patch>`
pub const PLUGIN_TRAIT_VERSION: &str = "1.0.0";

pub trait ModulationPlugin: Send + Sync {
    /// Return this plugin's static metadata.
    fn info(&self) -> &PluginInfo;

    /// Encode `data` bytes into a vector of normalised audio samples (-1.0 … +1.0).
    fn modulate(&self, data: &[u8], config: &ModulationConfig) -> Result<Vec<f32>, ModemError>;

    /// Decode audio samples back to the original bytes.
    fn demodulate(&self, samples: &[f32], config: &ModulationConfig)
        -> Result<Vec<u8>, ModemError>;

    // ── everything below is defaulted ──

    fn demodulate_soft(&self, samples: &[f32], config: &ModulationConfig)
        -> Result<Vec<f32>, ModemError> { /* hard-slice fallback, see below */ }
    fn frame_geometry(&self, _config: &ModulationConfig) -> Option<FrameGeometry> { None }
    fn supports_soft_demod(&self) -> bool { false }
    fn supports_mode(&self, mode: &str) -> bool { /* case-insensitive supported_modes match */ }
    fn estimate_afc_hz(&self, _s: &[f32], _c: &ModulationConfig) -> Option<f32> { None }
    fn estimate_snr_db(&self, _s: &[f32], _c: &ModulationConfig) -> Option<f32> { None }
    fn acquire_copy_offset(&self, _s: &[f32], _c: &ModulationConfig) -> Option<usize> { None }
    fn occupied_bandwidth_hz(&self, _mode: &str) -> Option<f32> { None }
    fn modulate_iq(&self, data: &[u8], config: &ModulationConfig)
        -> Result<(Vec<f32>, Vec<f32>), ModemError> { /* Hilbert shift of modulate() */ }
}
```

Three design decisions carry weight:

Plugins are stateless. The trait is `Send + Sync` and every method takes `&self`. There is no per-call mutable state, so one plugin instance is safely shared across threads — the daemon's multi-mode receive monitor relies on this explicitly (`crates/openpulse-daemon/src/monitor.rs:11`). Any state a demodulator needs (PLL phase, equalizer taps) is local to the call.

Two methods are required; everything else degrades gracefully. The default `demodulate_soft` is four lines: call `demodulate`, then map each hard bit to ±1.0 (`plugin.rs:162-173`). That is why the engine can offer soft-decision FEC to every mode without every plugin implementing an LLR path — and why `supports_soft_demod()` defaults to `false`, so the engine can warn when a soft-FEC mode is paired with a plugin that only fakes LLRs.

The soft path has a written contract. The trait doc (`plugin.rs:131-154`) specifies the LLR convention in three clauses, each with a named enforcing test:

1. *Sign*: positive means "bit more likely 0"; hard-slicing every LLR (`bit = llr <= 0`, LSB-first per byte) must reproduce `demodulate()`'s bytes exactly — enforced by `crates/openpulse-modem/tests/llr_convention_conformance.rs`.

2. *Scale*: per-plugin, deliberately not normalised across plugins; cross-mode combining must weight per frame (`combine_llrs_weighted`), never add raw LLRs from different plugins.
3. *Calibration*: a calibrated plugin divides distances by its estimated σ^2, so repeated observations of the same bits combine by summing (`combine_llrs_map`) — never by re-weighting with $1/\sigma^2$, which would apply σ^{-2} twice. `crates/openpulse-modem/tests/llr_calibration.rs` fails any plugin whose `mean(|LLR|)` stops growing with SNR.

This is a pattern worth naming: the contract lives in the trait doc *and* in a cross-plugin conformance test, so a new plugin cannot quietly invert the sign convention or emit uncalibrated LLRs and still pass the suite.

`FrameGeometry` (`plugin.rs:96-109`) exists because of a specific recorded bug. Before it existed, the engine guessed frame dimensions from trailing digits of the mode name — "*wrong for every mode whose name does not end in its baud rate (OFDM52's 52 is a subcarrier count; SCFDMA52-64QAM-P4 parsed as 4 baud) — and assumed a 32-symbol preamble (true only for BPSK)*" (`plugin.rs:90-95`, verbatim). The heuristic still exists as a fallback (`ModemEngine::frame_scan_geometry`, `engine.rs:1397-1434`); every production plugin should override `frame_geometry`.

3.2.2 Version gate and registry

Every plugin declares `trait_version_required: "<major>.<minor>"` in its `PluginInfo`. `PluginRegistry::register` validates it (`plugin.rs:291-338`): compatible iff `framework_major == plugin_major && framework_minor >= plugin_minor`; a malformed string is a distinct `PluginError::InvalidTraitVersionFormat`. Four registry tests pin both outcomes (`plugin.rs:422-460`).

The registry itself is deliberately unclever — a `Vec<Box<dyn ModulationPlugin>>` with a linear reverse scan:

```rust
// plugin.rs:341-347
pub fn get(&self, mode: &str) -> Option<&dyn ModulationPlugin> {
    self.plugins.iter()
        .rev() // later registrations take precedence
        .find(|p| p.supports_mode(mode))
        .map(|p| p.as_ref())
}
```

Later registrations shadow earlier ones for the same mode string — pinned by the test `later_registration_shadows_earlier_for_the_same_mode` (`plugin.rs:410-420`). With ten plugins and a total of 78 mode strings (that total is a sum over the ten verified `supported_modes` vectors), linear scan is a non-issue, and the shadowing rule gives tests a clean way to substitute an instrumented plugin.

The ten plugins and their mode counts: BPSK (5), QPSK (14), 8PSK (10), 64QAM (3), FSK4 (1: `FSK4-ACK`), MFSK16 (2), OFDM (6), SC-FDMA (12), Pilot (20), JS8 (5). Most modes are manually selectable only; the adaptive `hpx_hf` ladder uses SL1–SL14, with MFSK16 at SL1.

3.2.3 Writing a new plugin

The smallest real plugin in the tree is the ACK-channel plugin, `plugins/fsk4/src/lib.rs`, and it is the template to copy. The complete recipe:

1. **Crate**: new directory under `plugins/`, depending on `openpulse-core` and (usually) `openpulse-dsp`. Add it to the workspace `members`.
2. **`PluginInfo`**: name, version (`env!("CARGO_PKG_VERSION")`), description, `supported_modes`, and `trait_version_required: "1.0"`.
3. **Implement the trait** — `Fsk4Plugin` in full is ~35 lines of trait impl:

```rust
impl ModulationPlugin for Fsk4Plugin {
    fn info(&self) -> &PluginInfo { &self.info }

    fn modulate(&self, data: &[u8], config: &ModulationConfig) -> Result<Vec<f32>, ModemError> {
        modulate::fsk4_modulate(data, config)
    }

    fn demodulate(&self, samples: &[f32], config: &ModulationConfig)
        -> Result<Vec<u8>, ModemError> {
        demodulate::fsk4_demodulate(samples, config)
    }

    fn frame_geometry(&self, config: &ModulationConfig) -> Option<FrameGeometry> {
        let n = (config.sample_rate as f32 / BAUD).round() as usize;
        Some(FrameGeometry {
            symbol_period_samples: n,
            preamble_samples: n * 4,
            min_frame_samples: n * 20,
            max_frame_samples: n * 64 * 4 * 11 / 10,
        })
    }

    fn occupied_bandwidth_hz(&self, mode: &str) -> Option<f32> {
        mode.eq_ignore_ascii_case("FSK4-ACK").then_some(300.0)
    }
}
```

Note what FSK4 deliberately does *not* override: `demodulate_soft` stays at the ±1.0 default, and the doc comment explains why that is acceptable (the ACK channel always uses hard-decision short-block FEC and never carries LDPC/turbo payloads) — and that `supports_soft_demod() == false` lets the engine warn on an accidental soft-FEC pairing. 4. **Register it** wherever it should be usable: `engine.register_plugin(Box::new(MyPlugin::new()))` in the daemon (`server.rs:108-127`), the CLI, and the testmatrix, as applicable. 5. **Tests**: an in-crate loopback round-trip (every plugin has one), and — if the plugin claims soft output — it will automatically face `llr_convention_conformance` and `llr_calibration` in `openpulse-modem`. If the mode is meant for HF use, it must also face a fading test; §3.5 explains why an AWGN-only validation is not accepted for ladder rungs.

Before touching a demodulator, read the "DSP acquisition & carrier-recovery playbook" in CLAUDE.md — blind acquisition is the most-churned area of the modem, and the playbook's first rule (diagnose an "AFC" failure with a swept-applied-correction experiment before blaming the estimator) has repeatedly relocated bugs from the accused component to the real one.

3.3 The engine pipeline

`ModemEngine` (`crates/openpulse-modem/src/engine.rs`) is the object every frontend drives. Conceptually it is a half-duplex software modem: bytes in, audio out; audio in, bytes out; with FEC, framing,

AFC, DCD/CSMA, rate adaptation and the HPX session state machine wired around that core. (The ARQ/HARQ retry logic that used to live in an `ArqSession` type now lives in `harq.rs` and `rate_policy.rs` in the same crate.)

3.3.1 Five stages, honestly single-threaded

The pipeline decomposition is explicit as types (`crates/openpulse-modem/src/pipeline.rs:8-16`):

```
/// Logical modem pipeline stages.
pub enum PipelineStage {
    InputCapture,
    DemodulateDecode,
    HpxStateUpdate,
    EncodeModulate,
    OutputEmit,
}
```

Three message types cross stage boundaries — `WirePayload { bytes }`, `AudioSamples { samples }`, `DecodedFrame { sequence, payload }` — and the engine's private stage functions map one-to-one onto the stages: `stage_encode_frame` / `stage_modulate_payload` (EncodeModulate), `stage_emit_output` (OutputEmit), `stage_capture_input` (InputCapture), `stage_demodulate_payload` / `stage_decode_frame` (DemodulateDecode).

An honesty point the code itself makes: `PipelineScheduler` does **not** run stages concurrently. The file header says the types exist "*so execution can be moved to threaded workers later without changing higher-level call sites*" (`pipeline.rs:1-4`), and `route` sends a message into a bounded `std::sync::mpsc::sync_channel` and immediately receives it back on the same thread. `BackpressurePolicy` has exactly one variant, `Block`. What the scheduler buys *today* is (a) a stable interface boundary and (b) instrumentation: per-stage `enqueued` / `dequeued` / `dropped` counters and an `in_flight` derivation, exposed as `PipelineMetricsSnapshot` via `ModemEngine::pipeline_metrics_snapshot()`. The consumer is the CLI, which folds the snapshot into `SessionDiagnostics::pipeline_metrics` and prints it as JSON (`openpulse-cli/src/commands/session.rs:795`, `session_metrics.rs:114`); the daemon does not currently publish it over the control port. The architecture is prepared for threading;

the threading is not there. This is a recurring style in the codebase — *prepared, not premature* — the plugin trait version is the same shape (a contract defined before a dynamic ABI exists to need it).

3.3.2 TX path

A transmit call runs: CSMA check (pre-encode, so a deferral does not burn a sequence number) → frame encode (`Frame` envelope: magic `OPLS`, version, 16-bit sequence, 8-bit length, payload ≤ 255 B, CRC-16 — 10 bytes total overhead, `frame.rs`) → FEC encode per the selected `FecMode` → plugin `modulate` → `stage_emit_output`.

`stage_emit_output` (`engine.rs:4934-4976`) is the TX front-end seam, the mirror image of the RX seam below: it applies TX attenuation → CE-SSB peak-to-average conditioning *only when the mode benefits* → tanh limiter → spectrum tap → write → flush → regulatory TX log (`record_tx_frame`). The CE-SSB gate is a code-level fact worth quoting because documentation got it wrong twice before the code was made the authority:

```rust
// engine.rs:701-717
pub fn cessb_benefits(mode: &str) -> bool {
    let m = mode.to_ascii_uppercase();
    if !m.starts_with("OFDM") { return false; }
    !(m.contains("8PSK") || m.contains("16QAM") || m.contains("32QAM")
      || m.contains("64QAM") || m.contains("32APSK"))
}
```

That is: **OFDM16 and OFDM52 only**. All SC-FDMA modes are excluded (single-carrier FDM is low-PAPR by construction), and all dense OFDM-HOM modes are excluded — the function's doc comment records the measured damage when CE-SSB was applied to them (e.g. SCFDMA52-{32,64}QAM: 5/30 decodes with CE-SSB vs 30/30 without, AWGN 35 dB; OFDM52-64QAM: 3/20 vs 20/20, soft FEC, AWGN), measured in `apps/openpulse-linksim/tests/cessb_ab.rs` and `crates/openpulse-modem/tests/cessb_power_evm.rs`.

3.3.3 The InputCapture seam — the load-bearing idea

Every receive path in the system funnels its raw samples through `route_audio_stage(PipelineStage::InputCapture, …)` exactly once, and the entire receiver front end lives *there* — not in any of the 19 `InputCapture` call sites (`route_audio_stage` is called 40 times in total across all five stages). From `engine.rs:5149-5175`, with two explanatory comments abridged:

```
// The receiver front end lives at this single seam: every capture path funnels its raw
// samples through `route_audio_stage(InputCapture)` exactly once, so placing front-end
// transforms here (rather than in any one capture entry function) covers them all by
// construction. Order: notch (remove interference) → AGC (normalise the cleaned level).
if stage == PipelineStage::InputCapture && !self.input_prerouted {
    let mut samples = routed.samples;
    self.dc_blocks_processed = self.dc_blocks_processed.wrapping_add(1);
    samples = apply_dc_block(samples);
    if self.notch_enabled {
        self.notch_blocks_processed = self.notch_blocks_processed.wrapping_add(1);
        let mode = self.rx_mode.clone();
        samples = self.apply_rx_notch(mode.as_deref(), samples);
    }
    // Carrier detect BEFORE the AGC, on the true (pre-boost) level.
    self.update_dcd_at_seam(&samples);
    if self.agc_enabled {
        self.agc_blocks_processed = self.agc_blocks_processed.wrapping_add(1);
        samples = self.apply_rx_agc(samples);
    }
    return Ok(AudioSamples { samples });
}
```

Four details to internalise:

1. **Fixed order.** DC block (always on) → notch (optional) → **DCD** → AGC (optional). DCD is placed before the AGC deliberately: "*the AGC only normalises the level for the demodulator; the squelch/CSMA must see the real channel energy*" (`engine.rs:5168-5169`). An AGC-boosted noise floor would otherwise latch the squelch open permanently.
2. **Tripwire counters.** `dc_blocks_processed`, `notch_blocks_processed`, `agc_blocks_processed`, `dcd_blocks_processed`, each with a public getter. Their documented purpose: an enabled feature that never runs on a given path — for instance a new capture path that skips the seam — leaves its counter at 0, and a test can assert the counter incremented on the production path. This is a *runtime assertion that the seam rule has not been violated*.
3. **`input_prerouted`** suppresses double application: `decode_burst` re-scans a burst that `accumulate_routed` already front-end-processed, and the stateful AGC and the DCD latch must not run a second time per scan slice.
4. **Why the seam exists at all**: it was built after a real bug. The receiver notch was originally placed in `stage_capture_input` only — which covered every `receive*`-family test but **never ran in the daemon**, because the daemon uses a different capture entry. That incident produced the seam, the tripwires, and the five-point checklist discussed in §3.5.2.

3.3.4 Two RX entry families

The seam matters because captured audio reaches the demodulator by two distinct routes:

- **The `receive*` family** — `stage_capture_input` → decode. Opens a stream, reads once, decodes. This is what tests and the CLI use.
- **The daemon's streaming path** — `accumulate_capture` (`engine.rs:1347`) → `accumulate_routed` → `decode_burst`. This is what `server::run`'s receive ticker actually calls in production.

`accumulate_routed` (`engine.rs:1363-1393`) is a DCD-gated burst gatherer: it routes each tick's samples through the InputCapture seam, accumulates into `rx_burst` while carrier energy exceeds the DCD threshold, and flushes the whole burst when the carrier drops (or a size cap is hit). The rationale is hardware, recorded in the code (`engine.rs:1340-1346`): cpal is a callback backend whose stream needs tens of milliseconds to start delivering after `play()`, so reopening a stream every tick never warms up on real hardware. `decode_burst` then scans onset offsets across the burst lead-in, because a DCD-detected burst is not sample-accurate and the demodulator settles AFC on the window start.

The existence of two entry families is not a footnote — it is *the* recurring source of "wired at one seam, not all paths" bugs, and the reason the acceptance table requires a test through the production entry (`crates/openpulse-modem/tests/ota_production_capture_path.rs`), not just the convenient one. That test's own header records that the gap is not hypothetical: an audit finding (the OTA decode loop re-running the InputCapture front end) shipped unnoticed precisely because nothing tested the production path.

3.4 The daemon and its concurrency model

For the operator: `openpulse-server` is the one long-running process. It owns the sound card, the PTT line, the modem engine, and a TCP control port (default 9000, WebSocket 9001) that the panel GUI, the CLI, and any script talk to using newline-delimited JSON. Everything the panel shows — spectrum, decode events, file-transfer progress — arrives over that port.

For the developer, the daemon is where three distinct concurrency models meet, and the boundaries between them are explicit.

3.4.1 The entry point and dependency injection

```
// crates/openpulse-daemon/src/server.rs:44
pub async fn run(cfg: OpenpulseConfig, modem_backend: Box<dyn AudioBackend>) -> Result<(), String>
```

The audio backend is a *parameter*, not something `run` constructs. The binary passes the config-selected backend; a test harness passes a `LoopbackBackend` whose shared sample buffers it bridges. This single signature is what makes the twin-daemon rig (§3.5.4) possible: two completely real daemons in one process, joined at their loopback taps.

Startup: build the `ModemEngine`, set the callsign and declared TX power (without which the Part-97 transmit log would stamp an empty callsign and 0 W on every frame), optionally initialise a `GpuContext` behind `#[cfg(feature = "gpu")]`, then register nine plugins (BPSK, FSK4, MFSK16, OFDM, 8PSK, 64QAM, QPSK, SC-FDMA, Pilot — `server.rs:108-127`). JS8 is *not* registered as a `ModulationPlugin` in the daemon; the discovery subsystem transmits JS8 beacon audio through a separate `transmit_raw_audio` seam. GPU-capable plugins go through a two-arm `macro_rules! register_gpu_plugin` that constructs `Plugin::with_gpu(ctx)` when a context exists and falls back to `Plugin::new()` otherwise; `GpuContext::init()` returns `Option<Arc<Self>>`, so a missing adapter is a logged, normal outcome, never an error.

3.4.2 The main loop: fair `select!`, and PTT defence in depth

The daemon's heart is a `tokio::select!` loop (`server.rs:670-…`) with three arms, and the comment on its first line is a design decision:

```
loop {
    tokio::select! {
        // No `biased`: fair scheduling so a command flood cannot starve the rx tick or the watchdog.
        _ = watchdog_ticker.tick() => { /* PTT force-release poll, 100 ms */ }
        Some(cmd) = handle.commands.recv() => { /* client command */ }
        _ = rx_ticker.tick() => { /* one receive tick */ }
    }
}
```

The receive ticker runs at `cfg.daemon.receive_tick_ms` (default **50 ms**, `DaemonConfig::default()` in `openpulse-config`); the watchdog ticker is fixed at 100 ms.

Because a stuck-keyed transmitter is a real-world hazard (a babbling station, an overheating PA), PTT release has **three independent mechanisms**, each documented in place:

1. An **independent OS thread** watchdog (`runtime_state.ptt.spawn_watchdog(...)`, `server.rs:603-611`) that force-releases on a max-duration deadline "*even while this async command loop is blocked inside a long handler (a QSY scan or an OTA send-retry burst), which the cooperative* `select!` *arm below cannot do.*" It holds a `Weak` to the shared PTT state and exits when the daemon drops it.
2. The cooperative `watchdog_ticker` arm above, decoupled from the rx tick so a command flood cannot starve it.
3. An idempotent belt-and-braces poll at the head of every rx tick.

On top of that, every keyed transmit uses an RAII guard — `ptt.keyed(...)` returns a guard that releases at block end or on unwind (REQ-PTT-01), and returns `Err` on assert failure so the transmit is skipped rather than left keyed.

One receive tick, in order (`server.rs:725-984`): watchdog poll → read active mode → under `block_in_place`, read the (lazily opened, held-open) capture stream and feed `engine.accumulate_capture(...)` → if an OTA rate session is active, `ota_decode_burst` and answer with an authenticated ACK (capped by a consecutive-Nack budget of 3, and gated on a valid callsign, since the ACK keys the transmitter) — otherwise the multi-mode monitor and `decode_burst` → unpack compression (the frame is self-describing, so RX always unpacks regardless of the local `[compression]` setting) → route bytes into handshake/SAR/QSY/file-transfer handlers → drain queued file fragments as one keyed burst → discovery tick (a due JS8 beacon is transmitted here, where the PTT controller lives) → metrics, spectrum tap, OTA status (~1 Hz), and periodic station ID (armed by polling the engine's `frames_transmitted` delta, so no `note_tx()` call has to be threaded through every transmit site; never auto-IDs as `N0CALL`).

3.4.3 Shared state and channels

The daemon's shared state is a set of named `Arc` aliases (`daemon/src/lib.rs:443-470`), all tokio primitives:

```
pub type SharedMode           = Arc<Mutex<String>>;
pub type ValidModes           = Arc<std::collections::HashSet<String>>;  // immutable after startup
pub type SharedAttenuation    = Arc<Mutex<f32>>;
pub type SpectrumTap          = Arc<RwLock<Vec<f32>>>;
pub type SharedMessageStore   = Arc<Mutex<MessageStore>>;
// … four more Mutex aliases (SharedQsyEnabled, SharedBandplanMode,
//   SharedTunerOnHighSWR, SharedStationId) — nine in all
```

The discipline is legible in the choices: `SpectrumTap` is the one `RwLock` — many readers (every connected client), one writer (the rx tick) — everything else is `Mutex`; `ValidModes` is captured once at startup from `engine.plugins().list()` so a bad `SetMode` is rejected before it mutates anything. Channels: a `broadcast::channel::<ControlEvent>(256)` fans events out to all clients; an `mpsc::channel::<ControlCommand>(64)` funnels commands from all clients into the single main loop. A background task republishes engine events (`engine.subscribe()`) onto the control broadcast, handling `RecvError::Lagged(n)` by warning and continuing — the correct treatment of a slow broadcast subscriber. `ClientCtx` bundles the per-client handles into one clonable struct.

The control protocol itself (`protocol.rs`) is NDJSON over TCP — externally-tagged serde enums, 30 `ControlEvent` variants and 41 `ControlCommand` variants (`protocol.rs:114` and `:342`) — plus a binary side-channel for spectrum frames (magic `OPSP`). The same protocol runs over two transports: plaintext TCP for loopback, or a Noise-encrypted channel (`openpulse-linksec`, PSK-authenticated) for non-loopback binds, selected by an `enum ClientWriter { Plain(...), Noise(...) }`; when control auth is required — a PSK on the TCP port, or a non-loopback WebSocket bind — the unauthenticated WebSocket listener is not spawned at all (`ws_disabled_for_auth`, `server.rs:1168`), so it cannot bypass the gate.

3.4.4 Three concurrency models, on purpose

There is no single threading model; there are three, layered:

Layer	Model	Why
Daemon (tokio with `rt-multi-thread`), TNC servers (tokio)	async	many sockets, timers, fan-out
Engine calls from async	`tokio::task::block_in_place`	the engine is entirely synchronous; a real audio backend blocks until samples arrive
PTT watchdog, TNC bridge workers, twin-rig bridge	OS threads	must run even when the async loop is blocked; or the worker is inherently synchronous

The `block_in_place` wrapping has a documented consequence (`twin.rs:56-57`): the daemon "*must run on a multi-thread Tokio runtime: the daemon receive tick uses* `block_in_place`*, which panics on a current-thread runtime*" — so every test that drives the daemon uses `#[tokio::test(flavor = "multi_thread")]`.

The ARDOP TNC bridge (`crates/openpulse-ardop/src/bridge.rs`) is a compact study in mixing primitives *by purpose* rather than by fashion: `Arc<std::sync::Mutex<ModemEngine>>` (a blocking mutex held across blocking DSP by a sync worker thread), `Arc<tokio::sync::RwLock<TncState>>` (read by async command handlers), a `std::sync::RwLock<String>` for the mode string the sync worker reads, `tokio::sync::broadcast` for fan-out to clients, a bounded `std::sync::mpsc::sync_channel(64)` for the async→sync TX handoff, and atomics for flags. The KISS bridge mirrors it.

One caveat for readers of CLAUDE.md: its stated convention ("`Arc<RwLock<T>>` for shared state; `crossbeam_channel` for inter-thread messaging") describes an aspiration more than the daemon's reality — the dominant daemon primitive is `Arc<Mutex<T>>`, and the daemon/TNC channels are tokio `mpsc` and `std::sync::mpsc` respectively (`crossbeam-channel` remains a declared workspace dependency, consumed by the testbench and the panel). When convention and code disagree, read the code.

3.5 Testing strategy: why the project tests the way it does

The operator-level summary: OpenPulseHF's entire test suite runs with no radio, no sound card, and no network — `cargo test --workspace --no-default-features` — because every physical dependency has a software stand-in: a loopback audio backend, a channel simulator (Watterson fading, Gilbert-Elliott bursts, AWGN, QRN/QRM), and a rig that runs two *real* daemons against each other in one process. The traceability ledger's 2026-07-18 entry records the suite at **2146 passed, 0 failed** (recorded there as an actually-run result; this book did not re-run the full suite, but did re-verify one crate: `openpulse-core --lib`, 316 passed). The corollary rule is absolute: *never add tests that require real audio*

hardware. And to be equally plain about the limits: the v0.13.0–v0.15.0 HF-fade results below are validated against the Watterson channel **simulator**, not on the air.

What makes the strategy worth a section is not the harness inventory but the three lessons encoded in it.

3.5.1 Test the contract inline; test the physics in `tests/`

Unit tests live inline (`#[cfg(test)] mod tests`) and pin *contracts*: the LLR sign convention, the registry shadowing rule, frame-codec round trips, the device-resolution rule that ambiguity refuses to guess (`ambiguous_match_refuses_to_guess` in `openpulse-core/src/audio.rs`). Integration tests live in `tests/` directories and pin *behaviour under channel physics*: fade decode rates, ladder climb, LLR calibration. `crates/openpulse-modem/tests/` alone holds 91 integration test files.

3.5.2 The seam rule: prove the wiring, not the function

The rule, from CLAUDE.md's cross-cutting RX/TX checklist, quotable in full:

> When adding a feature that must run on every receive or transmit: (1) trace top-down from the binary — `server::run`'s `rx_ticker`/`tx` path, not just the engine API — to find what the running daemon actually calls; (2) place the transform at the single shared seam (`route_audio_stage(InputCapture)` for RX), never in one of the many caller functions; (3) never claim "covers all paths" from a callers-grep — prove it with a test that fails without the wiring; (4) add a runtime tripwire (a processed-block counter) and assert it increments on the production path; (5) add at least one test through the production entry (`accumulate_capture` or the twin daemon harness), not only the convenience seam.

Every one of those five points has a concrete artefact in the tree: the seam itself (`engine.rs:5153`), the four tripwire counters with public getters, and the production-entry test `ota_production_capture_path.rs` (OFDM52-16QAM, `SoftConcatenated`

FEC, 800-sample tick chunks, 30 trials, 3 HARQ attempts, 10 dB SNR on Watterson `moderate_f1`). The rule was paid for: the receiver notch shipped wired into `stage_capture_input`, passed every `receive`-family test, and never ran in the daemon.

The generalisable insight for any codebase: a correct transform in the wrong place passes every test that calls the transform. Only a test through the production entry point, plus a runtime counter, distinguishes "the function works" from "the system runs the function".

3.5.3 Sabotage verification and the vacuous-gate lesson

A test's name is a claim, and the project has been burned — three times, per its own memory notes — by tests whose assertions established less than their names claimed. The countermeasure is **sabotage verification**: after writing a test, deliberately break the thing it guards and watch it fail. The traceability ledger records the practice repeatedly; the cleanest worked example: with one expected CAT frame replaced by `DE AD BE EF`, the assertion failed showing the real bytes (`left: [254, 254, 148, 224, 28, 0, 1, 253]`) — "*proving the log is genuinely populated rather than compared empty-to-empty.*"

The failure mode it guards against is the **vacuous gate**. `docs/dev/project/release-1.0-criteria.md` makes "no known vacuously-passing gate" release criterion D3, and lists five found and fixed (`bpsk_hardening` SNR sweep, `tx_limiter`, CAT `write_log`, `fec_decision_gate`, `relay_empty_buffer`); the ledger entry of 2026-07-18 repairs six, four of which are the same ones — **seven distinct gates** to date. The criterion is not a count but a standard: that a fresh sweep finds no more. Real examples, each with its specific defect (all from the ledger):

Test	Defect
`tx_limiter.rs::limiter_bounds_peak_amplitude`	never inspected an amplitude — transmitted, received, and dropped the result; it was the *only* engine-path coverage of the TX limiter
`generic_cat_integration.rs` (`*_sends_correct_bytes` ×5)	never read `MockTransport.write_log` — the field declared "inspectable by tests" was boxed away with the transport; only `Ok` was asserted
`fec_comparison.rs::fec_decision_gate`	zero assertions: printed ACCEPTED or REJECTED and passed either way
`repeater_integration.rs::relay_empty_buffer_returns_none`	discarded the result ("both are acceptable") — would equally pass a repeater relaying garbage from an empty buffer
PTT ≤ 50 ms timing test in `noop.rs`	timed a bool flip on `NoOpPtt`; cannot fail
`hpx_hf_rungs_survive_fade.rs`	cited as "every rung" while skipping MFSK16 and the LdpcHighRate rungs, with a loose `checked >= 6` tail that would tolerate rungs silently vanishing

The resolution principle is itself in the ledger: "*Fix the gate where the gate can be fixed; correct the claim where it cannot.*" The first four became real assertions — the limiter test required a new `LoopbackBackend::clone_shared()` handle so it could inspect emitted samples, *plus a control test proving an unlimited transmit exceeds the threshold* (otherwise "peak ≤ 0.5" could hold on a signal the limiter never touched). The PTT timing claim could not be fixed on a no-op backend, so a real-I/O timing test over a mock rigctld TCP path replaced it. The fade-sweep test's skips were legitimate, so the acceptance-criteria *row* was corrected to say what is actually swept, and the loose count replaced by an exact one derived from the profile.

Note the pattern in the limiter fix: sabotage verification generalises to a **control test** — a gate that asserts "X stays under the threshold" is only meaningful next to a companion proving that without the mechanism, X exceeds it.

3.5.4 The harness ladder: share the production code, don't model it

Four harnesses form a ladder of increasing realism, and the design rule that runs through them is: *a harness that reimplements production policy will lie about it*.

1. **`ChannelSimHarness`** (`crates/openpulse-modem/src/channel_sim.rs`) — two `ModemEngine`s, two loopback backends, one `ChannelModel`. Explicitly unidirectional: it validates one-way modem correctness under simulated HF propagation, and its doc says so. The usage is five lines:

```rust
let mut harness = ChannelSimHarness::new();
let mut channel = AwgnChannel::new(AwgnConfig { snr_db: 20.0, seed: Some(1) }).unwrap();
harness.tx_engine.transmit(b"hello", "BPSK250", None).unwrap();
harness.route(&mut channel);
let rx = harness.rx_engine.receive("BPSK250", None).unwrap();
```

The lower-level primitive `bridge_through(src, dst, channel)` operates on externally owned backends, so forward and reverse directions can each carry their own channel model — which is exactly what the twin rig uses.

2. **The benchmark harness** (`crates/openpulse-modem/src/benchmark.rs`) — replays HPX state-machine events and gates on `passed == total && mean_transitions <= 20.0` (run via the CLI, checked with `jq`). Its limitation is stated by the gate built to complement it: it exercises the session state machine with no modem in the loop.

3. **`openpulse-linksim`** — a two-station bidirectional ARQ simulator measuring *effective two-way transfer rate* including ACK air time, turnaround, and retransmission. Its header states the crucial property: rate control is **not reimplemented** — it drives the *shared* receiver-led `OtaRateController` and the *same* `ModemEngine::rx_snr_db` estimator the daemon uses (which is `pub` for exactly this reason, per its doc comment). This harness hosts the **goodput regression gate** (`mod goodput_gate`, four tests): the full modulate → channel → demodulate → FEC → rate-control stack, seeded and deterministic, asserting effective bps stays above ~65 % of a measured baseline — e.g. `psk_ladder_goodput_floor_awgn` asserts ≥ 250 bps on AWGN 20 dB against a ~397 bps baseline noted in the assertion message (baseline figures are comments, not runtime-computed). Its companion `psk_ladder_climbs_off_the_entry_rung_on_a_fade` (Watterson `moderate_f1` @ 20 dB, simulator) asserts `delivery_ratio > 0.9` *and* `avg_level >= 3.0` — deliberately a level, not a bps figure, because bps on a short run is dominated by the slow rungs during the climb. That test's doc comment records why it exists: it is "*the gate whose absence hid #934 for two releases*" — every earlier fade gate called the demodulator directly and proved the rungs *decode*, while the real rate controller kept the link pinned on its entry rung at ~5 bps. The gate goes through the controller because the bug lived in the controller.

4. **The twin-daemon rig** (`crates/openpulse-daemon/src/twin.rs`) — the top of the ladder, and the clearest statement of the philosophy, verbatim from its header: "*Both daemons run the REAL* `crate::server::run` *stack —* `RateAdapter`*,* `HpxReactor`*, OTA rate-stepping, QSY, repeater — unlike* `openpulse-linksim`*, which reimplements the policy layers.*" `spawn_bridged_pair` starts two genuine daemons on their own threads, bridges A's loopback TX through a forward channel model into B's loopback RX and vice versa, and returns two live control-port addresses — so two real panel GUIs can attach, one per station, and watch both directions. The acceptance gate for file transfer runs here: `cargo test -p openpulse-daemon --test twin_daemon_bridge a_file_crosses`.

3.5.5 The acceptance table and the traceability ledger

Two artefacts close the loop between claims and evidence. CLAUDE.md's acceptance-criteria table maps ~53 requirements to named tests, under the rule "*write the test first, confirm it fails, implement until it passes; do not mark a task done if its test does not exist.*" And `docs/dev/project/traceability.md` (6888 lines, newest first) records each substantive change as a chain — requirement → design decision → implementation → tests → **test results, actually run** — with the enforcement clause that makes it non-vacuous: the results link must be a real run with pass/fail counts, "*never* '*covered*' *asserted from a callers-grep.*" The deliberate anti-bureaucracy decision is stated alongside: no separate heavyweight matrix that rots; the chain lives in the artefacts that already travel with the change (commits, PRs, the ledger, the acceptance table).

Even the acceptance table itself is subject to the vacuous-claim discipline: the "every `hpx_hf` rung" row was found to over-claim (see the table in §3.5.3) and was corrected to say what the test actually sweeps.

3.6 Build, features, and gates

For the operator building from source: the one command that matters is `cargo build --workspace` (Linux needs `libasound2-dev`), and the one trap is that **real audio is a compile-time feature**

with inconsistent names across crates. Building a TNC binary without its audio feature yields a program that silently falls back to the loopback backend no matter what `config.toml` says.

3.6.1 Feature matrix

All read from the manifests:

Crate	Features	Default
`openpulse-audio`	`cpal-backend`	`[]` (off)
`openpulse-cli`	`cpal-backend`, `serial`, `generic-serial`, `gpio`	`["cpal-backend"]` — **CPAL on by default**
`openpulse-daemon`	`gpu`, `cpal`, `serial`, `generic-serial`, `gpio`	`["gpu"]`
`openpulse-ardop` / `openpulse-kiss`	`cpal`	off
`apps/openpulse-testbench`	`cpal`	off
`openpulse-radio`	`serial` (→ `serialport`), `generic-serial`, `gpio` (→ `gpiocdev`)	`[]`
`plugins/{bpsk,qpsk,psk8,64qam,scfdma}`	`gpu` (→ `openpulse-gpu`)	`[]`
`openpulse-modem`	*(no `[features]` section)*	—

Consequences, spelled out: `cargo build -p openpulse-cli` includes CPAL; `cargo build -p openpulse-kiss` does not. The feature is spelled `cpal-backend` for the CLI and audio crates but `cpal` for the daemon, TNCs and testbench — `--features cpal` errors on the CLI, `--features cpal-backend` errors on the daemon. The runtime `--backend cpal` flag warns at startup when the feature is absent. (One more piece of doc drift, flagged rather than repeated: `openpulse-audio/src/lib.rs:7` claims `cpal-backend` is "enabled by default"; its own manifest says `default = []`, and the manifest is authoritative.) Platform limits from the manifest comments: `serial`/`generic-serial` are Unix-only, `gpio` is Linux-only.

GPU acceleration deserves its own row of honesty: five plugins (BPSK, QPSK, 8PSK, 64QAM, SC-FDMA) have optional wgpu paths against six WGSL kernels in `openpulse-gpu`; OFDM is not GPU-accelerated. Because the standard `--no-default-features` gates never compile the `gpu` cfg paths, they would rot silently — the CI workflow therefore has a dedicated `gpu-feature-gates` job that compiles and lints (but does not run — CI runners have no wgpu adapter) the GPU paths on every change; its comment cites the PR that found a build break reachable only there.

3.6.2 The canonical gate set

```
./scripts/check-toolchain.sh                                    # requires rustc >= 1.94.0
cargo build --workspace                                         # libasound2-dev on Linux
cargo test  --workspace --no-default-features
cargo clippy --workspace --no-default-features --all-targets -- -D warnings
cargo fmt --all -- --check
cross check --workspace --target aarch64-unknown-linux-gnu --no-default-features   # Raspberry Pi
```

`--all-targets` on clippy is not decoration — without it, test code is never linted, and CLAUDE.md records exactly that mechanism letting an unused binding sit in `session_key.rs`. A documented fallback excludes `pki-tooling` (which drags in sqlx/database machinery) when the local toolchain cannot build it.

3.6.3 CI: configured, and deliberately disabled

`.github/workflows/ci.yml` defines eight jobs — the full test/audit runner (`pr-hook-long-runner`), cross-compile check, core and full workspace gates, a compile-only macOS build (the comment notes macOS runners as a 10x cost multiplier), the GPU rot-guard, a Pi5 smoke profile, and a `workflow_dispatch`-only, `continue-on-error` virtual-loopback smoke run. But `gh workflow list --all` reports the CI, docs-checks and doc-stamping workflows as `disabled_manually`, and the git hooks path points at an empty no-hooks directory. This is the maintainer's deliberate arrangement on a single-maintainer project: **the gates are run locally, by hand, before every PR** — the traceability ledger's per-change "test results (actually run)" lines are the enforcement record, in place of a green checkmark. A contributor should treat the command set above as mandatory pre-PR, because nothing in the hosted pipeline will run it for them.

3.7 What to take away

Five load-bearing ideas, each anchored to an artefact:

1. **One seam per cross-cutting concern.** RX front-end DSP lives at `route_audio_stage(InputCapture)`; TX conditioning at `stage_emit_output`; each with tripwire counters and a production-entry test. A transform in a caller instead of the seam is a latent daemon bug that the convenient tests cannot see.
2. **The gate must be able to fail.** Sabotage-verify new tests; add control tests to threshold assertions; when a test's claim

outruns its assertions, fix the gate or fix the claim. Seven vacuous gates have been found and repaired to date.

3. **Harnesses share production code.** The linksim drives the real rate controller and the real SNR estimator; the twin rig runs the real `server::run`. Reimplemented policy in a harness is how a harness lies.
4. **Prepared, not premature — and honest about it.** The pipeline scheduler defines the threading boundary without threading; the plugin version gate defines the compatibility contract without a dynamic ABI. Both say so in their own doc comments.
5. **The strain is real and measured.** A 5713-line engine with 47 fields and 155 `pub fn` items, and a hand-maintained producer↔consumer invariant in the FEC dispatch — recorded as "Invariant (audit G-7)" at `engine.rs:2842`, where the demodulator populates exactly one of `raw_wire`/`llrs` and each match arm's `.unwrap()` is guarded only by that pairing — are the running costs of the current architecture. Knowing where they are is part of being able to extend the codebase.

Chapter 4 — Use cases, setup and configuration

This chapter is the practical one: twelve scenarios, each self-contained, each with copy-pasteable commands and configuration that use the real key names and real defaults from version 0.15.0 of the code. Scenarios that need hardware say so at the top in bold; everything else runs on a bare Linux machine with a Rust toolchain.

Two conventions used throughout:

- **Hardware required** flags a scenario (or step) that needs a radio, a sound device, or both. Everything not flagged runs against the built-in `LoopbackBackend`.
- Configuration lives in one file: `~/.config/openpulse/config.toml` on Linux (`~/Library/Application Support/openpulse/config.toml` on macOS, `%APPDATA%\openpulse\config.toml` on Windows). Every section is optional — a partial file is legal, and missing keys take built-in defaults (test: `missing_fields_get_defaults` in `crates/openpulse-config/src/lib.rs`). Precedence is always **CLI flag > config file > built-in default** (test: `cli_override_pattern`).

One caveat that applies to the whole chapter: the HF-fade performance work in v0.13.0–v0.15.0 (the `hpx_hf` ladder, the differential QPSK rung, the OFDM mid-rungs) is validated against the Watterson channel *simulator*. None of it has been validated on the air yet; on-air regulatory validation, JS8 discovery Phase H and file-transfer Phase F all remain open.

4.1 Scenario 1 — bench validation with no radio

No hardware required. This is where every installation should start: prove the modem, the FEC, the rate ladder and the state machine on your machine before any cable is plugged in.

4.1.1 Build

```
./scripts/check-toolchain.sh          # requires rustc >= 1.94.0
cargo build --release -p openpulse-cli --no-default-features
```

`--no-default-features` builds the CLI without the CPAL audio backend, so it needs no ALSA headers and no sound hardware — the `loopback` backend is all you get, which is exactly right for the bench. (The default CLI build *does* include CPAL — see §4.2.1.)

Confirm the binary and the mode registry:

```
./target/release/openpulse --version    # openpulse 0.15.0
./target/release/openpulse --backend loopback --log error modes
./target/release/openpulse --backend loopback --log error devices
```

`devices` on the loopback build prints a single virtual device supporting 8000, 16000, 44100 and 48000 Hz. `modes` prints the full registered mode registry — BPSK, QPSK (including the differential `-D` variants), 8PSK, 64QAM, OFDM, SC-FDMA, PILOT, MFSK16 and FSK4-ACK families.

4.1.2 First transmit (loopback)

```
openpulse --backend loopback --log error transmit "CQ CQ DE N0CALL" --mode BPSK250 --fec rs
```

Output:

```
Transmitted 15 bytes in BPSK250 mode (fec=Rs).
```

The `--fec` flag accepts (case-insensitive, `_` and `-` interchangeable): `none`, `rs`, `rs-interleaved`, `concatenated`, `rs-strong`, `soft-concatenated`, `ldpc`, `turbo` (`parse_fec` in `crates/openpulse-cli/src/main.rs`). Two `FecMode` variants — `LdpcHighRate` and `ShortRs` — exist in the engine but are profile/engine-internal and are not accepted by the `openpulse` CLI's `--fec` (the linksim's own `--fec` does accept `ldpc-high-rate`).

A note on `receive`: the loopback backend does not persist audio across process invocations, so a two-terminal `transmit` | `receive` demo does **not** work with `--backend loopback`. For a genuine two-process TX→RX run on one machine, use the virtual audio loopback rig (§4.1.6).

4.1.3 Watch the rate ladder climb — `openpulse adaptive`

The single best no-hardware demonstration of what the adaptive rate controller does:

```
openpulse --backend loopback --log error adaptive \
  --profile hpx_hf --channel awgn --snr 14 --frames 6 --seed 42
```

```
adaptive session: profile=hpx_hf channel=awgn frames=6 payload=64B
  start: level=SL2 mode=BPSK31
  frame 0: mode=BPSK31 decoded=ok snr=14.0dB ack=ACK-UP → SL3 (BPSK63)
  frame 1: mode=BPSK63 decoded=ok snr=14.0dB ack=ACK-UP → SL4 (BPSK100)
  frame 2: mode=BPSK100 decoded=ok snr=14.0dB ack=ACK-UP → SL5 (BPSK250)
  frame 3: mode=BPSK250 decoded=ok snr=14.0dB ack=ACK-UP → SL6 (QPSK250-D)
  frame 4: mode=QPSK250-D decoded=ok snr=14.0dB ack=ACK-UP → SL7 (OFDM52)
  frame 5: mode=OFDM52 decoded=ok snr=14.0dB ack=ACK-UP → SL8 (OFDM52-8PSK)
  final: level=SL8 mode=OFDM52-8PSK | 6/6 frames decoded, 6 transitions, ~23 bps
```

Every `hpx_hf` session starts at SL2 (`BPSK31+Rs`) and climbs one rung per clean decode. The `~23 bps` figure is the *effective* rate of this six-frame run including the climb through the slow rungs — do not read it as a steady-state throughput. `--channel` also accepts `clean`, `watterson-good-f1` and `watterson-poor-f1`; pass `--seed` for determinism and `--json` for machine-readable output.

4.1.4 Ask the mode advisor

```
openpulse mode-advisor --snr 12 --profile hpx_hf
openpulse mode-advisor --snr 3  --profile hpx_hf
```

```
profile=hpx_hf snr_db=12.0 recommended_speed_level=SL9 recommended_mode=OFDM52-16QAM reason="Using profile 'hpx_hf' floor: snr_db=12.0 meets SL9 floor
(12.0 dB)."
profile=hpx_hf snr_db=3.0 recommended_speed_level=SL2 recommended_mode=BPSK31 reason="Using profile 'hpx_hf' floor: snr_db=3.0 meets SL2 floor (3.0 dB)."
```

The `--profile` help lists all twelve accepted values — `hpx500`, `hpx_modcod`, `hpx_pilot`, `hpx_pilot_rrc`, `hpx_pilot_fast`, `hpx_pilot_fast_rrc`, `hpx_hf`, `hpx_ofdm_hf`, `hpx_wideband`, `hpx_wideband_hd`, `hpx_narrowband`, `hpx_narrowband_hd` — because clap takes them straight from `SessionProfile::PROFILE_NAMES` (`crates/openpulse-core/src/profile.rs`), so the help cannot drift from what `by_name` accepts. Profile names are case-insensitive and `-`/`_` interchangeable.

4.1.5 Run the HPX conformance benchmark

```
openpulse --backend loopback --log error benchmark run
```

This emits a JSON report of ten HPX state-machine scenarios:

```
{ "total": 10, "passed": 10, "failed": 0, "mean_transitions": 5.1, "mean_wall_ms": 0.0, "scenarios":
        [ ... ] }
```

The regression gate (run locally — the repository's GitHub CI workflows are disabled by the maintainer's deliberate choice, so these gates are operator-run):

```
openpulse --backend loopback --log error benchmark run > /tmp/bench.json
jq '.passed == .total and .mean_transitions <= 20.0' /tmp/bench.json   # must print true
```

The full CI-equivalent suite, if you want the whole workspace validated:

```
cargo test --workspace --no-default-features
```

4.1.6 The virtual audio loopback (two real processes, one machine)

Needs sudo once (kernel module), no radio. The three-rung validation ladder — documented in `scripts/run-loopback-virtual.sh` — is *virtual* → *hardware* → *on-air*, each rung gated on the previous:

```
virtual   snd-aloop, single clock, no analog     -> a failure is DSP/code/config
hardware  two USB soundcards + analog cable      -> adds dual clocks + analog path
on-air    real radios                            -> adds RF
```

Setup and run:

```
# One-time: load snd-aloop and publish the aloop_tx / aloop_rx ALSA plug PCMs
./scripts/setup-virtual-loopback.sh

# Needs a CPAL-enabled CLI build (the default features include it):
cargo build --release -p openpulse-cli

# Drive every registered mode TX -> snd-aloop -> RX through the real cpal+ALSA path:
./scripts/run-loopback-virtual.sh

# Or target specific modes:
MODES="BPSK250 QPSK250-D OFDM52" ./scripts/run-loopback-virtual.sh
```

The script's tunables are environment variables: `OPENPULSE_BIN` (default `target/release/openpulse`), `TX_DEVICE`=`aloop_tx`, `RX_DEVICE`=`aloop_rx`, `PRE_WAIT`=`7` seconds (lets the receiver's AFC-settling buffer of ~6.4 s fill before TX), `POST_WAIT`=`6`, `LISTEN_MS`=`120000`, `PAYLOAD_BYTES`=`32`, `RETRIES`=`3`, `OUTPUT_DIR`=`docs/dev/test-reports`.

4.2 Scenario 2 — first HF station

Hardware required: an HF transceiver, a sound interface (Digirig, SignaLink, a rig's built-in USB codec, …), and — for CAT PTT — `rigctld` from Hamlib talking to the rig.

4.2.1 Build with real audio

The feature spelling differs per crate, and getting it wrong produces a binary that silently uses the loopback backend. For the CLI, CPAL is already a default feature:

```
sudo apt-get install libasound2-dev        # Linux: ALSA headers for CPAL
cargo build --release -p openpulse-cli     # cpal-backend is a DEFAULT feature here
```

For every other binary the audio feature is opt-in:

```
cargo build --release -p openpulse-daemon    --features cpal    # openpulse-server
cargo build --release -p openpulse-ardop     --features cpal    # openpulse-tnc
cargo build --release -p openpulse-kiss      --features cpal    # openpulse-kisstnc
cargo build --release -p openpulse-tui       --features cpal-backend
cargo build --release -p openpulse-testbench --features cpal
```

Note that `openpulse-daemon`'s default feature set is `["gpu"]`, so the command above also pulls in wgpu; add `--no-default-features --features cpal` for a lean CPU-only daemon.

Note `--features cpal` does not exist for the CLI (it would error); its feature is named `cpal-backend` and is only needed explicitly if you build the CLI with `--no-default-features`. Watch for the fallback behaviours: with the feature missing, `backend = "cpal"` in config logs a warning and falls back to loopback, but `backend = "default"` falls back **silently** (`crates/openpulse-ardop/src/main.rs`; the CLI prints `note: cpal backend not compiled in; falling back to loopback` to stderr).

4.2.2 The wiring

```
              +--------------------+           +--------------+
 USB          |  Sound interface   |  audio    |              |
============== |  (e.g. Digirig)    |===========|  Transceiver |
 |            |                    |  in/out   |              |
 |            +--------------------+           +--------------+
 |                                                    ^
 |            +--------------------+                  |
 +==========> |  rigctld (Hamlib)  | --- CAT/serial --+   PTT + frequency
              |  127.0.0.1:4532    |
              +--------------------+
```

PTT and CAT are separate concerns: the modem asserts PTT through one of seven backends, and (optionally) reads/sets frequency and meters through the CAT layer.

`--ptt` / `[modem] ptt_backend`	Mechanism	Needs
`none`	no keying (RX-only, or VOX-less bench)	—
`rts` / `dtr`	serial control line	`--rig /dev/ttyUSBn` (serial build)
`vox`	radio's own VOX keys on audio	nothing (but see `arq` timing notes)
`rigctld`	Hamlib CAT PTT over TCP	rigctld at `--rig addr:port`
`cm108`	CM108 sound-chip GPIO	`/dev/hidrawN` (empty = autodetect); GPIO pin `[modem] ptt_gpio` (default `3`, "the near-universal default")
`gpio`	Linux GPIO character device	`--rig chip:line[:active_low]` (e.g. `gpiochip0:17`); needs a `--features gpio` build
`generic`	TOML-defined serial CAT command set	`--rig <serial>` **and** `--rig-file <toml>`; Unix-only, `--features generic-serial` build. Present in code but absent from the `--ptt` help string — treat it as an undocumented option

Shipped rig-definition files for the generic backend: `docs/config/rig-icom-ic7300.toml` and `docs/config/rig-yaesu-ft817.toml`. Shipped example station configs: `docs/config/openpulse-kx3.toml` and `docs/config/openpulse-tx500.toml`.

4.2.3 config.toml from scratch

Generate the fully-commented template (it prints to stdout — it does not write a file):

```
mkdir -p ~/.config/openpulse
openpulse config init > ~/.config/openpulse/config.toml
```

A minimal working HF station config — every key below is real, with real semantics:

```
[station]
callsign = "N0CALL"          # CHANGE THIS. The daemon and mesh daemon REFUSE to start on N0CALL.
grid_square = "AA00"         # Maidenhead locator
auto_id_interval_secs = 600  # periodic station ID (REQ-REG-10); 0 disables
auto_id_signoff_idle_secs = 10  # end-of-exchange ID after TX quiet; 0 disables
tx_power_watts = 25.0        # operator-DECLARED power for the TX log (default 0.0);
                             # the modem cannot measure PA output

[audio]
backend = "cpal"             # "cpal" | "loopback" | "default"
device = "USB Audio CODEC"   # "" = system default; resolution is hotplug-safe:
                             # exact name -> stable ALSA CARD= token -> substring
tx_limiter_threshold = 0.0   # 0.0 = disabled; soft limiter s -> t*tanh(s/t)

[modem]
mode = "BPSK250"             # default fixed mode
profile = "hpx_hf"           # the SpeedLevel ladder (SL1 MFSK16 ... SL14 OFDM52-64QAM+LdpcHighRate)
ptt_backend = "rigctld"      # none|rts|dtr|vox|rigctld|cm108|gpio
ptt_device = ""              # serial path (rts/dtr) or /dev/hidrawN (cm108)
dcd_squelch = 0.01           # busy-channel detector threshold (engine default)

[radio]
cat_backend = "rigctld"      # also "generic" (Unix, generic-serial feature) or "none"
rigctld_addr = "127.0.0.1:4532"
meter_poll_ms = 500          # ALC/power/SWR polling -> RigStatus events; 0 disables;
                             # uses a dedicated rigctld connection (never contends with PTT)

[logging]
level = "info"               # RUST_LOG still overrides
```

The device-resolution order in `[audio] device` is a genuine operator feature: a USB interface the OS renames or reorders (gains a `(2)` suffix, changes index) still resolves, because the matcher falls back from exact name to the stable ALSA `CARD=` token to a case-insensitive substring.

On first run, a 32-byte Ed25519 identity seed is generated at `~/.config/openpulse/identity.key`, created atomically with mode `0600`. A group- or world-readable key file is refused at load (`ConfigError::InsecureSecretPermissions`, REQ-SEC-CTL-05; test: `load_identity_refuses_group_readable_key`).

4.2.4 Calibrate before you key

`openpulse calibrate` has four subcommands; three are safe anywhere, one keys the transmitter:

```
openpulse calibrate audio    # measure input level and headroom to clip
openpulse calibrate ptt      # measure PTT assert/release latency vs the 50 ms target
openpulse calibrate afc      # measure AFC frequency offset (BPSK250 loopback burst)
openpulse calibrate drive    # HARDWARE: finds the TX attenuation that lands the rig's
                             # ALC in a moderate band. Requires a real radio + rigctld +
                             # the cpal-backend feature. KEYS THE TRANSMITTER —
                             # run into a dummy load.
```

Each prints a JSON verdict (`--output <path>` also writes it to a file). On the loopback backend the answers are trivial by construction — for example `{ "test": "ptt", "result": "pass", "latency_ms":`

`0.00036 }` (the exact figure varies per run and per machine) is the `NoOpPtt` controller, not a real radio. Run them again on the real backend and PTT hardware; the 50 ms PTT target is an acceptance criterion backed by `cargo test -p openpulse-radio`.

4.2.5 First transmit

Hardware required. This keys your transmitter. Set the rig to USB data mode, dial power down, and:

```
openpulse --backend cpal --ptt rigctld --rig 127.0.0.1:4532 \
  transmit "CQ CQ DE <YOURCALL>" --mode BPSK250 --fec rs --center-frequency 1500
```

`--center-frequency` defaults to 1500 Hz (the audio-passband centre); the CLI only calls `set_center_frequency()` when the value differs from 1500. `--max-power <W>` (global flag, default 100) feeds the regulatory TX-power cap. PTT release is guaranteed on TX error.

For receive-only monitoring:

```
openpulse --backend cpal receive --mode BPSK250 --listen-ms 120000
```

On a wired or direct-USB audio path where TX and RX share a clock, add `--no-afc`: the flag exists precisely because AFC can produce spurious corrections on near-zero-offset signals.

4.3 Scenario 3 — ARDOP TNC drop-in for Pat/Winlink

Hardware required for RF use (the TNC itself runs fine on loopback for protocol tests).

`openpulse-tnc` (crate `openpulse-ardop`) presents an ARDOP-compatible TCP interface — command port and data port — that Winlink clients such as Pat connect to as if it were ARDOP:

```
 Pat / Winlink client                openpulse-tnc                      radio
+---------------------+    TCP     +------------------+    audio/PTT   +-------+
|  ARDOP TNC driver   | ---------> | cmd  port  8515  | -------------> |  rig  |
|                     | ---------> | data port  8516  |                +-------+
+---------------------+            +------------------+
```

Build and run:

```
cargo build --release -p openpulse-ardop --features cpal
./target/release/openpulse-tnc --cmd-port 8515 --data-port 8516 \
  --mode QPSK500 --backend cpal
```

All flags are optional overrides of the `[ardop]` config section (unset = keep the config value). There are **no** `ARDOP_*` environment variables — port/mode/bind configuration is config-file plus flags only. The config section, with its real defaults:

```
[ardop]
bind_addr = "127.0.0.1"
cmd_port = 8515
data_port = 8516
# Opt-in: run an adaptive ARQ session so the rate ladder + host ARQBW/ARQTIMEOUT take effect.
# Default false = fixed-mode operation (ARQBW/ARQTIMEOUT are accepted-and-echoed no-ops).
enable_adaptive_arq = false
adaptive_profile = "hpx500"
```

That comment is worth reading twice: with `enable_adaptive_arq = false` (the default), the host commands `ARQBW` and `ARQTIMEOUT` are accepted and echoed **but inert**. Set it to `true` (and pick `adaptive_profile`, e.g. `hpx_hf`) to let the rate ladder drive the link.

The implemented host command set (`crates/openpulse-ardop/src/command.rs`): `VERSION` (replies `VERSION 1.0-OpenPulseHF`), `MYID`, `LISTEN`, `CONNECT`, `DISCONNECT`, `ABORT`, `STATE`, `BUFFER`, `PTT`, `GRIDSQUARE`, `ARQBW`, `ARQTIMEOUT`, `CWID`, `SENDID`, `FECSEND`, `FECRCV`, `WAVEFORM`, `PING` (→ `PONG`), `CLOSE`. Data-port framing is u16 big-endian length-prefixed binary, both directions.

On the message layer: OpenPulseHF's B2F implementation supports Winlink **Type D (Gzip) proposals only**. Type C (LZHUF) is not supported — the LH5 implementation was removed in v0.15.0 because it was never validated against a real RMS Express/RMS Gateway Type C blob, and an inbound Type C proposal is now answered `Reject` rather than risking a silent corrupt decode (`crates/openpulse-b2f/src/compress.rs`, `session.rs`; the `ProposalType::C` wire variant survives only so an inbound proposal can be parsed and rejected).

4.4 Scenario 4 — KISS/AX.25 TNC

Hardware required for RF use. `openpulse-kisstnc` (crate `openpulse-kiss`) is a TCP KISS TNC with AX.25 UI framing, for packet-style clients:

```
cargo build --release -p openpulse-kiss --features cpal
./target/release/openpulse-kisstnc --bind 127.0.0.1 --port 8100 \
  --mode BPSK250 --backend cpal
```

Config section and defaults:

```
[kiss]
bind_addr = "127.0.0.1"
port = 8100
```

The KISS codec implements full FEND/FESC/TFEND/TFESC byte stuffing; AX.25 UI frames carry callsign+SSID addressing (Control `0x03`, PID `0xF0`). One regulatory detail worth knowing: even though AX.25 frames carry their own source callsign, the TNC still records the *operator* identity and declared power in the regulatory TX log — `main.rs` calls `engine.set_callsign(cfg.station.callsign)` and `engine.set_max_power_watts(cfg.station.tx_power_watts)` at startup, so set `[station]` properly even for KISS use.

4.5 Scenario 5 — Winlink CMS gateway (no radio)

No radio required — this is a plain Internet TCP path, useful for testing your Winlink message flow end-to-end before RF, or as a fallback when no RF path exists.

`openpulse-gateway` speaks the B2F session protocol directly to a Winlink CMS over TCP:

```
cargo build --release -p openpulse-gateway --no-default-features
./target/release/openpulse-gateway --host cms.winlink.org --port 8772 \
  --callsign <YOURCALL> \
  send --to K5ABC --subject "Test" --message "Hello over OpenPulse"
```

- `--host` defaults to `cms.winlink.org`, `--port` to `8772`.
- `--callsign` overrides `[station] callsign` from config. Set a real callsign — the CMS authenticates you as a licensed station.
- Omit `--message` to read the body from stdin.
- The session runs both phases on one connection: ISS (your outbound proposals and compressed blobs), then IRS (the CMS's replies to you). Compression is Type D Gzip only, per §4.3.

4.6 Scenario 6 — two-station ARQ link and the twin-daemon rig

4.6.1 `openpulse arq` between two real stations

Hardware required (two stations) — or two hosts wired through the audio loopback rigs. The `arq` subcommand pair targets VOX or wired/full-duplex audio paths (keying is per transmission):

```
# Station B (receiver):
openpulse --backend cpal arq listen --mode BPSK250 --frames 1 --session openpulse-arq

# Station A (sender):
openpulse --backend cpal arq send --payload "hello" --mode BPSK250 --retries 3
```

Both sides accept `--profile <name>` to run the ARQ over an adaptive ladder instead of a fixed mode, and `-d/--device` to pin the audio device.

4.6.2 The two-station link simulator (no hardware)

`openpulse-linksim` simulates a *bidirectional* ARQ link — FSK4 ACK channel, turnaround time, retransmission and over-the-air rate adaptation — and reports the effective two-way transfer rate:

```
cargo run --release -p openpulse-linksim --no-default-features -- \
  --profile hpx_hf --channel awgn --snr 20 --frames 20 --payload 64
```

```
Two-station link — profile hpx_hf | 20 frames × 64 B | FEC Rs | turnaround 250 ms

      profile |    fwd channel | deliver |  effective | avg level |     final | air time
-------------------------------------------------------------------------------------------
       hpx_hf |      AWGN 20dB |    100% |   63.4 bps | avg SL 10.0 | final SL14 |  161.6 s
```

Again: 63.4 bps is the effective rate of a 20-frame run *including* the climb from SL2, not a steady-state figure. Useful flags: `--channel` (includes `qrm` with `--qrm-tones "freq:amp"` pairs), `--sweep start:stop:step` for SNR sweeps, `--seed` (default 49374), `--turnaround` (default 0.25 s), `--max-attempts` (default 6), `--json`, and `--no-cessb`, whose help string is the precise statement of CE-SSB scope: it "only affects QPSK-subcarrier OFDM — `OFDM16`/`OFDM52`. Dense OFDM-HOM and all SC-FDMA are excluded; see `ModemEngine::cessb_benefits`".

4.6.3 The twin-daemon rig — two real daemons, one command, no hardware

The highest-fidelity software rig in the repository is `scripts/demo-twin-panel.sh`: **entirely in software — no radios, no sound hardware, no sudo.** It launches:

1. the `twin_station` example — **two real `openpulse-server` daemons in one process**, bridged through a deterministic channel model. Both run the full on-air stack (RateAdapter, HpxReactor, OTA rate-stepping, QSY, repeater), so — quoting the script — "what you see is the true modem behaviour over a simulated channel, not a reimplemented simulator". Control ports: station A `127.0.0.1:9000`, station B `127.0.0.1:9002`;
2. `openpulse-twinview` — one GUI window, both stations, both directions (spectrum, waterfall, HPX state, rate ladder);
3. `scripts/twin-traffic.sh` — random A↔B messages so the panels light up.

```
./scripts/demo-twin-panel.sh               # defaults: SNR 20 dB, message every 3 s
TWIN_SNR_DB=8 INTERVAL=2 SIZE=128 ./scripts/demo-twin-panel.sh   # rougher channel
NO_VIEW=1 ./scripts/demo-twin-panel.sh     # headless: just the bridged link
```

Environment overrides: `TWIN_SNR_DB` (default `20`), `INTERVAL` (`3` s), `SIZE` (`64` B), `COUNT` (`0` = forever), `NO_TRAFFIC=1`, `NO_VIEW=1`. The build is CPU-only (`--no-default-features --example twin_station`), so it needs neither ALSA headers nor wgpu.

To step up to real audio with two independent clocks (still no radio): `scripts/setup-twin-loopback.sh` + `scripts/run-twin-station-audio.sh` run two real `openpulse-server` daemons over a bidirectional `snd-aloop` path, and `scripts/demo-hwloop-panel.sh` does the same over two physical USB soundcards joined by an analog cable — "no radios, no RF — but a real cpal+ALSA audio path with two independent clocks" (its default mode is `BPSK250`, chosen as "robust on the dual-clock rig").

4.7 Scenario 7 — mesh, relay and repeater

Three related but distinct capabilities:

Capability	Runs as	Enabled by
Mesh beacon re-broadcast	`openpulse-mesh` binary	`[mesh] enabled = true`
Multi-hop relay forwarding	inside the TNC/daemon receive loop	`[relay] enabled = true`
Cross-band repeater	inside `openpulse-server`	`[repeater]` + `openpulse daemon enable-repeater`

There is **no** `openpulse-repeater` binary — `crates/openpulse-repeater` is a library, and the repeater is driven through the daemon.

4.7.1 Mesh daemon

```
[mesh]
enabled = true            # default false; the daemon exits quietly if false
max_hops = 3
relay_policy = "balanced" # RESERVED for future trust-level filtering — currently a
                          # deny-list only is enforced; this value is not
store_forward_ttl_s = 300
beacon_interval_s = 60    # 0 disables
peer_cache_capacity = 256
peer_cache_ttl_s = 3600
```

```
cargo build --release -p openpulse-mesh --features cpal
./target/release/openpulse-mesh --mode BPSK250 --max-hops 3 --backend cpal
```

The mesh daemon hard-fails on `[station] callsign = "N0CALL"` (its source cites §97.119 as the rationale), and exits with `mesh is disabled in config; set [mesh] enabled = true to start` when disabled.

4.7.2 Relay forwarding

```
[relay]
enabled = false
max_hops = 3
store_forward_ttl_s = 300
deny_list = []    # originator peer IDs (lower-hex, 64 chars)
allow_list = []   # non-empty = forward ONLY these originators
```

The config doc-comment is candid about the allow-list's limits: "the originator id is not cryptographically authenticated at the relay, so this is a defense-in-depth control, not strong authentication".

4.7.3 Cross-band repeater

Hardware required (two rigs for cross-band).

```
[repeater]
enabled = false
mode = "BPSK250"     # same mode both directions
tx_hang_ms = 500     # ignored when full_duplex
full_duplex = false

[radio.rig_b]
rigctld_addr = "127.0.0.1:4533"   # the daemon reads ONLY this field of rig_b,
                                  # for the repeater's TX-side PTT
```

Then, with the daemon running:

```
openpulse daemon enable-repeater
openpulse daemon disable-repeater
```

(`[radio.rig_a]` exists in the template but is explicitly documented as "Currently unused" — the primary rig is the top-level `[radio]` section; `rig_a` is kept for a planned multi-rig refactor.)

4.8 Scenario 8 — QSY frequency agility

QSY lets two authenticated stations negotiate a coordinated frequency change (signed QSY_REQ/QSY_LIST frames, candidate scanning, a timed switchover). It is off by default and gated on trust:

```
[qsy]
enabled = false
allow_trustlevels = ["verified", "psk_verified"]
bandplan_mode = "ham-iaru-r1"          # also ham-iaru-r2 / ham-iaru-r3
bandplan_awareness_enabled = true
enforce_max_channel_width = true
enforce_segment_conventions = true
candidate_freqs_hz = []                # e.g. [7101000, 7103000, 7105000]
scan_dwell_ms = 500                    # S-meter dwell per candidate
switchover_offset_s = 5                # seconds between QSY_ACK and the switch
allow_integrated_tuner_on_high_swr = false
auto_qsy_on_interference = false       # requires [modem] notch_enabled +
                                       # notch_persistence > 0 + candidate_freqs_hz
```

CLI surface:

```
openpulse qsy init --rig 127.0.0.1:4532   # initialise QSY against the rig
openpulse qsy status                       # print the live config + compliance state
```

`qsy status` output includes the trust gate, bandplan mode and enforcement flags, candidate list, dwell and switchover values. If you have set `bandplan_awareness_enabled = false`, the status includes a line:

```
Compliance exception:  bandplan-awareness override is active (responsible operator required)
```

— that line only appears when the operator has overridden the bandplan guardrail; the default is `true`. In a live session, the daemon side exposes token-based consent:

```
openpulse daemon accept-qsy <TOKEN>
openpulse daemon reject-qsy <TOKEN>
```

4.9 Scenario 9 — JS8 discovery and rendezvous

Off by default, and deliberately so. The FF-15 feature set — a native JS8-compatible 8-GFSK waveform, heartbeat/`@OPULSE`-hint beaconing, and a two-message Propose/Accept rendezvous that hands off into an HPX connection on a working channel — is fully shipped in code (Phases A–G) but has **not** been validated on the air (Phase H remains open). The TX side transmits only with a configured callsign and the system clock within ±2 s of UTC, and the config doc-comment states plainly that "the operator is responsible for §97.221 automatic-control compliance (see `docs/regulatory.md`)".

```
[discovery]
enabled = false
mode = "rx_only"     # "rx_only" | "beacon" | "full"
submode = "normal"   # MVP: the only value
idle_grace_secs = 120
dwell_secs = 900     # 0 = dwell until preempted
heartbeat_interval_slots = 8   # x 15 s JS8 NORMAL slots
hint_interval_beacons = 3
station_ttl_secs = 3600
max_clock_skew_ms = 2000
group = "OPULSE"     # RESERVED — a non-default value has no effect and logs a
                     # startup warning
```

(`query_new_stations` and `max_queries_per_10min` also exist but are reserved: accepted-but-unused.)

Two frequency tables ship as defaults. `[discovery.calling_freqs_hz]` holds the JS8 calling frequency per band (160 m 1 842 000 Hz … 20 m 14 078 000 Hz … 10 m 28 078 000 Hz), and `[discovery.rendezvous_channels_hz]` holds three post-rendezvous working channels per band (e.g. 40 m: 7 101 000 / 7 103 000 / 7 105 000 Hz). A rendezvous `Propose` carries an *index* into the current band's channel list, not a frequency — "so both stations resolve the same Hz without spelling out frequencies on-air" — and the daemon bandplan-validates every entry at startup.

Operating it, with the daemon running:

```
openpulse daemon enable-discovery
openpulse daemon stations     # heard-station table; is_opulse flags OpenPulse peers
openpulse daemon peers        # the shared peer cache
openpulse daemon disable-discovery
```

Escalation path: `rx_only` (listen and build the station table) → `beacon` (add JS8 heartbeats + `@OPULSE` hints) → `full` (add rendezvous). RX decode performance is gated at −18 dB SNR by `cargo test -p js8-plugin --test snr_sweep gate_at_minus_18_db` — a simulator gate, like everything else in this feature.

4.10 Scenario 10 — file transfer

The `OPFX` direct P2P file-transfer protocol (FF-16, Phases A–E shipped; on-air Phase F deferred) moves files over an established HPX session with block-level resume, manifest verification and operator consent. Off by default:

```
[file_transfer]
enabled = false                # when false, inbound offers are rejected on air
                               # with "feature-disabled"
download_dir = "~/.local/share/openpulse/downloads"   # per-peer subdirs beneath
auto_accept_max_bytes = 0      # 0 = always ask the operator
max_file_bytes = 1048576       # 1 MiB, both directions
per_peer_quota_bytes = 0       # 0 = no limit
require_verified_peer = true   # signature-verified CONREQ/CONACK peer required
allowed_peers = []             # empty = any peer passing the trust policy
offer_timeout_secs = 120
partial_ttl_hours = 72         # partials at download_dir/<peer>/.partial/<sha256>/
burst_max_secs = 20.0          # max estimated on-air seconds per keyed TX burst
```

`burst_max_secs` is a real safety parameter: it splits a large transfer into airtime-bounded bursts "so a large transfer never holds PTT past the radio's watchdog and yields the channel between bursts. Keep it well under any PTT time-out (typically 180 s)".

Driving it, with the daemon running and a connected peer:

```
openpulse daemon send-file K5ABC /path/to/photo.jpg
openpulse daemon files             # print files received this session, as JSON
openpulse daemon accept-file <ID>  # operator consent for an inbound offer
openpulse daemon reject-file <ID>
openpulse daemon cancel-file <ID>
```

Interrupted transfers resume from `.partial` state at the block level. Behavioural gates: `cargo test -p openpulse-filexfer` (protocol edges: offer/accept/reject/timeout/cancel/ verify/tamper), `cargo test -p openpulse-modem --test filexfer_loopback` (blocks survive the modem), and `cargo test -p openpulse-daemon --test twin_daemon_bridge a_file_crosses` (a file crosses two real daemons).

4.11 Scenario 11 — operator panel, TUI, testbench

4.11.1 The daemon these frontends talk to

`openpulse-server` (crate `openpulse-daemon` — note the binary name differs from the crate name) takes **no CLI flags at all**: it is entirely config-file driven. It refuses to start until `[station] callsign` is changed from `N0CALL`:

```
[daemon]
tcp_bind_addr = "127.0.0.1"
tcp_port = 9000              # NDJSON control protocol (one JSON object per line)
websocket_bind_addr = "127.0.0.1"
websocket_port = 9001
receive_tick_ms = 50         # lower = more responsive QSY, higher CPU

[control_security]
require_auth = false         # auth is ALWAYS required on a non-loopback bind
                             # regardless; this forces it on loopback too
psk_key_id = "control-psk"
```

```
cargo build --release -p openpulse-daemon --features cpal
./target/release/openpulse-server
```

When auth is required, the 32-byte control PSK is currently supplied via the `OPENPULSE_CONTROL_PSK` environment variable (64 hex characters) on both the daemon and the panel; with auth required and no PSK set, the daemon refuses to start (fail closed). Every `openpulse daemon <sub>` command talks to `--addr 127.0.0.1:9000` by default over that NDJSON TCP protocol. There are 41 of them (link, PTT, QSY, messaging, repeater, discovery, file transfer, spectrum, config, OTA-ladder and RX/TX-tuning groups).

4.11.2 The iced operator panel

```
cargo build --release -p openpulse-panel
./target/release/openpulse-panel     # no CLI args; enter the daemon address in the UI
```

Defaults inside the UI: control `127.0.0.1:9000` (TCP) and `ws://127.0.0.1:9001` (WebSocket). Tabs: Info, Stats, Files, Config, Messages (the default tab), Discovery, Log. Band selection, spectrum/waterfall, the rate ladder, and Dark/Light/Contrast/System themes.

4.11.3 The terminal dashboard

```
cargo build --release -p openpulse-tui --features cpal-backend   # or without, for loopback
./target/release/openpulse-tui --mode BPSK250
```

Note the TUI's `--backend` default is `loopback` (unlike the CLI's `default`), and its `--log` default is `warn`. Three panels: colour-coded HPX state, AFC/rate meters with a DCD energy bar, and a scrollable transitions log. Keys: `q`/Ctrl+C quit, `p` pause, arrows scroll.

4.11.4 The signal-path testbench

```
cargo build --release -p openpulse-testbench                  # synthetic source
cargo build --release -p openpulse-testbench --features cpal   # + live audio capture
./target/release/openpulse-testbench
```

Four live columns — TX (clean), noise channel, mixed, RX (decoded) — each with an FFT spectrum and a waterfall, over the eight selectable noise/channel models returned by `NoiseModel::all()` (AWGN, Gilbert-Elliott, Watterson, flat fading, QRN, QRM, QSB, chirp), with an SNR slider and FEC toggle. With the `cpal` feature it can also capture the default system input live at 8 kHz.

(Caveat: this chapter's GUI descriptions are drawn from source; the GUI binaries were not run during verification of this book.)

4.12 Scenario 12 — diagnostics, session metrics, audit bundles, benchmarking

4.12.1 Live event stream

```
openpulse --backend cpal monitor --mode BPSK250
```

Streams NDJSON `EngineEvent`s to stdout — session starts, rate changes, DCD changes, AFC updates — one JSON object per line, flushed per event. Pipe it into `jq` for live filtering.

4.12.2 Session diagnostics

```
openpulse session state
openpulse session list
openpulse session log --follow
openpulse session-metrics --format json
openpulse diagnose handshake --peer K5ABC
openpulse diagnose session  --peer K5ABC
openpulse trust show <STATION_ID>
openpulse trust explain <STATION_ID>
openpulse trust revoke --station-or-key <STATION_OR_KEY>
```

Every `session` / `identity` / `trust` / `diagnose` / `session-metrics` subcommand shares the same diagnostic option block: `--format <text|...>` (default `text`), `--verbose`, `--diagnostics`, `--no-color`, `--timeout <s>` (default `5`).

Note `trust revoke` takes a **named** `--station-or-key` flag, not a positional argument (the current `docs/cli-guide.md` shows a positional; the binary rejects it).

4.12.3 Audit bundles

Enable audit recording in the daemon first:

```
[observability]
audit_mode = true                                  # default false; records the
                                                   # control-event stream (REQ-OBS-01)
archive_dir = "~/.local/share/openpulse/audit"     # events.ndjson lands here

[logging]
level = "info"
file = "~/.local/share/openpulse/openpulse.log"    # also appends to a daily-rolled
                                                   # <path>.YYYY-MM-DD (REQ-OBS-02)
```

Then package the evidence:

```
openpulse audit-bundle --label "field-day-2026"
# --archive-dir defaults to [observability] archive_dir
# --output      defaults to <archive_dir>/bundles
```

`audit-bundle` requires that the daemon has actually run with `audit_mode = true` — there is nothing to bundle otherwise.

For QSO records rather than engineering evidence, the ADIF logbook is separate and also opt-in:

```
[logbook]
enabled = false                                    # one ADIF record per connect->disconnect
adif_path = "~/.local/share/openpulse/openpulse.adi"
[logbook.peer_grids]                               # callsign -> Maidenhead, fills
"K5ABC" = "EM12"                                   # GRIDSQUARE when not exchanged on air
```

4.12.4 The test matrix and benchmark harness

```
# Quick tier (virtual channels, no hardware):
cargo run -p openpulse-testmatrix --no-default-features

# Full matrix (all propagation channels and payload sizes):
cargo run -p openpulse-testmatrix --no-default-features -- --full --output docs/test-reports

# Matrix, then the throughput benchmark:
cargo run -p openpulse-testmatrix --no-default-features -- --bench --bench-frames 50 --bench-payload 223

# Throughput benchmark only (skips the matrix pass):
cargo run -p openpulse-testmatrix --no-default-features -- --bench-only --bench-frames 50 --bench-payload
        223
```

`--bench-payload` defaults to 128; its help documents a 223-byte ceiling — the RS(255,223) single-block limit — but note that the ceiling is documentation, not a runtime check: the binary does not clamp or reject a larger value. Reports land in `<output>/latest/`. One verified inconsistency to be aware of: the binary's `--output` default is `docs/test-reports`, but the `scripts/run-test-matrix.sh` wrapper archives from `docs/dev/test-reports/latest/` — the wrapper only archives correctly when you pass `--output docs/dev/test-reports`.

The HPX conformance benchmark and its `jq` gate are in §4.1.5. The other locally-run regression gates worth knowing by name: the linksim goodput gate (`cargo test -p openpulse-linksim goodput_gate` — `hpx_hf` AWGN-20 dB effective throughput floor 250 bps against a ~397 bps baseline, 200-byte payload, 40 frames, seed 5) and the fade-survival gate (`cargo test -p openpulse-modem --test hpx_hf_rungs_survive_fade`). Both are Watterson-simulator gates.

4.13 Troubleshooting

All symptoms below are real strings or verified behaviours from the 0.15.0 tree.

Symptom	Cause	Fix
`note: cpal backend not compiled in; falling back to loopback` (CLI stderr)	CLI built with `--no-default-features`	rebuild: `cargo build --release -p openpulse-cli` (its audio feature `cpal-backend` is on by default)
TNC/daemon transmits but nothing reaches the radio; log shows `cpal backend not compiled in (build with --features cpal); using loopback`	binary built without its `cpal` feature, config says `backend = "cpal"`	rebuild with the crate's feature: `--features cpal` (ardop/kiss/daemon/mesh/testbench) or `--features cpal-backend` (tui)
Same as above but **no warning at all**	binary built without `cpal`, config says `backend = "default"` — this fallback is silent	same rebuild; prefer `backend = "cpal"` in config so at least a warning fires
`Error: unknown backend 'X'` (CLI) / `unknown audio backend 'X' — use 'default', 'cpal', or 'loopback'` (TNC binaries)	typo in `--backend`	valid values: `loopback`, `default`, `cpal`
daemon exits immediately: `invalid callsign N0CALL in configuration; set [station].callsign before starting daemon`	callsign never set	edit `[station] callsign` — the daemon and mesh daemon both refuse to start on `N0CALL`
`openpulse-mesh` exits: `mesh is disabled in config; set [mesh] enabled = true to start`	mesh off (default)	set `[mesh] enabled = true`
daemon refuses to start with control auth required and no key	fail-closed PSK gate	export `OPENPULSE_CONTROL_PSK` (64 hex chars = 32 bytes) for daemon *and* panel
identity key refused at load (`InsecureSecretPermissions`)	`~/.config/openpulse/identity.key` group/world-readable	`chmod 600 ~/.config/openpulse/identity.key`
two-terminal `transmit` → `receive` with `--backend loopback` decodes nothing	loopback audio does not persist across process invocations	use the snd-aloop rig: `scripts/setup-virtual-loopback.sh` + `scripts/run-loopback-virtual.sh`
`run-loopback-virtual.sh`: `ERROR: cpal CLI not found at ...`	loopback rig needs a CPAL CLI build	`cargo build --release -p openpulse-cli`
spurious AFC corrections on a wired/direct-USB audio path	AFC misbehaves on near-zero-offset signals	pass `--no-afc` to `receive`
Pat sends `ARQBW`/`ARQTIMEOUT`, TNC echoes them, nothing changes	`[ardop] enable_adaptive_arq = false` (default): those hints are accepted-and-echoed no-ops	set `enable_adaptive_arq = true` and an `adaptive_profile`
`openpulse beacon ... --max-hops 2` → `error: unexpected argument '--max-hops' found`	doc drift in `docs/cli-guide.md`; the flag is `--ttl`	`openpulse beacon --callsign <CALL> --ttl 2`
`openpulse trust revoke FOO` → `error: unexpected argument 'FOO' found`	doc drift; the argument is a named flag	`openpulse trust revoke --station-or-key FOO`
CPAL build fails on Linux with ALSA errors	missing ALSA headers	`sudo apt-get install libasound2-dev`
audio device "not found" after replug/reboot	device renamed/reordered by the OS	leave `[audio] device` as a stable substring or the ALSA `CARD=` token — resolution is exact name → `CARD=` token → case-insensitive substring
`qsy status` shows `Compliance exception: bandplan-awareness override is active`	you set `bandplan_awareness_enabled = false` (default is `true`)	intentional override — restore `true` unless you accept responsible-operator duty
inbound file offer rejected on air with `feature-disabled`	`[file_transfer] enabled = false` (default)	set `enabled = true`; offers still require operator `accept-file` unless `auto_accept_max_bytes > 0`
Winlink peer proposes Type C and the session rejects it	Type C (LZHUF) is not supported in 0.15.0 — inbound Type C proposals are answered `Reject` by design	none needed; OpenPulseHF↔OpenPulseHF and Type-D-capable peers use Gzip
`run-test-matrix.sh` archives nothing	binary default `--output docs/test-reports` vs wrapper reading `docs/dev/test-reports/latest/`	pass `--output docs/dev/test-reports`

4.14 Where to go next

- The mode and FEC ladder in engineering depth: `docs/mode-fec-ladder.md` — the `hpx_hf` rung table there is enforced against `SessionProfile::hpx_hf` by `cargo test -p openpulse-core --test ladder_doc_matches_profile`.
- The on-air validation ladder and its scripts: `docs/dev/virtual-loopback.md`, `scripts/onair-preflight.sh`, `scripts/run-onair-validation-flow.sh` (preflight → matrix run → evidence bundle → report scaffold). **Hardware required**, and operator responsibility applies throughout.
- Regulatory posture (station ID, §97.221 considerations for discovery beaconing): `docs/regulatory.md`.

Provenance and honest limits

How this book was produced. Each chapter was researched directly from the repository at v0.15.0, drafted, and then fact-checked by an independent pass that re-derived every checkable assertion — file paths, symbol names, CLI flags, config keys, mode strings, test names, and numeric measurements — against the source, correcting what did not hold. Fifty-three corrections were applied that way. The commands in Chapter 4 were executed, not merely written.

That process is worth describing because this project has been bitten by the opposite. A documentation audit in July 2026 found the README advertising Winlink Type C wire-compatibility one hundred and forty-eight lines below the release note retracting that exact claim, and found two acceptance tests named for "loopback correctness" that never called the receive path at all. A book is a worse place for that failure than a README, because nobody greps a book against the source.

What remains unverified. Three figures quoted in Chapter 2A trace only to the project's own engineering notes rather than to a test or a source comment: a correlation-window energy value in the normalised-acquisition discussion, one endpoint of a sync-fix decode figure, and a CPU-cost ratio for the convolutional codec. They are cited as recorded rather than re-measured. Chapter 1's test-suite total is likewise taken as reported.

What is simulated, not flown. Every fading result — the differential-QPSK rescue, the re-seated `hpx_hf` ladder, the evidence-based climb, the OFDM rungs — comes from the Watterson channel simulator. The project's own history shows why that matters: in v0.14.1 the link simulator was found unable to transmit its own sub-floor rung, which had made a working link read as total failure, and every fade measurement taken before that fix was suspect. On-air validation is the first gate in `docs/dev/project/release-1.0-criteria.md` and it has not been passed.

What is off by default. JS8 discovery and rendezvous, direct file transfer, the mesh and repeater roles, QSY frequency agility, and audit mode all ship disabled. Chapter 4 marks each one.

Where to go next. `docs/openpulse-manual.md` is the operator reference and goes deeper on day-to-day commands. `docs/mode-fec-ladder.md` is the authoritative rung table, gated against the code. `docs/dev/design/protocol-wire-spec.md` is the wire format. `CLAUDE.md` carries the engineering contract and the "known sharp edges" that much of Chapter 2A elaborates.